SetupVibe

SETUPVIBE

Guia do usuário

Ambiente de desenvolvimento com versões fixadas, em um comando

Instalação, lista de itens, TUI, shell, servidor com segurança, agentes por MCP, migração do SetupVibe 1 e a referência de CLI, configuração, exit codes e MCP.

Edição v2.0.0-dev.25

Gerado em 13 de novembro de 2026

Fonte editorial: docs/user/

Sumário

SetupVibe 7

- Um binário, três modos 7
- Duas edições, dois binários 7
- Versões fixadas, não versões do dia 8
- Como ler este guia 8
- A máquina de exemplo 9
- O que este guia não ensina 10
- Documentação relacionada 10
- Próximo passo 10

Visão geral 11

- As peças 11
- O ciclo de uso 11
- Versões fixadas 12
- Duas edições 12
- O que o binário guarda 13
- Para quem este guia escreve 13
- Próximo passo 13

Instalação e primeiro uso 14

- O que a máquina precisa 14
- 1. Instale o binário 15
- 2. Abra a interface e leia o próximo passo 16
- 3. Veja o que a lista mostra 16
- 4. Prepare a máquina 18
- 5. Instale os primeiros itens 20
- 6. Ou deixe o `auto` fazer tudo 21
- 7. Confira o resultado 21
- O que mudou na máquina 22
- Se algo der errado 23
- Próximo passo 23

Instalação automática 24

- O que o `auto` faz 24
- Perfis, itens e dependências 25
- Respostas sem argumento 25

Veja o plano antes de executar 26
Acompanhe a execução 26
Repita a seleção em outra máquina 27
Provisionamento 27
Proxy, CA e cache 29
Próximo passo 29
Itens e versões fixadas 30
O que a versão de um item significa 30
Instalar 31
Remover 31
Atualizar 32
Conferir 32
Ferramenta fora da lista 32
Flags que mudam o comportamento por máquina 33
Próximo passo 33
A TUI 34
Início 34
Cabeçalho, rodapé e idioma 35
Teclas 35
Inventário 35
Doctor e Logs 36
Tailscale e Máquina 37
Tema, ícones e mouse 37
Abrir num módulo e em terminal estreito 37
Próximo passo 37
Shell e terminal 38
O que o `shell apply` escreve 38
O bloco e o arquivo local 39
Aliases 39
Ver o que está aplicado 39
Remover sem perder o resto 40
Omarchy 40
Próximo passo 40
Manutenção 41
Diagnóstico 41

Atualizar o binário 41
Relatório para uma issue 42
Logs e o último resultado 42
Repetir uma seleção em outra máquina 43
Remover o SetupVibe 43
Chave SSH num servidor 43
Skills de agente 43
Documentação embutida 44
Rotina 44
Próximo passo 44
Servidor e segurança 45
Instalar a edição server 45
Usuário de trabalho 45
Módulo de segurança 46
O fluxo de um VPS novo 46
Portainer e portas publicadas 47
Acesso remoto 47
Conferir o servidor 48
Se algo der errado 48
Próximo passo 48
Agentes de IA e MCP 49
O que o servidor expõe 49
Resources 50
O prompt `operate` 51
Listar antes de conectar 51
Configurar o cliente 51
Um agente em ação 52
Skill do próprio binário 52
Limites 52
Próximo passo 53
Windows e WSL 54
Instalar no Windows 54
O que entra no Windows 54
O WSL 55
O binário dentro do WSL 55

Diagnóstico no Windows 56
Se algo der errado 56
Próximo passo 56
Vindo do SetupVibe 1 57
Como a máquina antiga é reconhecida 57
O que acontece sem migrar 57
O plano do `migrate` 58
Executar 58
O que conferir depois 59
Se algo der errado 59
Próximo passo 60
Problemas comuns e glossário 61
Comece pelo `doctor` 61
Sintomas e saídas 61
Sudo sem senha 62
Falha parcial 62
Ferramenta de outro gerenciador 62
Onde ler mais 63
Glossário 63
Fechamento 64
Referência 65
Apêndice A. CLI 66
Convenções 66
Options globais 66
Saída em texto e em JSON 67
Como ler as seções de comando 68
Commands 68
Apêndice B. Configuração e estado 137
Arquivos em `~/.setupvibe/` 137
`settings.json` 139
`credentials.json` 142
O que a cifra protege 143
Rotação da master key 144
Usar uma credencial no código 144
Variáveis de ambiente 144

Rede e cache 147
Apêndice C. Códigos de saída 148
Por command 151
Em shell 152
No `--json` 153
No código 153
Apêndice D. MCP 154
Como o servidor responde 154
Configuração no cliente 154
Listar sem iniciar 155
Tools 155
Resources 171
Prompts 174
Configurar nos agentes 175
Como adicionar uma tool 175
Testar 176

SetupVibe

Uma máquina de desenvolvimento se degrada por acúmulo. Cada pessoa instala o runtime de que precisa na versão do dia, acrescenta um CLI para resolver um problema da semana e ajusta o `.zshrc` até o prompt ficar aceitável. Seis meses depois, duas máquinas da mesma equipe rodam versões diferentes de Node e ninguém sabe qual linha do shell veio do instalador e qual veio de um teste.

O SetupVibe 2 trata esse preparo como parte do projeto. Um binário único lê uma lista de itens com versões fixadas, instala o que você marcar, registra o que fez em `~/.setupvibe/` e confere depois se a máquina continua com o que foi combinado. A mesma abordagem atende um notebook e um servidor, e a lista que o binário segue vem embutida nele, sem serviço externo.

Um binário, três modos

O mesmo executável atende três públicos, sem três bases de código.

MODO	COMO ENTRAR	PARA QUEM
TUI	`setupvibe` sem argumentos em um terminal interativo	pessoas que querem ver o estado e agir pelo teclado
CLI	`setupvibe <command> [options]`, com saída em texto ou `--json`	pessoas, scripts e agentes
MCP	`setupvibe mcp`, servidor Model Context Protocol via stdio	agentes de IA que operam a máquina

Os três leem a mesma identidade, a mesma configuração, o mesmo cofre de credenciais e a mesma tabela de exit codes. Um comando novo entra no mesmo registro e chega ao `--help`, ao `--json`, à descrição OpenCLI e à referência gerada, sem uma segunda implementação.

Duas edições, dois binários

BINÁRIO	EDIÇÃO	SISTEMAS
`setupvibe`	desktop: ambiente de desenvolvimento, local ou remoto	macOS, Ubuntu, Debian, Mint, Zorin, Arch, Omarchy, Windows e WSL
`setupvibe-server`	server: infraestrutura com Docker Swarm	Ubuntu e Debian

As duas edições usam a mesma pasta `~/.setupvibe/` e o mesmo prefixo `SETUPVIBE_` nas variáveis. Elas são exclusivas: o `init` recusa uma máquina que já tem a outra edição preparada, porque só um binário fica instalado por máquina.

Versões fixadas, não versões do dia

Cada item da lista carrega a versão, ou a política de versão, que o SetupVibe instala. `setupvibe check` compara o que está na máquina com essa versão e devolve o exit code correspondente. `setupvibe update` troca só os itens cuja versão fixada mudou desde o binário que você tem. Ferramentas fora da lista entram pelo mise com `install --external`, também com versão fixada.

Como ler este guia

O guia tem três partes. Se é a sua primeira vez com o SetupVibe, leia a Parte 1 na ordem e siga a Parte 2 como consulta operacional. A Parte 3 é referência, para quando você precisa da lista exata de uma opção, de um arquivo de estado ou de um exit code.

Parte 1. Primeiro uso

O produto, o binário no lugar, o preparo da máquina, a instalação sem perguntas e o controle do que entra e sai.

CAPÍTULO	PERGUNTA QUE RESPONDE
Visão geral	De que peças o SetupVibe é feito e o que cada uma guarda?
Instalação e primeiro uso	Como instalar o binário, preparar a máquina e instalar os primeiros itens?
Instalação automática	Como preparar uma máquina sem perguntas, com perfil, respostas e plano?
Itens e versões fixadas	O que a lista controla e como instalar, remover, atualizar e conferir?

Parte 2. Operação

O uso que continua depois da instalação: interface, shell, manutenção, servidor, agentes de IA, Windows, migração e falhas.

CAPÍTULO	O QUE VOCÊ APRENDE
A TUI	os sete módulos, as teclas, o mouse, o idioma e o tema
Shell e terminal	o que muda no `.zshrc`, no Starship e no tmux, e onde ficam as suas linhas
Manutenção	`doctor`, `upgrade`, `report`, `config export` e os logs
Servidor e segurança	`setupvibe-server`, usuário de trabalho, UFW, fail2ban e Swarm

CAPÍTULO	O QUE VOCÊ APRENDE
Agentes de IA e MCP	como um agente lê o estado e executa o SetupVibe por MCP
Windows e WSL	o installer do Windows, o WSL 2 e o que o binário preserva
Vindo do SetupVibe 1	o que o `migrate` converte, o que ele pergunta e o que conferir
Problemas comuns e glossário	sintoma, causa, comando e exit code, termo por termo

Parte 3. Referência

Os apêndices são gerados na compilação a partir da documentação embutida no binário, que outros agentes mantêm. No repositório, a coluna da direita leva à fonte.

APÊNDICE	CONTEÚDO	FONTE
CLI	convenções, opções globais, saída em texto e JSON, referência de cada command	desktop/docs/cli.md
Configuração e estado	`settings.json`, arquivos de estado, cofre, master key e variáveis	desktop/docs/configuration.md
Códigos de saída	a tabela completa e como tratar cada classe de falha	desktop/docs/exit-codes.md
MCP	tools, resources, prompts e configuração nos clientes	desktop/docs/mcp.md

A máquina de exemplo

Os comandos do guia vêm de um host Debian 13 ×64, com 20 GB de RAM e 74 GB livres, que já tinha passado pelo SetupVibe 1. Isso permite mostrar o mesmo comando em dois estados: com o item ausente e com o item já instalado na máquina por outro gerenciador.

Os blocos `text` foram copiados de execuções reais do binário com dois ajustes: a pasta pessoal aparece como `~` e as listas longas aparecem cortadas com `…`. Números de memória, de disco e de versão mudam de máquina para máquina, e cada bloco diz de qual máquina veio.

Commands, opções, keys e campos do `--json` ficam em inglês, como o binário escreve. O texto em volta é em português do Brasil.

O que este guia não ensina

O guia não substitui a documentação do mise, do Docker Swarm, do Model Context Protocol nem do Git. Ele mostra o que o SetupVibe faz com essas ferramentas e aponta a fonte oficial quando o assunto fica fora do escopo do produto. Também não ensina a especificação interna: o código de `shared/` e o documento da especificação ficam no repositório, para quem mantém o SetupVibe.

Documentação relacionada

- `setupvibe --skill` imprime a documentação embutida no binário, escrita para agentes lerem sem o repositório. `setupvibe docs` aceita os mesmos tópicos e mais `inventory`, gerado da lista embutida.
- Repositório: código, releases, changelog e os installers.
- Site: apresentação, edições e downloads.

Próximo passo

A visão geral mostra as peças e o vocabulário que o resto do guia usa. O capítulo seguinte instala o binário e prepara a primeira máquina.

Visão geral

Preparar uma máquina de desenvolvimento envolve quatro trabalhos diferentes: instalar pacotes do sistema, escolher versões de runtime, configurar o shell e registrar o que foi feito. Quando cada trabalho fica num script separado, o resultado depende de quem rodou por último e de qual versão estava disponível naquele dia.

O SetupVibe 2 junta os quatro num binário com uma lista única. A lista diz o que entra e em qual versão; o binário executa, grava o que fez e confere depois. A diferença aparece no segundo mês: a máquina continua com as versões combinadas e o histórico de execução mostra o que mudou.

As peças

PEÇA	ONDE VIVE	O QUE FAZ
binário	`~/.local/bin/setupvibe` ou `/usr/local/bin/setupvibe`	executa os commands, abre a TUI e serve o MCP
lista de itens	`desktop/inventory.json` e `server/inventory.json`, embutidos no binário	define cada item, a versão fixada, as dependências, o método de instalação e a verificação
installers	`install.sh`, `install-server.sh` e `install.ps1`	baixam o binário do release e conferem o checksum
estado	`~/.setupvibe/`	guarda a seleção, o resultado das execuções, as preferências e o cofre
documentação embutida	dentro do binário	responde `--skill` e `docs` sem o repositório

A lista viaja dentro do binário. Isso resolve um problema prático de provisionamento: a máquina que executa não precisa de acesso ao repositório para saber o que instalar. O binário e a lista têm a mesma versão, então uma máquina preparada com `2.0.0-dev.25` instala exatamente o que aquele binário conhece.

O ciclo de uso

O trabalho segue uma ordem com seis passos, e cada um pode ser repetido sem desfazer o anterior.

1. **detectar**: `setupvibe system` mostra o sistema, a variação da máquina (local, remota ou sem interface), o suporte e o estado da versão.

2. **preparar**: `setupvibe init` confere os requisitos e instala as dependências de compilação e o mise.
3. **escolher**: `setupvibe list` mostra os itens e o estado de cada um; a TUI permite marcar e instalar pelo teclado.
4. **instalar**: `setupvibe install <itens>` instala a seleção explícita; `setupvibe auto` instala sem perguntas.
5. **conferir**: `setupvibe check` compara a máquina com a versão fixada; `setupvibe doctor` olha o conjunto.
6. **manter**: `setupvibe update` troca versões, `setupvibe upgrade` atualiza o binário e `setupvibe remove` tira o que você não quer manter.

Nada nesse ciclo depende de um serviço do SetupVibe estar no ar. O binário consulta a versão nova uma vez e avisa; a instalação e a conferência são locais, com downloads dos fornecedores de cada ferramenta.

Versões fixadas

Cada item carrega uma política de versão. Um pacote nativo costuma ficar numa linha (`php@8.5`), uma CLI que se atualiza sozinha fica com versão mínima e uma ferramenta de release fixo fica na versão exata. O `setupvibe check` aplica a mesma política na conferência, e o `setupvibe update` troca só o item cuja versão fixada mudou entre o binário que você tem e o novo.

Isso não congela as dependências do seu projeto. O SetupVibe cuida das ferramentas da máquina; o `composer.lock`, o `package-lock.json` e o `mise.toml` do projeto continuam sendo a fonte das versões do projeto.

Duas edições

EDIÇÃO	BINÁRIO	ESCOPO
desktop	`setupvibe`	ambiente de desenvolvimento em macOS, Linux, Windows e WSL, local ou por acesso remoto
server	`setupvibe-server`	máquina Linux de operação, com Docker, Swarm, usuário de trabalho e módulo de segurança

As duas compartilham a pasta `~/.setupvibe/`, o prefixo `SETUPVIBE_` das variáveis e a tabela de exit codes. São exclusivas por máquina: o `init` recusa uma máquina com a outra edição preparada, e o `self uninstall` remove a instalação antes da troca.

O que o binário guarda

Três arquivos explicam quase todo comportamento que surpreende numa primeira sessão:

- `config.json` guarda a edição, o sistema detectado, o registro do `init`, a identidade do Git e o canal de release. É estado, não preferência.
- `selection.json` guarda os itens escolhidos, a versão instalada e o status do último `check`. É dele que `list`, `check` e `update` partem.
- `settings.json` guarda preferências em texto claro, com keys em notação de ponto, e é o único feito para você mudar por comando.

O apêndice de configuração lista todos os arquivos, quem escreve cada um e o que acontece quando o formato muda.

Para quem este guia escreve

Para quem mantém a máquina, não para quem mantém o SetupVibe. Você precisa saber usar terminal, `sudo`, variáveis de ambiente e Git. Não precisa saber Bun, mise, Docker Swarm nem Model Context Protocol: o guia mostra o que o produto faz com cada um e onde está a documentação oficial quando o assunto passa do escopo.

O guia não ensina a especificação interna nem o código de `shared/`. Quem mantém o produto encontra essa parte no repositório e em `V2.md`.

Próximo passo

O capítulo Instalação e primeiro uso instala o binário, prepara a máquina com `init` e instala o primeiro conjunto de itens.

Instalação e primeiro uso

Uma máquina nova não está pronta para trabalhar. Falta compilador, `git`, runtime, CLI, container e um shell que carregue tudo no lugar certo. Feito à mão, esse preparo vira uma sequência de comandos que ninguém revisa e que muda quando alguém instala a versão mais nova de um runtime.

O SetupVibe 2 separa o preparo em duas etapas que você pode repetir. A primeira instala o binário: um arquivo com versão do release e checksum conferido. A segunda, `setupvibe init`, prepara a máquina com as dependências de compilação e o mise. Só depois disso os itens entram, um a um ou pelo `auto`.

Neste capítulo você percorre as duas etapas num host Linux, olha a lista de itens antes e depois, instala o primeiro conjunto e confere o resultado. O Windows e o servidor têm capítulos próprios; aqui o caminho é o de uma máquina de trabalho com terminal.

O que a máquina precisa

O binário recusa um sistema fora da lista e avisa quando a versão é anterior à mínima. A checagem acontece antes de qualquer alteração, em todo comando que mexe na máquina.

SISTEMA	VERSÃO MÍNIMA	ARQUITETURAS
Ubuntu	24.04	x64, arm64
Debian	12	x64, arm64
Mint	22	x64
Zorin	18	x64
Arch	rolling	x64
Omarchy	4	x64
macOS	12	x64, arm64
Windows	build 22621	x64
WSL	WSL 2	x64, arm64

Além do sistema, o `init` confere cinco coisas e explica cada uma quando falha:

- **disco**: 10 GB livres na pasta pessoal na edição desktop, 5 GB na server. O número cobre as cadeias de ferramentas que o mise mantém e os caches de download.
- **memória**: abaixo de 2 GB o `init` avisa que a compilação pode ficar sem memória e sugere `setupvibe install swap`.

- **locale UTF-8**: sem ele, ferramentas que tratam nomes de arquivo com acento, como `git` e `mise`, podem gravar o nome errado.
- **rede e relógio**: o `init` compara o relógio com o de um servidor HTTPS. Mais de cinco minutos de diferença derrubam a validação de TLS e de checksum.
- **usuário**: o `init` não roda como root. Numa máquina remota, crie o usuário de trabalho antes, como o capítulo de servidor mostra.

1. Instale o binário

O installer lê a versão do canal em `version.json`, baixa o binário do release correspondente ao sistema e confere o `SHA256SUMS` antes de gravar.

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/
setupvibe/v2/install.sh | sh
```

Num Linux x64 com `cosign` instalado, a saída termina assim:

```
Downloading setupvibe 2.0.0-dev.25 for linux-x64...
Signature of SHA256SUMS verified with cosign.
Installed ~/.local/bin/setupvibe (2.0.0-dev.25).
```

A linha da assinatura aparece só quando o `cosign` existe na máquina. Nesse caso, o installer confere a assinatura Sigstore do `SHA256SUMS` contra o workflow de release do repositório e aborta quando ela não corresponde. Sem `cosign`, a conferência fica no checksum.

O destino é `~/.local/bin` para quem não é root e `/usr/local/bin` para root. Quando a pasta não está no `PATH`, o installer avisa em vez de editar o seu shell. Confira a versão:

```
setupvibe --version
```

```
setupvibe 2.0.0-dev.25
```

Para instalar a edição server, escolha a edição na linha de comando. As duas são exclusivas por máquina.

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/
setupvibe/v2/install.sh | sh -s -- --edition server
```

O canal `beta` existe para testar uma versão antes da estável e usa a mesma verificação:

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install.sh | SETUPVIBE_CHANNEL=beta sh
```

No Windows, o installer é outro e grava em `%LOCALAPPDATA%\Programs\SetupVibe`. O capítulo Windows e WSL mostra a linha completa, o `PATH` do usuário e o que o binário não altera no PowerShell.

2. Abra a interface e leia o próximo passo

Num terminal de verdade, o binário sem argumentos abre a TUI no **Início**. Sete cartões resumem os módulos: Inventário, Usuário, Doctor, Logs, Tailscale, Máquina e Sobre. Acima deles, o próprio produto indica o passo que resolve o estado atual da máquina. Numa instalação nova, o passo é concluir o `init`.

As teclas que bastam na primeira sessão:

TECLA	AÇÃO
`1` a `7`	abre o módulo do número
`←` `→` `↑` `↓`	move entre os cartões do Início
`Enter`	abre o cartão selecionado
`Esc`	volta ao Início; no Início, sai
`?`	mostra a lista completa de teclas
`q`	sai

Num pipe, sem terminal interativo, o binário imprime a ajuda e sai com exit code 2. O comando explícito `setupvibe tui` sem TTY sai com 8, `NO_TTY`. Isso mantém um agente ou um script longe de uma tela que nunca vai responder.

O resto do capítulo segue pelo CLI. Cada passo fica registrado num comando que você repete numa máquina diferente, e a TUI executa os mesmos comandos em outro processo.

3. Veja o que a lista mostra

`setupvibe list` imprime os itens por categoria, com a versão fixada e o estado na máquina. Este é o começo da saída desta máquina, que já tinha ferramentas instaladas por Homebrew, `rbenv` e `npm` global:

```
System

  ○ build-deps              distro        init
  ○ mise                    2026.11.6     init
  ◖ sshpass                 distro        other (/usr/bin/sshpass)
  ○ wsl                     latest        unavailable

Languages

  ◖ node                    24.22.3       other (/home/linuxbrew/.linuxbrew/bin/node)
  ◖ python                  3.14.10       other (/usr/bin/python)
  ○ uv                      0.13.7        available
  ◖ php                     8.5           other (/usr/bin/php)
  ◖ bun                     1.4.4         other (~/.bun/bin/bun)
  …
```

A coluna do meio é a versão que o binário instala, não a versão que está na máquina. Numa máquina limpa, quase todas as linhas aparecem como `available`. O estado explica o que fazer a seguir:

ESTADO	LIST	TUI	SIGNIFICADO
`installed`	●	●	selecionado e conferido com a versão fixada
`outdated`	●	↑	selecionado com uma versão diferente da que este binário fixa
`failed`	●	✗	o último `check` do item não passou
`other`	◖	◖	o comando existe fora das pastas do mise e de `~/.local/bin`, e a linha mostra o caminho
`available`	○	○	instala neste sistema e ainda não foi instalado
`unavailable`	○	·	não tem entrada de instalação para este sistema
`system`	○	◆	já vem com o sistema, como no macOS e no Omarchy

ESTADO	LIST	TUI	SIGNIFICADO
`init`	○	◇	entra pelo `init`, não pela seleção
`gui`	○	○	item gráfico, deixado de fora numa máquina sem interface

O `list` usa ● para os três estados de item selecionado e ○ para todo o resto, o que basta quando você só quer saber se o item entrou. A TUI separa os casos com símbolos próprios, e é nela que a diferença entre instalado, desatualizado e com falha aparece na primeira olhada.

`other` não é erro. A ferramenta está na máquina, instalada por outro gerenciador, e o SetupVibe instala a própria cópia pelo método que a lista define para aquele item. Ele não sobrescreve a cópia que veio de APT, Homebrew, npm ou do instalador do fornecedor. As duas convivem, e o `PATH` define qual responde primeiro.

Para ver só uma categoria, filtre pelo id. Para rodar a verificação de cada item selecionado junto com a lista, use `--check`:

```
setupvibe list --category languages
setupvibe list --check
```

4. Prepare a máquina

O `init` altera pacotes do sistema e grava o primeiro estado. Rode com `--dry-run` e leia o plano antes:

```
setupvibe init --dry-run
```

As primeiras linhas desta máquina, com a lista de pacotes do passo 2 cortada:

```
Requirements

✓ user: not root
✓ edition: desktop
✓ disk: 74.0 GB free (minimum 10 GB)
✓ memory: 20.5 GB
✓ locale: en_US.UTF-8
✓ network: github.com reachable
✓ clock: 8 s from the server

init (dry run)

1. Refresh the package index
   $ sudo apt-get -o DPkg::Lock::Timeout=600 update
2. Install build-essential, git, curl, wget, unzip, xz-utils, file, procps, ca-
certificates, gnupg, libssl-dev, … with the system package manager
   $ sudo apt-get install -y -o DPkg::Lock::Timeout=600 …
3. Install mise 2026.11.6 to ~/.local/bin/mise (SHA-256 from the release)
   → download https://github.com/jdx/mise/releases/download/v2026.11.6/mise-
v2026.11.6-linux-x64, verify it with https://github.com/jdx/mise/releases/download/
v2026.11.6/SHASUMS256.txt and write ~/.local/bin/mise
…
```

O plano lista cada comando com o argumento completo e, quando existe, a condição que faz o passo ser pulado numa segunda execução. O `mise` é baixado do release oficial e conferido contra o `SHASUMS256.txt` do mesmo release.

Quando a checagem falha, o `init` para antes de alterar qualquer coisa e devolve o exit code da classe:

FALHA	EXIT CODE	O QUE FAZER
sistema ou versão fora da lista	11 `UNSUPPORTED_SYSTEM`	confira a tabela do começo do capítulo
execução como root	12 `ROOT`	rode como o seu usuário; no servidor, crie o usuário antes
menos espaço que o mínimo	14 `DISK_SPACE`	libere espaço na pasta pessoal
rede ou relógio	5 `NETWORK_OR_API`	confira a conexão e ative o NTP

Com o plano conferido, execute sem `--dry-run`:

```
setupvibe init
```

O `init` instala as dependências de compilação e o mise, grava o registro em `~/.setupvibe/config.json` e para. Ele não instala o resto da lista. Numa máquina sem identidade Git configurada, ele pergunta o nome e o e-mail uma vez, grava no Git e no estado do SetupVibe. Numa máquina que já tem ferramentas, ele registra o que passa na verificação como `adopted` em `~/.setupvibe/selection.json`, sem instalar nem remover nada. A saída informa quantos itens foram adotados e lista os ids.

Itens adotados não saem num `remove` sem `--force`, porque não foi o SetupVibe que os instalou. Se um passo falhar, o `init` continua os outros e termina com 10 `PARTIAL`; o log da execução mostra cada resultado.

5. Instale os primeiros itens

Comece pelo ambiente do terminal, que é onde o resto do trabalho acontece: `zsh`, `starship` e `tmux`. O `--dry-run` mostra as dependências puxadas junto e a ordem:

```
setupvibe install zsh starship tmux --dry-run
```

O plano inclui os requisitos de cada item e a configuração do shell quando a seleção tem `zsh`, `tmux`, `tpm`, `starship` ou `oh-my-zsh`. Passos que já estão prontos aparecem como pulados. Sem `--dry-run`, a execução instala e confere cada item antes de gravar a seleção:

```
setupvibe install zsh starship tmux
```

O `install` grava em `~/.setupvibe/selection.json` o item e a versão instalada. Se o item já passa na verificação com a versão fixada, ele é pulado e o motivo aparece na saída; `--force` reinstala. Quando existe outra cópia da ferramenta na máquina, o comando avisa o caminho antes de instalar a cópia do SetupVibe, que fica em pasta própria e não substitui a outra.

Se você pular o `init`, o comando recusa e aponta o que falta:

```
✗ init has not been completed. Run: setupvibe init
```

Essa recusa sai com exit code 3, `CONFIG`. É o mesmo código que um `settings.json` inválido devolve, por isso leia a mensagem além do número.

Para instalar uma ferramenta que não está na lista, use `--external`. A versão fica fixada no registro em `~/.setupvibe/external.json`:

```
setupvibe install --external terraform@1.13.4
```

6. Ou deixe o `auto` fazer tudo

Quando a máquina precisa ficar igual a outra, ou quando não há ninguém para responder, o `auto` percorre a instalação inteira sem perguntas. Ele lê as respostas de opções, variáveis `SETUPVIBE_*` ou de um arquivo `--config`, instala os itens, configura o shell e aplica o módulo de segurança quando cabe.

```
setupvibe auto --dry-run
```

O `--dry-run` mostra o plano completo sem alterar nada, e o `--plan arquivo.json` grava o mesmo plano para revisão. O capítulo Instalação automática mostra os perfis, o arquivo de respostas, a retomada depois de uma falha e o uso em provisionamento.

SITUAÇÃO	CAMINHO
máquina pessoal, você quer escolher item por item	`init` e depois `install`
máquina de equipe, VPS ou CI, que precisa ficar igual à outra	`auto` com `--profile` ou `--config`
quer revisar antes de executar	`auto --dry-run` ou `auto --plan`

7. Confira o resultado

`setupvibe check` roda a verificação de cada item selecionado e compara com a versão fixada. Sem itens na seleção, ele confere o perfil `essencial`. O comando não altera nada; ele responde com exit code 0 quando tudo confere e 10 quando algum item falha. Com `--json`, a mesma informação chega no envelope que um script lê.

```
setupvibe check
setupvibe check zsh tmux --json
```

`setupvibe doctor` olha a máquina inteira: runtime, estado do SetupVibe, pré-requisitos pendentes, itens e restos do SetupVibe 1. Trecho da saída desta máquina, que veio do SetupVibe 1 com a pasta de estado em modo 775:

```
Runtime

✓ bun: 1.4.4; minimum 1.4.4
! stdin: Not a TTY
! stdout: Not a TTY
✗ directory: Mode 775, expected 700; doctor --fix sets it
! settings: Not created
! master-key: none
! credentials: Not created
✓ binary: setupvibe 2.0.0-dev.25
✓ platform: linux-x64
```

`✓` é aprovado, `!` é aviso e `✗` é problema. O estado do SetupVibe deve ficar numa pasta `0700`, porque ela guarda o cofre; o `doctor --fix` corrige o modo e as demais correções seguras (pastas e bloco do shell), e deixa o resto para você decidir. Sem `--fix`, o comando não aplica correção alguma.

O que mudou na máquina

CAMINHO	O QUE É
`~/.local/bin/setupvibe` ou `/usr/local/bin/setupvibe`	o binário instalado pelo installer
`~/.local/bin/mise`	o gerenciador de versões instalado pelo `init`
`~/.setupvibe/config.json`	edição, sistema detectado, registro do `init` e identidade do Git
`~/.setupvibe/selection.json`	itens escolhidos, versão instalada e status do último `check`
`~/.setupvibe/settings.json`	preferências, criado na primeira gravação
`~/.setupvibe/last-run.json`	resultado completo do último `auto`, quando você usa esse caminho
`~/.setupvibe/logs/`	um log por execução, com um evento por linha
`~/.zshrc` e `~/.tmux.conf`	bloco entre `# >>> setupvibe >>>` e `# <<< setupvibe <<<` (no Omarchy, `~/.bashrc`)
`~/.zshrc.local` e `~/.tmux.conf.local`	o lugar das suas linhas, fora do bloco
`~/.config/mise/config.toml`	versões globais do mise escolhidas pelos itens

A pasta `~/.setupvibe/` é criada com permissão `0700` e os arquivos que o binário grava ficam com `0600`. O `shell apply` guarda uma cópia do arquivo original em `~/.setupvibe/backups/` antes da primeira alteração.

Se algo der errado

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
`setupvibe: command not found`	`~/.local/bin` fora do `PATH`	abra um terminal novo ou acrescente a pasta ao `PATH`
exit code 3 com `init has not been completed`	comando de item antes do `init`	rode `setupvibe init`
exit code 12 `ROOT`	execução como root	rode como o seu usuário; no servidor, use `setupvibe user create`
exit code 14 `DISK_SPACE`	menos de 10 GB livres na desktop	libere espaço e repita
exit code 20 `REBOOT_PENDING`	o sistema pede reinício	reinicie ou use `auto --reboot`
exit code 10 `PARTIAL`	um passo falhou e os outros seguiram	leia `~/.setupvibe/logs/` e rode `setupvibe doctor`
installer recusa a assinatura	`SHA256SUMS` não corresponde ao workflow	não use o arquivo baixado; baixe de novo e confira o release

Próximo passo

A máquina já tem o binário, os pré-requisitos e os primeiros itens. O capítulo Instalação automática mostra como transformar esse mesmo caminho num provisionamento repetível, com perfil, arquivo de respostas e retomada depois de uma falha.

Instalação automática

Preparar uma máquina à mão funciona uma vez. Na terceira vez, você já não lembra qual item foi marcado no notebook, qual opção foi passada no servidor e por que aquele runtime ficou numa versão diferente. O `auto` existe para esse caso: ele percorre a instalação inteira sem perguntar nada e transforma o que foi escolhido num passo que outro computador, um pipeline ou um agente pode repetir.

A diferença em relação ao capítulo anterior é o ponto de partida. O `init` mais `install` serve para uma máquina que você opera e quer entender passo a passo. O `auto` serve para uma máquina que precisa ficar igual a outra, ou que não tem ninguém para responder.

O que o `auto` faz

O comando encadeia as etapas na ordem abaixo, e uma falha não interrompe as seguintes.

ETAPA	O QUE ACONTECE
fase root	com `--user`, cria o usuário de trabalho, instala a chave SSH e continua como ele
fase do usuário	confere `sudo -n`, disco, memória, locale, rede e relógio
seleção	monta os itens do perfil, aplica `--only` e `--exclude` e puxa as dependências
plano	inclui o `init`, cada item, o shell, o swap, o Portainer, a segurança, o Git, o Tailscale e o Swarm quando cabe
execução	segura `~/.setupvibe/run.lock`, grava o run log e segue com os passos restantes depois de uma falha
conferência	roda a verificação de cada item selecionado e grava o resultado em `~/.setupvibe/last-run.json`
reinício	com reinício pendente, sai com 20 ou agenda a retomada com `--reboot`

Como root, `--user` é obrigatório: o `auto` cria o usuário, copia o binário compilado para `/usr/local/bin` e se executa de novo com `runuser`. A regra de sudo sem senha que ele cria no começo é removida no fim, salvo `--keep-nopasswd`.

```
setupvibe auto --user deploy --ssh-key ~/.ssh/id_ed25519.pub
```

Perfis, itens e dependências

Sem `--profile`, entra todo item compatível com o sistema. Itens gráficos ficam de fora numa máquina sem interface, e itens que já vêm no sistema, como no macOS e no Omarchy, são pulados.

Na edição desktop, o perfil `essencial` repete a seleção padrão da TUI, e o `remoto` estende o essencial com `linger`, `mosh`, `tailscale`, `openssh-server`, `swap`, `ufw`, `fail2ban`, `unattended-upgrades` e `time-sync`. Na edição server existe o `essencial`, voltado à operação.

```
setupvibe auto --profile essencial --dry-run
setupvibe auto --profile remoto --exclude docker
```

`--only` acrescenta itens e `--exclude` tira. Nenhum dos dois substitui dependências: se o item excluído é exigido por outro da seleção, ele entra mesmo assim. Para ver os perfis e os itens de cada categoria antes de escolher, use a documentação gerada da lista:

```
setupvibe docs inventory
```

O tópico `inventory` imprime cada perfil e uma tabela por categoria, com versão, política, sistemas e flags de cada item.

Respostas sem argumento

Cada resposta vem de uma option, depois de uma variável de ambiente e por último de um arquivo `--config`. Tokens de Tailscale e de Swarm entram apenas por variável de ambiente, que o run log mascara.

RESPOSTA	OPTION	VARIÁVEL
usuário de trabalho	`--user`	`SETUPVIBE_USER`
chave pública	`--ssh-key`	`SETUPVIBE_SSH_KEY`
identidade do Git	`--git-name`, `--git-email`	`SETUPVIBE_GIT_NAME`, `SETUPVIBE_GIT_EMAIL`
perfil	`--profile`	`SETUPVIBE_PROFILE`
ajustes na seleção	`--only`, `--exclude`	`SETUPVIBE_ONLY`, `SETUPVIBE_EXCLUDE`
fuso	`--timezone`	`SETUPVIBE_TIMEZONE`
Tailscale	`--tailscale-authkey`	`TAILSCALE_AUTHKEY`

RESPOSTA	OPTION	VARIÁVEL
Swarm	`--manager`, `--join`, `--advertise-addr`	`SETUPVIBE_SWARM_*`
reinício e retomada	`--reboot`	`SETUPVIBE_REBOOT`

O restante das opções está no apêndice de CLI, gerado da documentação embutida. A tabela acima cobre o que muda entre máquinas.

Veja o plano antes de executar

Duas opções mostram o mesmo plano sem alterar a máquina. `--dry-run` imprime em texto, e `--plan` grava JSON para comparar duas máquinas.

```
setupvibe auto --dry-run
setupvibe auto --plan /tmp/plano.json
```

O plano nomeia cada item, a versão fixada, os passos de sistema e o que foi pulado. Numa máquina já preparada, a segunda execução mostra quase tudo como pulado, porque cada passo carrega a condição que o dispensa.

Acompanhe a execução

Em texto, cada etapa sai com `✓`, `-` ou `✗`, e o fechamento lista o que falhou e o que foi pulado, inclusive partes de segurança. Com `--json`, cada evento entra no stdout como uma linha NDJSON antes do envelope final, o que permite acompanhar um provisionamento longo sem esperar o fim.

O resultado completo fica em `~/.setupvibe/last-run.json`, com item, versão, status e a mensagem de cada falha. O log da execução fica em `~/.setupvibe/logs/`, um evento por linha.

SITUAÇÃO NO FIM	EXIT CODE	PRÓXIMA AÇÃO
tudo instalado e conferido	0	abrir um shell novo
parte dos itens falhou	10 `PARTIAL`	ler o `last-run.json` e o log
reinício pendente	20 `REBOOT_PENDING`	reiniciar ou repetir com `--reboot`
outra execução em andamento	30 `LOCKED`	esperar o PID de `run.lock`
sudo pede senha sem terminal	13 `SUDO_PASSWORD`	criar a regra com `user nopasswd` ou rodar no terminal

Com `--reboot`, o Linux instala uma unit systemd de execução única que roda o `auto` de novo depois do reinício. No Windows, o comando cria a tarefa `SetupVibeAutoResume`, que roda com privilégio máximo e se apaga no fim.

Repita a seleção em outra máquina

`config export` grava a seleção e a identidade do Git num arquivo sem segredo. `config import` instala o que está nele. O arquivo é o mesmo que o `auto` aceita em `--config`.

```
setupvibe config export selecao.json
setupvibe config import selecao.json
```

O arquivo exportado grava `profile: "none"`, para não inventar um perfil que não existe. O `auto --config` lê esse valor como ausência de perfil: valem a lista `only` do arquivo ou o inventory inteiro. Com `--profile`, valem o perfil indicado e a lista `only`, não a seleção exportada sozinha.

Provisionamento

Os exemplos abaixo baixam o binário com o installer e entregam o resto ao `auto`. Nenhum segredo aparece na linha de comando.

cloud-init, num VPS que começa como root:

```
#cloud-config
runcmd:
  - >-
    curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install.sh
    | sh -s -- auto --user deploy
    --ssh-key "ssh-ed25519 AAAA... deploy@laptop"
    --timezone America/Sao_Paulo --profile remoto
```

Ansible, com a chave do Tailscale vindo do vault:

```
- name: SetupVibe nas máquinas de desenvolvimento
  hosts: dev
  become: false
  environment:
    TAILSCALE_AUTHKEY: "{{ vault_tailscale_authkey }}"
  tasks:
    - name: Instalar e executar o auto
      ansible.builtin.shell: >-
        curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/
install.sh
        | sh -s -- auto --profile remoto
      args:
        creates: "{{ ansible_env.HOME }}/.setupvibe/last-run.json"
```

O `creates` evita repetir a execução a cada rodada. Remova a linha quando quiser que o `auto` rode de novo, porque ele é idempotente.

Dockerfile, com um recorte de itens:

```
FROM debian:13
RUN apt-get update && apt-get install -y --no-install-recommends curl ca-
certificates sudo locales \
 && sed -i 's/^# *en_US.UTF-8/en_US.UTF-8/' /etc/locale.gen && locale-gen
ENV LANG=en_US.UTF-8
RUN useradd -m -s /bin/bash dev && echo 'dev ALL=(ALL) NOPASSWD:ALL' > /etc/
sudoers.d/dev
USER dev
RUN curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install.sh
\
    | sh -s -- auto --profile essencial --exclude docker --skip-network-check
```

Serviços que dependem de systemd, como Docker, servidor SSH e o módulo de segurança, não se aplicam a uma imagem comum. Para esses, parta de uma imagem com systemd.

GitHub Actions, com `--json` para as etapas seguintes lerem os eventos:

```
jobs:
  setup:
    runs-on: ubuntu-24.04
    steps:
      - name: SetupVibe
        env:
          GITHUB_TOKEN: ${{ github.token }}
        run: |
          curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/
install.sh \
            | sh -s -- auto --only node,python,uv,gh --profile essencial --json
```

O `GITHUB_TOKEN` mantém o mise abaixo do limite anônimo da API do GitHub.

Proxy, CA e cache

`HTTPS_PROXY`, `HTTP_PROXY` e `NO_PROXY` valem também para os comandos que rodam com sudo. Em rede corporativa, `--ca-file` acrescenta o bundle de CA para Node, Bun, curl, Git e ferramentas Python. `--cache-dir` guarda os downloads do mise, npm, uv, Homebrew e do próprio SetupVibe numa pasta que várias máquinas podem compartilhar; o que já está no cache instala sem rede.

Numa máquina que fica sempre atrás da mesma CA ou do mesmo cache, grave os dois em `~/.setupvibe/config.json` para não repetir a option:

```
{ "network": { "caFile": "/etc/ssl/certs/corp.pem", "cacheDir": "/srv/setupvibe-
cache" } }
```

A ordem de leitura é option, variável de ambiente e esse bloco.

Próximo passo

A máquina ficou preparada sem uma única pergunta. O capítulo Itens e versões fixadas mostra o que fazer depois: ver o que entrou, remover o que não serve, atualizar quando o binário muda e conferir item por item.

Itens e versões fixadas

O SetupVibe não instala o que estiver na versão mais nova do repositório de cada fornecedor. Ele instala o que está na lista embutida no binário, com a versão que aquela lista fixa, e confere depois. A lista é o arquivo `desktop/inventory.json` na edição desktop e `server/inventory.json` na server, compilado dentro do executável.

Cada item dessa lista declara o id, o nome, a categoria, a versão, a política de versão, as dependências, o método de instalação por sistema e o comando que verifica o resultado. Quando você instala `node`, o binário não procura a última versão: ele usa a entrada de `node` na lista, instala pelo método daquele sistema e confere com o comando de verificação que a entrada declara.

O que a versão de um item significa

A política de versão decide como o item é instalado e conferido.

POLÍTICA	COMO FUNCIONA	EXEMPLO
`exact`	versão fixada, instalada e conferida exatamente	Node 24.22.3
`series`	pacote nativo preso em `major.minor`	PHP 8.5
`minimum`	ferramenta que se atualiza sozinha; a verificação exige o piso	Claude Code
`commit`	fonte Git presa a um commit	ferramenta instalada por clone
`repository`	última estável de um repositório preso por impressão digital da chave	pacote de repositório APT
`distro`	pacote que vem do sistema	`build-essential`
`managed`	o instalador do fornecedor escolhe a versão	instalador oficial
`system`	configuração sem versão	bloco de configuração do WSL

O `list` mostra a versão fixada, não a versão que está instalada. Para comparar com a máquina, use `check`.

Instalar

`setupvibe install <itens>` instala os itens citados e as dependências deles, na ordem que a lista define. Um item que já passa na verificação é pulado, e o motivo aparece na saída.

```
setupvibe install docker
```

Numa máquina que vai receber dois runtimes e as ferramentas de terminal, o `--dry-run` mostra as dependências que entram junto e o que já está pronto:

```
setupvibe install node php bat eza jq --dry-run
```

O plano mostra o método de cada item. Em Debian e Ubuntu, um pacote nativo passa pelo APT com a chave do repositório conferida contra a impressão digital da lista; uma ferramenta de versão fixa passa pelo mise; um release binário é baixado, conferido contra o SHA-256 e extraído. A ordem de resolução do método é o sistema, a família (`debian-family` ou `arch-family`), `linux`, `unix` e `all`.

Quando o item já está instalado por outro gerenciador, o comando avisa o caminho e instala a cópia do SetupVibe numa pasta própria. As duas convivem, e o `PATH` decide qual responde. Para substituir a instalação existente por completo, remova a outra cópia depois de conferir a versão que ficou ativa.

`--force` reinstala um item que já passa na verificação. É o caminho para reparar uma instalação quebrada sem mexer no resto da seleção:

```
setupvibe install node --force
```

Remover

`remove` tira o que foi instalado no espaço do usuário e atualiza a seleção. Pacote do sistema só sai com `--include-native`, e item adotado pelo `init` só sai com `--force`.

```
setupvibe remove go
setupvibe remove docker --include-native --dry-run
```

Antes de remover, o comando verifica se outro item selecionado depende daquele. Se depende, ele recusa e explica; `--force` remove mesmo assim. Sem terminal, a confirmação precisa vir de `--yes`. A resposta negativa ou um `Ctrl+C` termina com 130, sem remover nada.

O que o SetupVibe não instalou fica. Item que veio do sistema, item do módulo de segurança e item marcado como `initOnly` não são removidos por `remove`, e o motivo aparece no texto. Para desfazer a preparação inteira, existe `self uninstall`, no capítulo de manutenção.

Atualizar

`update` não consulta a internet. Ele compara a seleção com a lista do binário que está rodando e reinstala só os itens cuja versão fixada mudou. Isso amarra a atualização à versão do binário: binário antigo conhece versões antigas.

```
setupvibe update --dry-run
```

O plano lista cada item com a versão de origem e a de destino. Depois de `setupvibe upgrade`, o `update` seguinte aplica o que a versão nova do binário trouxe:

```
setupvibe upgrade
setupvibe update
```

Item instalado com `--external` não entra no `update`; ele é atualizado pela própria ferramenta ou por um novo `install --external`.

Conferir

`check` roda o comando de verificação de cada item selecionado. A saída é uma linha por item, com `✓` ou `✗`, a saída do comando e a versão esperada.

```
setupvibe check
setupvibe check node php --json
```

Sem argumentos, ele usa a seleção. Sem seleção, usa o perfil `essencial`. Itens `initOnly`, do módulo de segurança e sem instalação possível neste sistema não entram na verificação. O exit code é 0 quando tudo confere e 10 quando algum item falha, o que permite usar o comando num pipeline sem interpretar texto:

```
setupvibe check --json | jq '.data.results[] | select(.status=="fail")'
setupvibe check node || echo "rode: setupvibe install node"
```

`list --check` junta as duas leituras: a lista completa, com o resultado da verificação ao lado de cada item selecionado. É útil para ver o que está fora da versão sem rodar `check` duas vezes.

Ferramenta fora da lista

Uma ferramenta que não está na lista entra pelo mise, com a versão resolvida na hora e gravada em `~/.setupvibe/external.json`. Sem versão no argumento, o binário pergunta ao mise a última e fixa o que ele devolveu.

```
setupvibe install --external terraform
setupvibe install --external terraform@1.13.4
```

Uma ferramenta que o mise não conhece termina com 6, `NOT_FOUND`. Para remover, use o mesmo caminho:

```
setupvibe remove --external terraform
```

Flags que mudam o comportamento por máquina

Três flags da lista explicam diferenças que aparecem na prática:

- `gui`: o item depende de interface gráfica e fica de fora em máquina sem display, como um VPS ou um contêiner.
- `service`: precisa de systemd ou de `linger` para rodar sem sessão aberta.
- `port`: publica uma porta. Nas variantes remotas, o `auto` grava a entrada de compose com a porta presa a `127.0.0.1`, para o serviço não ficar exposto.

Uma flag pode valer só para um sistema. O `list` e a tabela do tópico `inventory` mostram a diferença por linha.

Próximo passo

A lista está conferida e as versões estão no lugar. O capítulo A TUI mostra como operar a mesma lista pela interface, com filtro, seleção e progresso ao vivo.

A TUI

A interface de terminal existe para responder a três perguntas sem digitar comando: em que estado a máquina está, o que falta, e o que acontece enquanto uma instalação roda. Ela não substitui o CLI. Cada ação da TUI executa o mesmo comando em outro processo, e o resultado aparece no mesmo `~/.setupvibe/` que o CLI lê.

Abra com `setupvibe` sem argumentos, num terminal interativo:

```
setupvibe
```

Fora de um terminal, o binário imprime a ajuda e sai com 2. O comando explícito `setupvibe tui` sem TTY sai com 8, `NO_TTY`.

Início

A TUI não tem abas. Ela abre no **Início**, um painel com sete cartões, um por módulo. Acima dos cartões fica o próximo passo recomendado: migrar uma máquina do SetupVibe 1, concluir o `init`, corrigir item com falha, atualizar versões ou instalar o que falta. Um clique no passo abre o módulo onde ele se resolve.

CARTÃO	O QUE RESUME
Inventário	barra de progresso, itens por status e uma barra por categoria
Usuário	usuário, pendências da checklist e estado do sudo
Doctor	problemas, avisos, estado do `init` e os primeiros achados
Logs	última execução, eventos com falha e as execuções anteriores
Tailscale	estado, IP, tailnet e máquinas online, a cada 15 segundos
Máquina	CPU, RAM, disco, carga, rede e histórico de CPU ao vivo
Sobre	versão, edição, sistema detectado e aviso de versão nova

`u` relê o estado depois de uma mudança feita no CLI, e `1` a `7` abrem o módulo do número correspondente, de qualquer lugar.

Cabeçalho, rodapé e idioma

A primeira linha é uma faixa com o selo, a versão, o módulo atual, o sistema detectado, o seletor `PT · EN` e um relógio. O rodapé mostra as teclas do módulo aberto e, depois de um separador, as globais. `?` abre a ajuda com as duas listas, e `q` sai. Quando a largura não comporta todas as teclas, as do fim saem; `?` e `q` ficam sempre.

`L` alterna o idioma e grava em `tui.language`. `T` alterna entre o tema escuro (`night`) e o claro (`day`), grava em `tui.theme` e pode ser acionado pelo ícone de lua ou sol no cabeçalho. Com `SETUPVIBE_LANG` definida, o idioma fica fixo nas próximas execuções.

Teclas

TECLA	AÇÃO
`1` a `7`	abre o módulo do número
`←` `→` `↑` `↓`	move entre os cartões do Início
`Enter` ou `espaço`	abre o cartão selecionado
`Esc` ou `Backspace`	volta ao Início; no Início, sai
`L`	troca o idioma
`T`	troca o tema
`u`	relê o estado
`?`	mostra ou esconde a ajuda
`q` ou `Ctrl+C`	sai

Dentro de um módulo, `Esc` e o botão ← **Voltar** retornam ao Início. O rodapé é clicável: cada tecla exibida também funciona como botão.

Inventário

O módulo abre com as categorias à esquerda e as ferramentas à direita. Cada categoria mostra `instalados/total` e uma barra; cada ferramenta mostra nome, status e versão, com a borda colorida quando o estado é instalado, desatualizado ou falhou.

TECLA	AÇÃO
`c` e `f`	levam o foco às categorias e às ferramentas
`Tab`	alterna o foco entre os dois painéis

TECLA	AÇÃO
`j` `k` ou `↓` `↑`	move no painel com foco; nas categorias, troca de categoria
`h` `l`, `[` `]`, `←` `→`	troca de categoria
`g` e `End`	vão ao primeiro e ao último item
`espaço`	marca ou desmarca o item
`a`	marca todas as ferramentas que ainda podem ser instaladas; de novo, desmarca
`s`	alterna o filtro: todos, para instalar, instalados, com problema
`i`	instala os itens marcados, ou o item atual quando nada está marcado
`r`	remove os itens marcados, ou o item atual

O filtro aparece no título do painel, com a contagem de itens visíveis sobre o total. `s` percorre os quatro estados e volta ao primeiro.

Instalar e remover usam duas etapas. A primeira monta o plano com `--dry-run` e mostra os passos na tela. A segunda executa; qualquer outra tecla cancela. Essa divisão evita que um clique instale algo que você não leu.

Durante a execução, o painel de detalhes vira o progresso ao vivo: barra, passos concluídos sobre o total, segundos decorridos, os últimos passos com `✓`, `✗` ou `↷` e a mensagem de cada falha. O conteúdo vem do log da execução, não de uma contagem paralela. No fim, a borda fica verde ou vermelha e qualquer tecla volta aos detalhes.

Doctor e Logs

O módulo Doctor agrupa os mesmos achados do `setupvibe doctor`: problemas, avisos e itens aprovados, cada um com o comando que resolve. `f` aplica as correções seguras (pastas, modo da pasta de estado e bloco do shell) e deixa as demais para você decidir; um `[f]` ao lado do comando indica que a correção é segura. `u` refaz a checagem.

O módulo Logs lista as execuções de `~/.setupvibe/logs`, da mais nova para a mais antiga, com os eventos de cada uma: tempo, status, item e ação. A mensagem de erro aparece abaixo de cada falha. Sem execução, o painel explica o que vai aparecer ali.

Tailscale e Máquina

O módulo Tailscale lê o cliente local, sem chamar a API. Mostra esta máquina, serviços publicados com `serve` e Funnel, avisos de saúde, teste de rede com `n` (UDP, IPv4, IPv6, UPnP, NAT-PMP, PCP, NAT difícil, portal cativo e latência por relay) e as máquinas da tailnet com sistema, IP, conexão direta ou por relay e último contato. `u` recarrega.

O módulo Máquina atualiza a cada segundo: sistema, kernel, virtualização, tempo ligado, usuário e shell; CPU com histórico de 60 segundos, carga e uma barra por núcleo; RAM e swap; rede por interface; discos com uso, leitura e escrita; e os processos que mais consomem CPU. `u` força uma leitura nova.

Tema, ícones e mouse

Duas paletas vivem em `shared/core/tui/theme.ts`: `night`, o padrão, e `day`. As cores têm nome de papel (`accent`, `textDim`, `border`), então a troca de paleta não muda as telas. A escolha fica em `tui.theme`.

Os ícones vêm da Nerd Font, instalada pelo SetupVibe. Num terminal sem a fonte, desligue com `setupvibe config set tui.icons false` ou `SETUPVIBE_ICONS=0`: a interface passa a usar símbolos que toda fonte tem.

Com `tui.mouse` ligado, que é o padrão, clique abre cartão, módulo, categoria, item, marcador, máquina da tailnet, execução e botão; a roda rola as listas. Para selecionar texto no terminal, segure `Shift` ao arrastar. O teclado continua completo.

Abrir num módulo e em terminal estreito

A TUI abre no Início, ou no módulo de `tui.initialTab`. A option `--tab` tem prioridade:

```
setupvibe --tab inventory
setupvibe config set tui.initialTab logs
```

Os valores são `overview`, `inventory`, `user`, `doctor`, `logs`, `tailscale`, `machine` e `about`. O nome `tab` ficou do modelo antigo por compatibilidade: `tui.initialTab` e `--tab` continuam com esse nome, mas a interface não tem abas.

Em terminal com menos de 110 colunas ou 32 linhas, o Inventário estreita as colunas, encurta o painel de detalhes e reduz os espaços. As listas rolam quando não cabem.

Próximo passo

A interface mostra o estado, mas quem grava configuração é o CLI. O capítulo Shell e terminal mostra os blocos que o SetupVibe escreve no `.zshrc`, no Starship e no tmux, e onde ficam as suas linhas.

Shell e terminal

Um ambiente preparado se perde no primeiro `git pull` do próprio dotfiles ou no primeiro instalador que reescreve o `.zshrc`. O SetupVibe evita essa disputa com um bloco delimitado: ele escreve só entre dois marcadores, guarda uma cópia do arquivo antes da primeira mudança e deixa um arquivo separado para as suas linhas.

Quem aplica esse bloco é o `auto`, ao fim da instalação, e o `shell apply`, quando você quer refazer. Os dois usam o mesmo código e convertem um arquivo do SetupVibe 1 quando encontram um, então o bloco novo nunca cai sobre o antigo.

O que o `shell apply` escreve

CAMINHO	CONTEÚDO
`~/.zshrc` (Debian, Ubuntu, Mint, Zorin, Arch, macOS e server)	`PATH`, `mise activate`, Oh My Zsh com plugins, zoxide, Starship, aliases e `~/.zshrc.local`
`~/.bashrc` (Omarchy)	`PATH`, `STARSHIP_CONFIG` para `~/.config/setupvibe/starship.toml`, aliases e `~/.bashrc.local`; os arquivos do Omarchy ficam intactos
`~/.tmux.conf` (menos Omarchy)	carrega `~/.setupvibe/shell/tmux.conf`, que carrega `~/.tmux.conf.local` antes do TPM
`~/.setupvibe/shell/aliases.sh`	aliases válidos em ZSH e Bash
`~/.setupvibe/shell/aliases-agents.sh`	aliases que abrem agentes sem confirmação
`~/.setupvibe/shell/tmux.conf`	plugins, tema e teclas do tmux
`~/.zshrc.local`, `~/.bashrc.local`, `~/.tmux.conf.local`	o lugar das suas linhas, criados uma vez e nunca sobrescritos

Antes da primeira alteração, o arquivo original vai para `~/.setupvibe/backups/` com o sufixo `.first` e uma cópia com data. No Linux, o `apply` também troca o shell de login para ZSH com `chsh`, usando sudo quando precisa.

```
setupvibe shell apply --dry-run
setupvibe shell apply
```

O primeiro comando mostra os arquivos e os blocos que seriam escritos; o segundo escreve. Depois de aplicar, abra um terminal novo ou recarregue o shell na sessão atual:

```
exec $SHELL -l
```

O bloco e o arquivo local

O bloco começa em `# >>> setupvibe >>>` e termina em `# <<< setupvibe <<<`. Ele é substituído inteiro a cada execução, então não edite dentro dele: a próxima atualização descarta a alteração. O que você escreve fora do bloco fica.

As suas linhas vão para o arquivo `.local`:

```
# ~/.zshrc.local
export EDITOR=vim
alias deploy='./scripts/deploy.sh'
```

O formato é o mesmo do `.zshrc`, e o arquivo é carregado no fim do bloco. Para tmux, `~/.tmux.conf.local` é carregado antes do TPM, o que permite trocar teclas e tema sem tocar no arquivo gerenciado.

Aliases

Os aliases passam por um guarda que só define o nome quando nada responde por ele: nem alias, nem função, nem comando instalado. O seu setup sempre vence, e nenhum alias do SetupVibe esconde um programa real. `df` é a exceção, porque envolve o próprio comando de propósito.

Nomes que escondiam programas comuns foram renomeados: `gs` virou `gstat` (Ghostscript), `ci` virou `cin` (RCS), `ts` virou `tsv` (moreutils), `dc` virou `dco` (a calculadora `dc`) e `gofmt` virou `gofmtw`. `pn` e `pnx` são pulados em silêncio quando o pnpm já instala comandos com esses nomes, porque fazem a mesma coisa. O `setupvibe doctor` lista os aliases pulados na máquina.

Os aliases de agente ficam em arquivo separado e abrem a ferramenta sem pedir confirmação: `cco` para Claude Code, `cxo` para `codex --yolo` e `oco` para `opencode --auto`. `cc` não serve para Claude Code porque já é o compilador C. Para deixar os três de fora e voltar a abrir os agentes pelo comando completo:

```
setupvibe config set shell.agentAliases false
setupvibe shell apply
```

A troca vale na próxima aplicação do shell. O `doctor` mostra o estado dos aliases na checklist do módulo Usuário e nos achados do Doctor.

Ver o que está aplicado

O comando `shell` sem subcommand imprime a ajuda. Para conferir o estado, use o `doctor`, que aponta o bloco ausente, o `PATH` incompleto e cada alias pulado:

```
setupvibe doctor
setupvibe config get shell.agentAliases
```

Remover sem perder o resto

`shell remove` apaga só o conteúdo entre os marcadores. Os arquivos `.local`, os backups, os itens instalados e a pasta `~/.setupvibe/` ficam.

```
setupvibe shell remove
```

O comando não desinstala pacote, não apaga o estado e não desfaz a troca do shell de login. Para desfazer tudo, use `self uninstall`, no capítulo de manutenção.

Se você removeu o bloco e quer o ambiente de volta, aplique outra vez:

```
setupvibe shell apply
```

Omarchy

O Omarchy já tem shell, prompt, tmux e aliases próprios. Nessa máquina, o SetupVibe escreve no `~/.bashrc` um bloco que aponta para o Starship do SetupVibe e para os aliases, sem substituir `.bashrc`, `starship.toml` nem a configuração de tmux do sistema. Tudo o que o Omarchy mantém continua valendo, e o que o SetupVibe acrescenta fica dentro do bloco.

Próximo passo

O ambiente do terminal está aplicado e as suas personalizações têm lugar próprio. O capítulo Manutenção cobre o que fazer daqui em diante: diagnosticar, atualizar o binário, tirar um relatório e remover a instalação quando ela não serve mais.

Manutenção

Uma máquina preparada continua sendo uma máquina: o sistema atualiza, o binário ganha versão nova, um item quebra e alguém precisa entender o que aconteceu seis meses depois. Este capítulo reúne os comandos que respondem a essas situações sem reinstalar tudo.

Diagnóstico

`setupvibe doctor` é o primeiro comando quando algo parece errado. Ele confere o runtime, o estado do SetupVibe, os pré-requisitos, os itens selecionados, o bloco do shell, o `PATH`, restos do SetupVibe 1 e, no Windows, a saúde do component store.

```
setupvibe doctor
setupvibe doctor --fix
```

`--fix` aplica as correções seguras: cria `~/.local/bin`, corrige o modo da pasta de estado e escreve o bloco do shell. Antes de gravar o bloco, ele converte os arquivos do SetupVibe 1, igual ao `auto` e ao `shell apply`. O que exige decisão sua, como remover uma duplicidade ou trocar de edição, fica listado com o comando que resolve.

Um achado de nível `fail` faz o comando sair com 3, `CONFIG`, mesmo que os outros estejam aprovados. Leia o texto antes de tratar como falha de configuração no sentido literal.

No Windows, a verificação de componentes usa `DISM /Online /Cleanup-Image /CheckHealth`, que é rápida. A reparação lenta é explícita:

```
setupvibe doctor --repair-windows
```

Ela roda `DISM /RestoreHealth` e depois `sfc /scannow`, exige PowerShell elevado e demora. Se um dos dois falhar, o comando sai com 10.

Atualizar o binário

O `upgrade` troca o executável pela release mais nova do canal. A verificação é a mesma do installer: `SHA256SUMS` sempre, assinatura Sigstore do `SHA256SUMS` quando o `cosign` está no `PATH`. Assinatura que não corresponde interrompe a troca.

```
setupvibe upgrade --check
setupvibe upgrade
```

`--check` só informa se existe versão nova. `--channel beta` acompanha o canal de teste, e o canal também pode vir de `config.json`. Quando o binário está em `/usr/local/bin`, a troca precisa de permissão de escrita nesse diretório.

Depois de atualizar o binário, rode o `update` para aplicar as versões que a lista nova fixa:

```
setupvibe upgrade && setupvibe update
```

O `upgrade` não mexe nos itens; o `update` não baixa binário. São duas etapas separadas de propósito, para você ver o que muda na máquina.

Relatório para uma issue

`setupvibe report` monta um Markdown com sistema, achados do doctor, itens, apps externos, último run e as últimas linhas do log. Valores de variáveis sensíveis saem mascarados.

```
setupvibe report
setupvibe report --output report.md
```

O relatório não substitui o log completo quando o problema depende de detalhe. Para anexar numa issue do GitHub sem abrir o editor:

```
setupvibe report --output report.md && gh issue create --body-file report.md
```

Logs e o último resultado

Cada execução grava `~/.setupvibe/logs/<data>-<command>.log`, com um evento NDJSON por linha. O `auto` grava também `~/.setupvibe/last-run.json`, com o resultado passo a passo.

Três formas de ler:

```
setupvibe --tab logs
tail -n 40 ~/.setupvibe/logs/*.log
jq '.steps[] | select(.status=="fail")' ~/.setupvibe/last-run.json
```

A TUI mostra a execução e os eventos dentro do módulo Logs, com a mensagem de erro abaixo de cada falha. No terminal, o `tail` resolve o caso de uma execução recente, e o `jq` ajuda quando o `last-run.json` é grande.

Repetir uma seleção em outra máquina

`config export` grava a seleção e a identidade do Git num arquivo sem segredo. `config import` instala o que está nele, com o mesmo resultado de um `install`, e aplica a identidade do Git.

```
setupvibe config export selecao.json
setupvibe config import selecao.json --dry-run
setupvibe config import selecao.json
```

O arquivo exportado não carrega credencial. Numa máquina nova, instale o binário e rode o `init` antes de importar, porque `install` e `config import` exigem a preparação concluída.

Remover o SetupVibe

`self uninstall` tira os blocos do shell, a regra de sudo, os binários encontrados e, sem `--keep-data`, a pasta `~/.setupvibe/`. Backups dos seus arquivos, volumes do Portainer e restos do SetupVibe 1 ficam.

```
setupvibe self uninstall --dry-run
setupvibe self uninstall --yes
```

`--keep-data` mantém estado, logs e backups. `--items` remove também os itens instalados no espaço do usuário, que de outra forma continuam na máquina. Sem terminal, `--yes` é obrigatório.

Chave SSH num servidor

`ssh copy-id` acrescenta a sua chave pública ao `authorized_keys` do servidor remoto uma única vez, com os modos corretos. A chave viaja na entrada padrão, nunca na linha de comando.

```
setupvibe ssh copy-id deploy@203.0.113.10
setupvibe ssh copy-id deploy@host --port 2222
```

Com `--password-stdin`, a senha do servidor é lida da entrada e respondida pelo `sshpass`. Sem chave pública na máquina, o comando termina com 6, `NOT_FOUND`.

Skills de agente

O comando `skill` executa o skills CLI da Vercel fixado na lista. Os argumentos vão depois de `--`, para que as options cheguem ao CLI:

```
setupvibe skill -- list
setupvibe skill -- add promovaweb/herdrfy -g -a claude-code
setupvibe skill -- add https://github.com/promovaweb/setupvibe/tree/v2/desktop/
skills/setupvibe
```

A última linha instala a skill que ensina um agente a operar o próprio binário. Ela está em `desktop/skills/setupvibe` e em `server/skills/setupvibe-server`.

Documentação embutida

O binário carrega a documentação que um agente lê sem o repositório:

```
setupvibe --skill
setupvibe docs inventory
setupvibe docs cli --lang en
```

`--skill` aceita `cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory` e `reference`. O `docs` aceita os mesmos tópicos e `--lang`. O tópico `reference` traz só a referência gerada do registro, com commands, options e exit codes.

Rotina

Uma sequência curta cobre a maior parte da manutenção:

FREQUÊNCIA	COMANDO	PARA QUE
depois de uma atualização do sistema	`setupvibe doctor`	ver o que mudou de estado
antes de abrir uma issue	`setupvibe report`	anexar o estado sem segredo
quando o binário avisa de versão nova	`setupvibe upgrade && setupvibe update`	trocar binário e versões fixadas
antes de formatar a máquina	`setupvibe config export`	levar a seleção para a próxima

Próximo passo

O capítulo Servidor e segurança trata da máquina de operação: usuário de trabalho, UFW, fail2ban, SSH e Swarm.

Servidor e segurança

Um VPS novo começa como root, sem usuário de trabalho, sem firewall e com SSH aceitando senha. Antes de clonar o primeiro projeto, essa máquina precisa de quatro coisas na ordem certa: um usuário que não seja root, a chave dele no lugar, o firewall fechado e o SSH só por chave. O SetupVibe faz as quatro com a edição server, que é separada da desktop de propósito.

A edição server instala só o que uma máquina de operação usa: Docker, Portainer, Ansible, GitHub CLI, Tailscale, servidor SSH, as CLIs de agente, shell e tmux. Ela não traz PHP, Ruby, Go nem os aplicativos gráficos da desktop. O que as duas compartilham é a pasta `~/.setupvibe/`, o prefixo `SETUPVIBE_` e a tabela de exit codes.

Instalar a edição server

O installer é o mesmo, com a edição escolhida na linha de comando:

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install.sh | sh -s -- --edition server
```

Também existe o endereço que já chega com a edição escolhida:

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install-server.sh | sh
```

A verificação é a mesma da desktop: versão lida de `version.json`, binário do release, `SHA256SUMS` e assinatura Sigstore quando o `cosign` existe. A edição server roda só em Linux, em Ubuntu e Debian; em macOS o installer recusa com 11, `UNSUPPORTED_SYSTEM`.

As edições são exclusivas por máquina. Se a máquina já tiver a desktop preparada, o `init` da server sai com 2 e pede o `self uninstall` antes da troca.

Usuário de trabalho

Como root, crie o usuário antes de qualquer outra etapa. O `user create` cria o usuário, o home, a entrada no grupo de sudo (`wheel` no Arch e no Omarchy), instala a chave pública em `authorized_keys` e, com `--nopasswd`, a regra de sudo sem senha validada por `visudo -cf` antes de entrar no lugar.

```
setupvibe user create deploy --ssh-key ~/.ssh/id_ed25519.pub
```

A saída termina com o próximo passo: entrar como o usuário novo e rodar o `init`. A regra sem senha é temporária quando criada pelo `auto`; quando criada por `user create --nopasswd`, fica até você removê-la:

```
setupvibe user nopasswd deploy --remove
```

A regra fica em `/etc/sudoers.d/90-setupvibe-deploy`. Outras regras em `/etc/sudoers.d/` não são tocadas.

Módulo de segurança

`security apply` aplica o módulo compartilhado: UFW negando entrada, com SSH liberado e, no servidor com `--swarm`, as portas do Swarm; o jail `sshd` do fail2ban; atualizações automáticas de segurança em Debian e Ubuntu; e sincronização de horário, com `--timezone` quando você quer fixar o fuso.

O SSH muda para login só por chave com `/etc/ssh/sshd_config.d/10-setupvibe.conf` apenas quando o usuário atual já tem uma chave autorizada. O arquivo passa por `sshd -t` antes de o serviço recarregar. O SetupVibe nunca liga autenticação por senha.

```
setupvibe security apply --dry-run
setupvibe security apply --timezone America/Sao_Paulo
```

Antes da primeira mudança, o módulo registra em `~/.setupvibe/state/security.json` se o UFW e o fail2ban estavam ativos e quais arquivos dele já existiam. Esse registro é o que permite desfazer sem adivinhar:

```
setupvibe security remove
```

O `remove` apaga os arquivos que o SetupVibe criou, desliga o fail2ban e o UFW quando eles estavam desligados antes e apaga o registro. Os pacotes continuam instalados, e arquivos que existiam antes ficam como estavam. Sem registro, o comando sai com 3, `CONFIG`.

O fluxo de um VPS novo

A sequência abaixo parte de uma máquina que acabou de subir, com a sua chave já na mão. O `auto` cria o usuário, continua como ele, instala a seleção da edição server e inicia o Swarm como manager.

```
curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install-
server.sh \
  | sh -s -- auto --user deploy --ssh-key ~/.ssh/id_ed25519.pub \
    --manager --advertise-addr 10.0.0.1:2377
```

O endereço anunciado precisa ser o que os outros nós alcançam. O `auto` remove a regra temporária de sudo no fim, salvo `--keep-nopasswd`. O módulo de segurança entra automaticamente na edição server e na desktop exposta.

Para juntar um worker, peça o token no manager e passe para a máquina nova:

```
TOKEN=$(ssh deploy@10.0.0.1 setupvibe-server swarm token)
curl -fsSL https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install-
server.sh \
  | sh -s -- auto --user deploy --ssh-key ~/.ssh/id_ed25519.pub \
    --join "$TOKEN" --advertise-addr 10.0.0.2:2377
```

`swarm token manager` devolve o token de manager, e `--rotate` troca o token de worker antes de imprimir. Com `--encrypted-network`, a rede overlay do Swarm passa a cifrar o tráfego. O token entra por variável de ambiente ou pela option, nunca pelo histórico do shell quando você usa `SETUPVIBE_SWARM_TOKEN`.

Portainer e portas publicadas

Nas variantes remotas, o `auto` prende o Portainer a `127.0.0.1` e grava `"ip": "127.0.0.1"` em `/etc/docker/daemon.json` quando o arquivo não tem endereço. Docker escreve as próprias regras de iptables e não passa pelo UFW; uma porta publicada sem esse cuidado ficaria aberta para a internet. Chegue ao Portainer por túnel SSH ou pela tailnet.

O `doctor` lista contêiner que publica porta em todas as interfaces e mostra o comando para prender a porta ao localhost. Para abrir só uma porta, publique com `-p 127.0.0.1:PORTA:PORTA` e alcance pelo túnel ou pelo Tailscale.

Acesso remoto

Três itens resolvem o acesso de trabalho: `openssh-server`, `tailscale` e `mosh`. O Tailscale entra com a auth key da variável `TAILSCALE_AUTHKEY`, e `--tailscale-ssh` liga o SSH do Tailscale depois que a máquina entra na tailnet.

```
setupvibe auto --profile remoto --tailscale-authkey "$TAILSCALE_AUTHKEY" --
tailscale-ssh
```

Com o SSH só por chave, a primeira conexão precisa acontecer depois de a chave estar no `authorized_keys`. O `ssh copy-id` faz isso antes de o módulo de segurança entrar:

```
setupvibe ssh copy-id deploy@203.0.113.10
```

Conferir o servidor

Depois da instalação, a conferência é a mesma da desktop, com o binário da edição server:

```
setupvibe-server check
setupvibe-server doctor
```

O `doctor` mostra o estado do Swarm, das portas publicadas, do firewall e dos itens. O `report` gera o Markdown para uma issue quando a falha precisa de análise de outra pessoa.

Se algo der errado

SINTOMA	EXIT CODE	O QUE FAZER
execução como root sem `--user`	12 `ROOT`	passe `--user` ou crie o usuário com `user create`
sudo pede senha sem terminal	13 `SUDO_PASSWORD`	crie a regra com `user nopasswd` ou rode no terminal
`security remove` sem registro	3 `CONFIG`	o módulo não foi aplicado por este binário; confira o estado à mão
parte dos passos falhou	10 `PARTIAL`	leia o log e rode `doctor`
edição desktop já preparada	2 `USAGE`	rode `self uninstall` antes de instalar a server

Próximo passo

O servidor está preparado e fechado. O capítulo Agentes de IA e MCP mostra como um agente lê esse estado e executa o SetupVibe por protocolo, sem depender de você copiar a saída do terminal.

Agentes de IA e MCP

Um agente de código consegue ler um terminal, mas não consegue adivinhar o estado de uma máquina. Se ele chama `setupvibe doctor` e interpreta texto, cada versão do binário pode quebrar a leitura. O servidor MCP existe para isso: o agente pergunta em protocolo, recebe um objeto com o mesmo envelope do `--json` e decide pelo exit code.

O servidor roda dentro do mesmo executável, sobre stdio, e não escreve nada além do protocolo em stdout. Diagnóstico vai para stderr.

```
setupvibe mcp
```

O que o servidor expõe

São 20 tools, 6 resources e 1 prompt. As tools que executam o CLI devolvem `{ exitCode, ok, command, data }` ou `{ exitCode, ok: false, error }`; as tools nativas, como `logs` e `machine`, devolvem o objeto próprio.

TOOL	O QUE FAZ	ALTERA A MÁQUINA
`about`	identidade e runtime do binário	não
`exit_codes`	tabela de exit codes	não
`skill`	documentação embutida, inteira ou por tópico	não
`config_list`	keys de `settings.json` em vigor	não
`config_get`	valor de uma key	não
`credentials_status`	nomes e origem da master key, sem valores	não
`system`	sistema, arquitetura, WSL, headless, IP público e variante	não
`list`	itens com o status neste sistema	não
`check`	itens instalados contra as versões fixadas	não
`health`	só os achados do SetupVibe no doctor	não
`doctor`	runtime e achados, com o comando que corrige	não

TOOL	O QUE FAZ	ALTERA A MÁQUINA
`report`	relatório em Markdown para uma issue	não
`logs`	execuções e os eventos de uma delas	não
`tailscale`	cliente local do Tailscale	não
`machine`	CPU, memória, discos, rede e processos	não
`install`	instala itens e dependências	só com `dry_run: false`
`remove`	remove itens do espaço do usuário	só com `dry_run: false` e `confirm: true`
`update`	atualiza itens com versão fixada nova	só com `dry_run: false`
`init`	pré-requisitos do SetupVibe	só com `dry_run: false`
`migrate`	migra uma máquina do SetupVibe 1	só com `dry_run: false` e `confirm: true`

A regra de escrita é deliberada. Instalar, remover, atualizar, preparar e migrar fazem dry run por padrão; o agente precisa pedir a execução real, e as duas operações destrutivas exigem confirmação explícita. Nenhuma tool devolve valor de credencial, e nenhuma grava `settings.json`: a configuração continua no CLI, onde há uma pessoa para confirmar.

Resources

Os resources são leitura passiva. O agente pede o URI e recebe JSON ou Markdown.

RESOURCE	CONTEÚDO
`setupvibe://inventory`	a lista da edição, com itens, versões e métodos
`setupvibe://selection`	o que esta máquina selecionou, com versão e status
`setupvibe://about`	identidade e runtime em JSON
`setupvibe://exit-codes`	a tabela de exit codes em JSON
`setupvibe://skill`	a documentação embutida em Markdown
`setupvibe://skill/{topic}`	uma seção da documentação

Os tópicos são os mesmos do `--skill`: `cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory` e `reference`. Um agente que recebe o binário sem o repositório lê a documentação inteira por um desses URIs.

O prompt `operate`

O prompt não executa nada. Ele monta a instrução inicial de um agente com o nome e a versão do binário, a ordem de leitura e a tarefa recebida.

```
{ "task": "descobrir por que o node está fora da versão fixada" }
```

A resposta é uma mensagem de usuário com a ordem de leitura e a tarefa. É a forma de começar uma sessão sem repetir no prompt o que o binário já sabe.

Listar antes de conectar

```
setupvibe mcp --list
setupvibe mcp --list --json
```

A lista mostra nome e descrição de cada tool, o URI de cada resource e o prompt, na ordem em que o servidor registra. Use-a para conferir o que a versão do binário expõe antes de configurar um cliente.

Configurar o cliente

O bloco mínimo aponta para o binário e para o subcommand `mcp`:

```
{ "mcpServers": { "setupvibe": { "command": "setupvibe", "args": ["mcp"] } } }
```

No OpenCode, o bloco fica na configuração do aplicativo:

```
{ "mcp": { "setupvibe": { "type": "local", "command": ["setupvibe", "mcp"],
"enabled": true } } }
```

O `mcp install` registra o servidor no Claude Code (`claude mcp add --scope user`), no Codex (`codex mcp add`) e no OpenCode, com cópia de segurança do arquivo antes de mesclar o bloco.

```
setupvibe mcp install --dry-run
setupvibe mcp install
```

Agente que não está no `PATH` aparece como `not installed` e fica de fora. A execução é idempotente: rodar de novo não duplica o que já está configurado.

Um agente em ação

O exemplo junta as três leituras que um agente faz antes de agir.

1. **Entender o problema.** `check` com o item citado devolve `status` e `expected`; `doctor` devolve os achados com o comando que corrige.
2. **Ver o que mudaria.** `install` com o padrão de dry run devolve `items` e `skipped`, sem alterar a máquina.
3. **Executar.** `install` com `dry_run: false` devolve `log` e `results`, e o agente confere o `exitCode` antes de relatar sucesso.

Para remover, o mesmo caminho com `dry_run: false` e `confirm: true`. Para uma máquina vinda do SetupVibe 1, `migrate` segue a mesma regra: dry run primeiro, confirmação depois.

O agente deve tratar `exitCode` antes de `isError` e antes da mensagem. Um `doctor` com achado de nível `fail` marca `isError` e devolve os achados junto, com exit code 3; ler só o `isError` esconde o que precisa ser corrigido.

Skill do próprio binário

Além do protocolo, existe uma skill que ensina o agente a operar o SetupVibe pelo CLI, com dry run, envelopes e exit codes. Ela é instalada pelo skills CLI:

```
setupvibe skill -- add https://github.com/promovaweb/setupvibe/tree/v2/desktop/skills/setupvibe
```

Na edição server, o caminho termina em `server/skills/setupvibe-server`. A skill e o servidor MCP se complementam: a skill diz como conduzir o trabalho, o servidor entrega o estado em objetos.

Limites

O servidor MCP não substitui o CLI para configuração. Ele também não executa comando arbitrário: cada tool tem schema próprio, com campos tipados e obrigatoriedade declarada. Um agente não consegue, por exemplo, gravar uma credencial nem mudar uma key de `settings.json` por MCP.

Próximo passo

O agente já consegue operar a máquina por protocolo. O capítulo Windows e WSL mostra o que muda quando o alvo é o Windows nativo e como o WSL entra nessa história.

Windows e WSL

O Windows não aceita o mesmo preparo de um Linux. O gerenciador de pacotes é outro, o shell é PowerShell, e boa parte das ferramentas que um projeto espera roda dentro do WSL. O SetupVibe trata os dois como alvos separados: o binário nativo prepara o Windows, e o WSL recebe o binário Linux quando você entra na distribuição.

Instalar no Windows

O installer do Windows é um script PowerShell servido pelo branch `v2`. Ele lê a versão do canal, baixa `setupvibe-windows-x64.exe`, confere o `SHA256SUMS`, instala em `%LOCALAPPDATA%\Programs\SetupVibe` e acrescenta a pasta ao `PATH` do usuário.

```
irm https://raw.githubusercontent.com/promovaweb/setupvibe/v2/install.ps1 | iex
```

Para passar argumentos ao binário, use a forma de scriptblock:

```
& ([scriptblock]::Create((irm https://raw.githubusercontent.com/promovaweb/
setupvibe/v2/install.ps1))) auto
```

No lançamento, `windows.setupvibe.dev` passa a servir o mesmo arquivo. Até lá, o endereço acima é o que está publicado no branch `v2`, e o endereço público continua apontando para o SetupVibe 1.

O binário exige Windows x64 com build 22621 ou mais recente. Uma arquitetura fora de x64 e uma build anterior saem com 11, `UNSUPPORTED_SYSTEM`. Como o `auto` precisa instalar pacotes e alterar serviços, rode num PowerShell elevado; o binário não abre o UAC sozinho.

O que entra no Windows

O winget instala os utilitários: Git, PowerShell 7, Windows Terminal, 7-Zip, FFmpeg, ImageMagick, Nerd Font, nmap, Speedtest, btop e Tailscale. O servidor OpenSSH usa instalador próprio, e o Claude Code e o Antigravity entram pelo instalador oficial de cada fornecedor.

Os runtimes e as ferramentas de terminal entram pelo mise, igual ao Linux: Node, Python, Go, Rust, uv, Bun, pnpm, `bat`, `eza`, `jq` e `ripgrep`. O WSL entra como configuração, não como distribuição: o item `wsl` prepara a base e deixa a escolha da distribuição para você.

O que não existe no Windows aparece como `unavailable` na lista, e o `auto` não tenta instalar. ZSH, Oh My Zsh, Starship, tmux e os itens gráficos de desktop ficam de fora, porque o alvo é o shell nativo.

O WSL

Instalar uma distribuição Linux no Windows é decisão sua. O SetupVibe não baixa Ubuntu nem Debian sozinho: ele prepara a base do WSL 2 e o arquivo de configuração que faz o resto funcionar.

O `auto` grava um bloco em `%USERPROFILE%\.wslconfig`, depois de uma cópia de segurança, com rede espelhada, DNS por tunelamento, proxy automático, firewall ligado, recuperação gradual de memória e disco virtual esparso:

```
[wsl2]
networkingMode=mirrored
dnsTunneling=true
autoProxy=true
firewall=true

[experimental]
autoMemoryReclaim=gradual
sparseVhd=true
```

A rede espelhada faz o WSL herdar as interfaces e a VPN do Windows, e o tunelamento de DNS resolve o caso de VPN corporativa. O firewall do Hyper-V continua ligado com o padrão que bloqueia conexões de entrada: um serviço que roda dentro do WSL e responde ali não responde a partir do Windows até você abrir a porta.

Isso é diferente do SetupVibe 1, que abria toda conexão de entrada. Para abrir só a porta necessária, num PowerShell elevado:

```
New-NetFirewallHyperVRule -Name setupvibe-3000 -DisplayName setupvibe-3000
-Direction Inbound -VMCreatorId '{40E0AC32-46A5-438A-A0B2-2B479E8F2E90}' -Protocol
TCP -LocalPorts 3000
```

Para trabalhar por SSH ou Tailscale, prefira publicar na própria máquina e alcançar pelo túnel, sem abrir porta no firewall.

O binário dentro do WSL

O binário nativo do Windows cuida do Windows. Dentro de uma distribuição WSL, instale o binário Linux com o installer normal, como no capítulo de instalação:

```
curl --proto '=https' --tlsv1.2 -fsSL https://raw.githubusercontent.com/promovaweb/
setupvibe/v2/install.sh | sh
```

O detectSystem reconhece o WSL pelo `WSL_DISTRO_NAME` ou pelo `/proc/version`, e a lista da edição desktop já tem entradas para esse sistema. O que muda é a origem de alguns itens: no WSL, o Docker CLI pode ser dispensado quando o Docker Desktop do Windows já o fornece, e o `auto` respeita isso.

Diagnóstico no Windows

`setupvibe doctor` confere o runtime, o estado, os itens e a saúde do component store com `DISM /Online /Cleanup-Image /CheckHealth`. A reparação lenta é explícita e demora:

```
setupvibe doctor --repair-windows
```

Ela roda `DISM /RestoreHealth` e depois `sfc /scannow`, exige PowerShell elevado e devolve 10 quando um dos dois falha. Um component store que não pode ser reparado pede a imagem de instalação da mesma build.

Quando o sistema pede reinício, o `auto` termina com 20, `REBOOT_PENDING`. Com `--reboot`, ele cria a tarefa `SetupVibeAutoResume`, que roda o `auto` de novo com privilégio máximo depois do reinício e se apaga no fim.

Se algo der errado

SINTOMA	CAUSA PROVÁVEL	O QUE FAZER
`init` ou `auto` param com 20	reinício pendente do Windows	reinicie ou repita com `--reboot`
`doctor` avisa do component store	componentes corrompidos	rode `doctor --repair-windows` num PowerShell elevado
serviço no WSL não responde de fora	firewall do Hyper-V bloqueia entrada	abra só a porta necessária ou use túnel
item marcado como `unavailable`	aquele item não existe no Windows	confira a tabela do `docs inventory`
`setupvibe: command not found`	pasta fora do `PATH` do usuário	abra um terminal novo ou acrescente a pasta

Próximo passo

Windows e WSL têm o que precisam. O capítulo Vindo do SetupVibe 1 trata da máquina que já foi preparada pelo produto anterior e precisa migrar sem perder configuração.

Vindo do SetupVibe 1

Uma máquina preparada pelo SetupVibe 1 tem valor: o shell funciona, as chaves estão no lugar, as ferramentas respondem. O problema é que aquele preparo foi escrito por scripts que não deixam versão em disco. O SetupVibe 2 não apaga o que encontra; ele reconhece os rastros, oferece converter e deixa você decidir o que sai.

Como a máquina antiga é reconhecida

O SetupVibe 1 não grava a versão em arquivo, então a detecção procura assinaturas conhecidas:

RASTRO	EDIÇÃO
`~/.zshrc` gerado por `conf/zshrc-*.zsh`, sem os marcadores do SetupVibe 2	desktop ou server
`~/.tmux.conf` gerado por `conf/tmux-*.conf`, sem marcadores	desktop ou server
bloco `# >>> SetupVibe Omarchy 4 >>>` em `~/.bashrc`	Omarchy
`~/.setupvibe/bin/ssh_copy_id` e `~/.setupvibe/portainer-compose.yml`	qualquer
`%ProgramData%\SetupVibe\*.json` e `~/.setupvibe/windows-utilities.json`	Windows

Um arquivo conta como SetupVibe 1 quando carrega a assinatura da versão 0.41.x ou quando pelo menos 60% das linhas estão nos originais embutidos no binário. Isso cobre releases anteriores da mesma série.

`setupvibe system` mostra o achado, `setupvibe doctor` repete com o comando que resolve e o Início da TUI sugere a migração até o `config.json` registrar `migratedFromV1`.

```
setupvibe system
setupvibe doctor
```

O que acontece sem migrar

O `auto` e o `shell apply` convertem um `.zshrc` e um `.tmux.conf` do SetupVibe 1 antes de escrever os blocos novos, então o bloco do SetupVibe 2 nunca cai sobre o arquivo antigo. A conversão é sem perda: o arquivo vai para `~/.setupvibe/backups/`, as linhas que você acrescentou passam para `~/.zshrc.local` ou `~/.tmux.conf.local` sob um comentário com data, e o arquivo é reescrito com o bloco.

Numa máquina que já tem ferramentas, o `init` monta a primeira seleção com o que passa na verificação e marca cada item como `adopted`. Item adotado não sai num `remove` sem `--force`.

O plano do `migrate`

`migrate` faz o mesmo e mais: converte o bloco do Omarchy, mantém o `ssh_copy_id` como atalho e pergunta sobre cada duplicidade. Comece pelo `--dry-run`, que mostra o plano e as linhas que vão mudar, com segredo mascarado:

```
setupvibe migrate --dry-run
```

Trecho do plano desta máquina, com a pasta pessoal reduzida a `~`:

```
migrate (dry run)

1. Move ~/.zshrc from SetupVibe 1 to the SetupVibe 2 block, keeping your lines in
~/.zshrc.local
2. Move ~/.tmux.conf from SetupVibe 1 to the SetupVibe 2 block, keeping your lines
in ~/.tmux.conf.local
3. Keep ssh_copy_id as a shortcut to setupvibe ssh copy-id, with the same options
4. Write the SetupVibe aliases and tmux settings to ~/.setupvibe/shell/
5. Starship gruvbox-rainbow preset in ~/.config/starship.toml
   skipped when: test -s ~/.config/starship.toml
6. SetupVibe block in ~/.zshrc
7. SetupVibe block in ~/.tmux.conf

Lines that move to ~/.zshrc.local:
  …

Duplicates kept (remove with --remove-legacy, or answer yes in a terminal): legacy-
go, legacy-rbenv, legacy-linuxbrew, legacy-npm-global, legacy-nodesource
```

O plano não altera nada. Ele mostra a ordem, os arquivos que serão copiados e as duplicidades que o comando encontrou. Leia antes de executar.

Executar

```
setupvibe migrate
```

Num terminal, o comando pergunta uma vez por duplicidade. Sem terminal, informe o que pode sair:

```
setupvibe migrate --remove-legacy go,rbenv
setupvibe migrate --remove-legacy all
```

As duplicidades são o Go em `~/.local/go`, o `rbenv`, o `~/.npm-global` e o repositório NodeSource. O Node instalado continua; o que sai é o repositório APT que a versão antiga acrescentou.

O Linuxbrew é a exceção: ele nunca é removido pelo SetupVibe, porque outras fórmulas podem depender dele. O comando imprime o desinstalador para você rodar quando quiser. O `ssh_copy_id` antigo continua funcionando como atalho de `setupvibe ssh copy-id`, com as mesmas options `--host`, `--user`, `--pass`, `--key` e `--port`.

No fim, o `config.json` registra `migratedFromV1` com a data e a edição de origem, e os itens encontrados entram como adotados.

O que conferir depois

1. **Abra um shell novo.** O bloco novo carrega o mise, o Starship e os aliases.
2. **Rode `setupvibe check`.** O comando confere os itens adotados contra a versão fixada; o que estiver fora aparece com `✗`.
3. **Leia o `doctor`.** Os avisos de restos do SetupVibe 1 devem desaparecer depois da migração; o que ficar tem um comando ao lado.
4. **Confira as suas linhas.** O conteúdo movido está em `~/.zshrc.local` e `~/.tmux.conf.local`, na mesma ordem do arquivo antigo.
5. **Confira os backups.** O original está em `~/.setupvibe/backups/` se algo precisar voltar.

Se algo der errado

SINTOMA	EXIT CODE	O QUE FAZER
sudo pede senha sem terminal	13 `SUDO_PASSWORD`	rode no terminal ou crie a regra com `user nopasswd`
id desconhecido em `--remove-legacy`	2 `USAGE`	use `go`, `rbenv`, `npm-global`, `nodesource` ou `all`
parte dos passos falhou	10 `PARTIAL`	leia o log; o que ficou está no plano
sistema sem suporte	11 `UNSUPPORTED_SYSTEM`	confira `setupvibe system`

Se a conversão não resolver, o arquivo original está em `~/.setupvibe/backups/` e pode ser copiado de volta. Nada do SetupVibe 1 é apagado sem confirmação.

Próximo passo

A máquina migrou e a configuração antiga está preservada. O capítulo Problemas comuns e glossário reúne os sintomas que aparecem com mais frequência e o vocabulário usado no resto do guia.

Problemas comuns e glossário

Quase toda falha do SetupVibe tem um comando que a diagnostica. O `doctor` lê o runtime, o `init`, os itens, o `PATH`, o bloco do shell e os restos do SetupVibe 1, e mostra o comando ao lado de cada achado. Este capítulo reúne os sintomas que aparecem com mais frequência, o que cada um significa e o vocabulário usado no resto do guia.

Comece pelo `doctor`

```
setupvibe doctor
```

Um `✗` é problema, um `!` é aviso, e cada linha que exige ação traz o comando que resolve. Quando o achado é de nível `fail`, o comando termina com 3, `CONFIG`; leia o texto antes de tratar como arquivo de configuração quebrado.

Sintomas e saídas

SINTOMA	CAUSA PROVÁVEL	COMANDO
`sudo asks for a password`	o passo privilegiado usa `sudo -n` e não há senha em cache	`setupvibe user nopasswd <user>` ou `sudo -v` antes
`Another SetupVibe run is in progress`	outra execução segura `run.lock`	`cat ~/.setupvibe/run.lock` e esperar
erro de TLS ou certificado em download	proxy ou CA corporativa	`--ca-file` e `HTTPS_PROXY`, se necessário
`github.com unreachable`	rede ou DNS	`setupvibe system` e `--cache-dir`
relógio mais de cinco minutos fora	NTP desligado	`sudo timedatectl set-ntp true`
`3.2 GB free, 10 GB needed`	pouco espaço na pasta pessoal	liberar espaço ou mover o cache
item com status `other`	outra cópia instalada por APT, Homebrew ou npm	`setupvibe install <item>` e conferir o `PATH`
`3 of 42 steps failed`	falha parcial; o resto ficou aplicado	ler o log e repetir só o que faltou
`SetupVibe 1 found`	máquina preparada pela versão 1	`setupvibe migrate --dry-run`
`! Restart needed`	`/var/run/reboot-required` existe	reiniciar ou `auto --reboot`
contêiner publicando em `0.0.0.0`	Docker não passa pelo UFW	publicar em `127.0.0.1`

SINTOMA	CAUSA PROVÁVEL	COMANDO
alias não definido	já existe comando com o mesmo nome	conferir com `type <nome>`
`settings.json` inválido	valor fora do schema	`setupvibe config list` e corrigir
`Run: setupvibe upgrade`	arquivo de estado de versão mais nova	`setupvibe upgrade`
comando em pipe imprime a ajuda e sai com 2	sem TTY	rodar no terminal ou usar o CLI
duas edições na mesma máquina	desktop e server são exclusivas	`setupvibe self uninstall` antes da troca

Sudo sem senha

O `auto` exige `sudo` sem senha porque não abre prompt. `init`, `migrate` e `shell apply` pedem a senha uma vez num terminal e seguem com a credencial em cache; sem terminal, param antes de alterar a máquina.

```
setupvibe user nopasswd deploy
sudo -v && setupvibe auto --user deploy
```

O exit code é 13, `SUDO_PASSWORD`. Com `--keep-nopasswd`, o `auto` mantém a regra temporária que ele criou.

Falha parcial

Um passo com falha não interrompe os seguintes. O resultado fica no log e no `last-run.json`, com status por item, e o comando termina com 10, `PARTIAL`. Corrija a causa e repita só o que faltou:

```
setupvibe doctor
setupvibe install node python --force
setupvibe check
```

Não é preciso recomeçar do zero: o que passou continua aplicado.

Ferramenta de outro gerenciador

`other` aparece quando o comando existe fora das pastas do mise e de `~/.local/bin`. O SetupVibe não sobrescreve a cópia existente; ele instala a dele ao lado. Para conferir qual responde primeiro:

```
setupvibe list --check
command -v node
setupvibe install node
```

Se a versão ativa continuar sendo a outra, ajuste o `PATH` ou remova a cópia antiga. O `doctor` mostra o caminho da cópia que ganhou a disputa.

Onde ler mais

- `setupvibe --skill troubleshooting` imprime a mesma matéria com o detalhe de cada caso, dentro do binário.
- O apêndice de códigos de saída, no fim do livro, lista a classe de cada número.
- O apêndice de configuração explica cada arquivo de `~/.setupvibe/`.
- `setupvibe report` gera o Markdown para anexar numa issue quando a falha precisa de outra pessoa.

Glossário

TERMO	SIGNIFICADO
item	uma ferramenta da lista embutida, com id, versão, método e verificação
seleção	o conjunto de itens escolhidos, gravado em `selection.json`
perfil	um recorte nomeado da lista, como `essencial` e `remoto`
versão fixada	a versão que o binário instala e confere, não a última do fornecedor
adotado	item que já estava na máquina e foi registrado pelo `init` sem ser instalado
outro gerenciador	cópia instalada por APT, Homebrew, npm ou instalador do fornecedor
bloco do shell	trecho entre `# >>> setupvibe >>>` e `# <<< setupvibe <<<`
run log	arquivo em `~/.setupvibe/logs/`, com um evento NDJSON por linha
cofre	`credentials.json`, cifrado com AES-256-GCM

TERMO	SIGNIFICADO
master key	chave de 32 bytes que abre o cofre; fica em `master.key` ou em variável
variante	classificação da máquina: local, remota sem interface ou remota exposta
headless	máquina sem sessão gráfica; itens `gui` ficam de fora
dry run	execução que mostra o plano e não altera nada
canal	`stable` ou `beta`, de onde o `upgrade` e o aviso de versão leem
exit code	número que classifica o resultado do comando, de 0 a 130
OpenCLI	descrição JSON da árvore de commands, impressa por `--opencli`
envelope	objeto `{ ok, command, data }` ou `{ ok, command, error }` do `--json`

Fechamento

O guia termina nos apêndices. Quando precisar da lista exata de uma option, do conteúdo de um arquivo de estado ou do significado de um exit code, comece por eles; quando precisar lembrar por que a máquina está naquele estado, volte ao capítulo da operação correspondente.

Referência

Os quatro apêndices a seguir vêm da documentação embutida no binário, mantida no repositório em `desktop/docs/`. Eles completam os capítulos com a lista exata de options, arquivos de estado, exit codes e tools do servidor MCP.

APÊNDICE	ASSUNTO
A	commands, options globais e a referência gerada do registry
B	`settings.json`, arquivos de estado, cofre e variáveis de ambiente
C	a tabela de exit codes e o tratamento por classe
D	tools, resources, prompts e configuração nos clientes

Apêndice A. CLI

O binário `setupvibe` tem uma TUI, um CLI e um servidor MCP no mesmo executável. Este manual cobre o CLI: as convenções, as options globais, o formato de saída e cada comando com argumentos, options, efeitos, permissões, recusas e exemplos. A referência gerada do registry, que acompanha `--skill cli`, repete a mesma árvore de forma resumida.

Convenções

- `setupvibe <command> [subcommand] [arguments] [options]`.
- Options aceitam `--key value` e `--key=value`. Não há agrupamento de flags curtas (`-abc`).
- `--` encerra a leitura de options; o que vem depois é argument. Use `--` antes de argumentos que o `skills` ou outro programa precisa receber.
- Options globais valem em qualquer posição, antes ou depois do command.
- Sem argumentos, em terminal interativo, o binário abre a TUI. Sem TTY, imprime o help e sai com exit code 2.
- Toda saída destinada a um consumidor vai para stdout. Avisos, progresso e erros em modo texto vão para stderr.
- Nomes de commands, subcommands, options, arguments, keys e campos de saída são em inglês. Mensagens do binário também.
- Os comandos que instalam ou removem itens, criam usuários, aplicam segurança, escrevem a configuração do shell, migram a máquina ou configuram agentes aceitam `--dry-run`. O plano mostra cada etapa e cada comando sem executar nada.
- O valor de uma credencial nunca entra por argumento: `credentials set` lê do stdin ou do terminal. Tokens de `auto` têm option por compatibilidade, mas a forma recomendada é a variável de ambiente, que o run log mascara.

Options globais

OPTION	EFEITO
`-h`, `--help`	Help do command atual. Exit code 0.
`-V`, `--version`	Imprime `setupvibe <version>`. Exit code 0.
`--json`	Saída em JSON com envelope estável. Erros também saem em JSON, em stdout.
`--skill [topic]`	Documentação embutida. Topics: `cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory`, `reference`.

OPTION	EFEITO
`--opencli`	Descrição OpenCLI 0.1 da árvore de commands em JSON.
`--no-color`	Desliga cores ANSI. `NO_COLOR=1` tem o mesmo efeito.

O comando `docs` aceita os mesmos topics de `--skill` e mais `inventory`, e tem `--lang` para escolher a língua.

Saída em texto e em JSON

Texto é o padrão quando `output.format` em settings.json é `text`. Com `--json`, ou com `output.format = json`, a saída segue este envelope:

```
{
  "ok": true,
  "command": "setupvibe config get",
  "data": { "key": "tui.theme", "value": "night" }
}
```

Em erro:

```
{
  "ok": false,
  "command": "setupvibe credentials get",
  "error": {
    "code": 4,
    "name": "CREDENTIALS",
    "message": "Credential \"x\" not found.",
    "details": { "name": "x" }
  }
}
```

O exit code do processo é sempre o mesmo de `error.code`. Leia os dois: o envelope para a mensagem, o code para a classe da falha. A tabela completa está no Apêndice C.

O comando `auto --json` acrescenta uma linha JSON por evento do run log antes do envelope final. Consumidores desse comando leem a última linha com `ok` como resultado e tratam as anteriores como progresso.

Como ler as seções de comando

Cada comando tem a mesma ordem de informação: finalidade, sintaxe, tabela de argumentos e options, saída em texto e o `data` do envelope `--json`, efeitos persistentes, permissões, recusas com exit codes, limites e cinco exemplos. A tabela usa `-` para o que não se aplica e `flag` para option sem valor.

Os comandos que só agrupam subcommands (`config`, `credentials`, `shell`, `self`, `ssh`, `user` e `security`) imprimem o help e saem com 0 quando chamados sem subcommand. As seções deles mostram a finalidade do grupo e exemplos que combinam os subcommands.

Commands

`setupvibe tui`

Abre a interface de terminal no Início, com um cartão por módulo: Inventory, User, Doctor, Logs, Tailscale, Machine e About.

Sintaxe

```
setupvibe tui [--tab <tab>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--tab`	enum	não	`tui.initialTab`, ou `overview`	`overview`, `inventory`, `user`, `doctor`, `logs`, `tailscale`, `machine`, `about`

Saída

Texto: a interface. Ao fechar, nada é impresso. Com `--json`, o envelope final traz `data = { "closed": true }`.

Efeitos persistentes

Nenhum arquivo é criado pela abertura. Os módulos podem gravar `settings.json` e os arquivos de estado quando você altera uma preferência ou executa uma ação nelas.

Permissões

Exige TTY em stdin e stdout. Não exige sudo nem root.

Recusas e exit codes

Sem TTY: `NO_TTY` (8), com a mensagem para usar `--help`.

Limites

É o mesmo modo aberto por `setupvibe` sem argumentos. `--tab` sobrepõe a preferência apenas nesta execução.

Exemplos

```
setupvibe tui   # abre no módulo configurado em tui.initialTab
```

```
setupvibe tui --tab inventory   # começa no módulo Inventory, com a lista de itens e versões fixadas
```

```
setupvibe tui --tab machine   # começa no módulo Machine, com CPU, memória, discos e rede
```

```
setupvibe tui --tab doctor   # começa no módulo Doctor, com o diagnóstico da máquina
```

```
setupvibe tui --tab about   # começa no módulo About, com versão, runtime e credits
```

`setupvibe system`

Mostra o sistema detectado, a variante da máquina, o estado da atualização e os restos do SetupVibe 1.

Sintaxe

```
setupvibe system [--no-update-check] [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--no-update-check`	flag	não	ausente	presente ou ausente

Saída

Texto: tabelas `System` e `Update`. `data` traz `edition`, `system`, `id`, `name`, `version`, `arch`, `wsl`, `headless`, `publicIp`, `variant`, `distro` quando houver, `unsupportedReason` quando houver, `update` quando a consulta roda e `setupvibe1`. `update` traz `current`, `latest`, `minimum`, `changelog`, `updateAvailable`, `updateRequired`, `checkedAt` e `source` (`network`, `cache` ou `offline`).

Efeitos persistentes

A consulta de atualização grava o cache de seis horas em `~/.setupvibe/state/update-check.json`. Nada mais é alterado.

Permissões

Nenhuma. Pode rodar como root.

Recusas e exit codes

Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11), com o motivo no texto e em `unsupportedReason`.

Limites

A consulta de rede tem timeout curto e nunca interrompe o comando. Sem rede, `source` é `offline` e `latest` fica ausente.

Exemplos

```
setupvibe system   # leitura rápida do sistema, da variante e da atualização
```

```
setupvibe system --no-update-check   # máquina offline: não consulta a rede
```

```
setupvibe system --json   # resultado completo para um agente ou script
```

```
setupvibe system | grep SetupVibe   # confere se há restos do SetupVibe 1
```

```
setupvibe system --json | jq '.data.update.updateAvailable'   # só o estado da atualização
```

`setupvibe init`

Confere os requisitos e instala os pré-requisitos: dependências de build e mise. Também registra o init e a primeira seleção.

Sintaxe

```
setupvibe init [--dry-run] [--verbose] [--cache-dir <dir>] [--ca-file <file>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente
`--verbose`	flag	não	ausente	presente ou ausente
`--cache-dir`	caminho	não	`SETUPVIBE_CACHE_DIR`, depois `config.json`	pasta existente ou criada
`--ca-file`	caminho	não	`SETUPVIBE_CA_FILE`, depois `config.json`	arquivo PEM

Saída

Texto: requisitos com ✓, ! ou ✗ e o plano executado. `data` traz `system` e `checks`; no plano, `dryRun` e `steps`; na execução, `log` e `results`. Cada resultado tem `id`, `status`, `durationMs` e `message` quando houver.

Efeitos persistentes

Instala os itens `initOnly` e o mise. Grava `config.json` com `edition`, `system`, `init` e, em terminal, a identidade do Git. Cria `selection.json`, quando ainda não há seleção, com os itens que já passam na verificação, marcados `adopted`. Cria `~/.setupvibe/` com modo 0700 e um run log em `logs/`.

Permissões

Recusa root. Usa sudo para pacotes do sistema. Não exige TTY, mas só pergunta o nome e o e-mail do Git quando há terminal.

Recusas e exit codes

Root: `ROOT` (12). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Menos de 10 GB livres no desktop ou 5 GB no server: `DISK_SPACE` (14). Falha de rede, locale ou relógio: `NETWORK_OR_API` (5). Etapa com falha: `PARTIAL` (10). Sudo com senha sem terminal: `SUDO_PASSWORD` (13).

Limites

Depois do init, `install`, `update` e as ações da TUI que instalam itens funcionam. Rode o init de novo quando o `doctor` apontar pré-requisito novo. A option `--skip-network-check` existe para testes, não aparece no `--help` e não substitui uma rede quebrada em uso real.

Exemplos

```
setupvibe init --dry-run   # mostra requisitos e comandos sem instalar nada
```

```
setupvibe init   # instala dependências de build e mise e registra o init
```

```
setupvibe init --json   # resultado completo para um log de provisionamento
```

```
setupvibe init --ca-file /etc/ssl/corp/ca.pem   # rede corporativa com CA própria
```

```
setupvibe init --cache-dir /var/cache/setupvibe   # reaproveita downloads entre execuções
```

`setupvibe auto`

Instala tudo sem perguntar, para máquinas sem interface, provisionamento e agentes. As respostas vêm de options, de variáveis `SETUPVIBE_*` ou de um arquivo `--config`.

Sintaxe

```
setupvibe auto [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--user`	texto	como root, sim	`SETUPVIBE_USER`	nome de usuário
`--ssh-key`	texto ou caminho	não	`SETUPVIBE_SSH_KEY`	chave pública ou arquivo
`--git-name`	texto	não	`SETUPVIBE_GIT_NAME`	nome livre
`--git-email`	texto	não	`SETUPVIBE_GIT_EMAIL`	e-mail
`--profile`	texto	não	inventory completo	nome de profile do inventory, ou `none`
`--only`	texto	não	`SETUPVIBE_ONLY`	ids separados por vírgula
`--exclude`	texto	não	`SETUPVIBE_EXCLUDE`	ids separados por vírgula
`--tailscale-authkey`	texto	não	`TAILSCALE_AUTHKEY`	auth key do Tailscale
`--manager`	flag	não	`SETUPVIBE_SWARM_MANAGER=1`	server: inicia o Swarm como manager
`--join`	texto	não	`SETUPVIBE_SWARM_TOKEN`	server: token de entrada no Swarm
`--advertise-addr`	texto	não	`SETUPVIBE_SWARM_ADVERTISE_ADDR`	endereço, como `10.0.0.1:2377`
`--timezone`	texto	não	`SETUPVIBE_TIMEZONE`	zona IANA, como `America/Sao_Paulo`
`--reboot`	flag	não	`SETUPVIBE_REBOOT=1`	presente ou ausente
`--keep-nopasswd`	flag	não	`SETUPVIBE_KEEP_NOPASSWD=1`	presente ou ausente
`--encrypted-network`	flag	não	`SETUPVIBE_SWARM_ENCRYPTED=1`	server: rede overlay cifrada
`--pm2-autostart`	flag	não	`SETUPVIBE_PM2_AUTOSTART=1`	presente ou ausente

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--tailscale-ssh`	flag	não	`SETUPVIBE_TAILSCALE_SSH=1`	presente ou ausente
`--config`	caminho	não	-	arquivo no formato de `config export`
`--plan`	caminho	não	-	arquivo JSON de saída do plano
`--dry-run`	flag	não	ausente	presente ou ausente
`--verbose`	flag	não	ausente	presente ou ausente
`--cache-dir`	caminho	não	`SETUPVIBE_CACHE_DIR`	pasta de cache
`--ca-file`	caminho	não	`SETUPVIBE_CA_FILE`	arquivo PEM

Cada resposta segue a precedência option, variável de ambiente, arquivo `--config`. Tokens entram apenas por variável de ambiente.

Saída

Texto: cada etapa com ✓, - ou ✗, o resumo, as partes puladas e o caminho do log. `data` no plano traz `dryRun`, `system`, `items`, `skipped`, `security` e `steps`. Na execução, traz `version`, `finishedAt`, `log`, `system`, `items`, `skipped`, `security`, `results` e `checks`. Com `--json`, cada evento do run log sai como uma linha JSON antes do envelope.

Efeitos persistentes

Instala itens e pré-requisitos, configura o shell, o prompt e o tmux, aplica o módulo de segurança quando cabe, configura Git, Tailscale e Swarm, grava `selection.json`, `last-run.json`, `config.json`, `~/.config/mise/mise.lock` e os logs, e cria ou remove a regra de sudo sem senha. Com `--reboot`, agenda a retomada em uma unit systemd ou em uma tarefa do Windows.

Permissões

Como root, exige `--user`: cria o usuário e continua como ele. Como usuário comum, exige sudo sem senha. Não pede confirmação e funciona sem TTY. No Windows, exige PowerShell elevado.

Recusas e exit codes

Root sem `--user`: `ROOT` (12). Sudo com senha: `SUDO_PASSWORD` (13). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Disco insuficiente: `DISK_SPACE` (14). Rede ou relógio fora do esperado: `NETWORK_OR_API` (5). Itens com falha: `PARTIAL` (10). Reinício pendente: `REBOOT_PENDING` (20). Outra execução em andamento: `LOCKED` (30). Arquivo `--config` inválido: `USAGE` (2).

Limites

Uma etapa com falha não interrompe as seguintes; o fechamento lista o que falhou e o que foi pulado, inclusive partes de segurança. Itens gráficos ficam de fora em máquina sem interface. `--plan` grava o plano e sai sem alterar nada. `--only` e `--exclude` não substituem dependências: elas entram mesmo quando não foram citadas. No arquivo `--config`, `profile: "none"`, o valor gravado por `config export`, vale como base vazia: entram os itens de `only` e os requisitos deles. Sem o campo `profile`, entra o inventory inteiro.

Exemplos

```
setupvibe auto --dry-run   # plano completo sem alterar a máquina
```

```
setupvibe auto --user deploy --ssh-key "ssh-ed25519 AAAA..."   # como root, cria o usuário e continua como ele
```

```
setupvibe auto --profile essencial --exclude docker --json   # recorte do inventory com saída para automação
```

```
setupvibe auto --config setupvibe.json --reboot   # repete a seleção de outra máquina e retoma após reiniciar
```

```
setupvibe auto --plan /tmp/plano.json   # grava o plano em JSON e encerra sem mudanças
```

`setupvibe list`

Mostra o inventory com o status de cada item nesta máquina.

Sintaxe

```
setupvibe list [--category <id>] [--check] [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--category`	texto	não	todas	id de categoria do inventory
`--check`	flag	não	ausente	presente ou ausente

Saída

Texto: uma seção por categoria, com marcador, id, versão e status. `data` traz `system` e `items`, e cada item traz `id`, `name`, `category`, `version`, `default`, `status`, `elsewhere` quando a cópia veio de outro gerenciador e `check` com `--check`.

Efeitos persistentes

Nenhum. `--check` executa os comandos de verificação dos itens selecionados, sem instalar nem remover.

Permissões

Nenhuma. Não exige init.

Recusas e exit codes

Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11).

Limites

O status `other` indica uma cópia fora das pastas do mise; o caminho aparece ao lado. `gui` marca item gráfico pulado em máquina sem interface. `--check` verifica só itens já selecionados.

Exemplos

```
setupvibe list   # inventory completo com o status de cada item
```

```
setupvibe list --category ai   # só a categoria de ferramentas de IA
```

```
setupvibe list --check   # status e verificação dos itens selecionados
```

```
setupvibe list --json   # resultado completo para um painel ou agente
```

```
setupvibe list | grep outdated   # itens cuja versão fixada mudou
```

`setupvibe install`

Instala itens do inventory e as dependências, ou um app fora do inventory com `--external`.

Sintaxe

```
setupvibe install <items>... [options]
setupvibe install --external <tool[@version]> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`items`	texto, variádico	sim	-	ids do inventory, ou uma ferramenta com `--external`
`--external`	flag	não	ausente	presente ou ausente
`--force`	flag	não	ausente	reinstala o que já passa na verificação
`--dry-run`	flag	não	ausente	presente ou ausente
`--verbose`	flag	não	ausente	presente ou ausente
`--cache-dir`	caminho	não	`SETUPVIBE_CACHE_DIR`	pasta de cache
`--ca-file`	caminho	não	`SETUPVIBE_CA_FILE`	arquivo PEM

Saída

Texto: cada etapa e, na execução, o caminho do log. `data` traz `dryRun`, `items` e `skipped` no plano; `log` e `results` na execução, com `checks` quando alguma verificação falha. Com `--external`, traz `tool` e `version`.

Efeitos persistentes

Instala pelo método do inventory (APT, pacman, Homebrew, mise, Git, arquivo verificado, installer oficial, Composer ou gem). Atualiza `selection.json` com a versão e o status. Com `--external`, grava o app em `external.json` e fixa a versão resolvida.

Permissões

Usa sudo quando o método exige. Exige init concluído. Não exige TTY.

Recusas e exit codes

Sem init: `CONFIG` (3). Item desconhecido ou `--external` com mais de uma ferramenta: `USAGE` (2). Ferramenta que o mise não conhece: `NOT_FOUND` (6). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Etapa com falha ou verificação com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

O item aprovado na verificação é pulado, salvo `--force`. Dependências entram mesmo quando não foram citadas. Uma cópia instalada por outro gerenciador é preservada e o aviso mostra o caminho. `--external` aceita uma ferramenta por execução.

Exemplos

```
setupvibe install docker   # instala o Docker e as dependências
```

```
setupvibe install go rust --dry-run   # mostra as etapas sem instalar
```

```
setupvibe install --external terraform@1.13.4   # fixa uma ferramenta fora do
inventory
```

```
setupvibe install docker --force   # reinstala mesmo com a verificação atual
aprovada
```

```
setupvibe install node --cache-dir /var/cache/setupvibe --json   # reaproveita
downloads e entrega o resultado em JSON
```

`setupvibe remove`

Remove itens do inventory, ou um app instalado com `install --external`.

Sintaxe

```
setupvibe remove <items>... [options]
setupvibe remove --external <tool> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`items`	texto, variádico	sim	-	ids do inventory, ou uma ferramenta com `--external`
`--external`	flag	não	ausente	presente ou ausente
`--include-native`	flag	não	ausente	remove pacotes do gerenciador do sistema
`--force`	flag	não	ausente	remove dependente ou item adotado
`--yes`	flag	não	ausente	não pede confirmação (`-y`)
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e os itens recusados com o motivo. `data` traz `dryRun` e `refused` no plano; `log` e `results` na execução.

Efeitos persistentes

Remove o que foi instalado no espaço do usuário e atualiza `selection.json`. Com `--external`, remove de `external.json`. Pacotes nativos só saem com `--include-native`.

Permissões

Usa sudo para pacotes do sistema. Pede confirmação em terminal; sem terminal, exige `--yes`.

Recusas e exit codes

Sem `--yes` e sem terminal: `USAGE` (2). Resposta negativa ou Ctrl+C: `INTERRUPTED` (130). Item desconhecido ou dependente selecionado sem `--force`: `USAGE` (2). Ferramenta externa desconhecida: `NOT_FOUND` (6). Etapa com falha: `PARTIAL` (10). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11).

Limites

Item adotado pelo init fica na máquina salvo `--force`. Um item exigido por outro item selecionado não sai sem `--force`. Item instalado pelo init ou pelo sistema não é removido; o motivo aparece no texto. `--dry-run` mostra o plano e não pede confirmação.

Exemplos

```
setupvibe remove go --yes   # remove o Go instalado pelo SetupVibe
```

```
setupvibe remove docker --include-native --dry-run   # mostra o que sairia,
inclusive o pacote do sistema
```

```
setupvibe remove --external terraform --yes   # remove um app instalado com install
--external
```

```
setupvibe remove node --force --yes   # remove mesmo com outro item dependendo dele
```

```
setupvibe remove go rust --yes --json   # remove dois itens e entrega o resultado em
JSON
```

`setupvibe update`

Atualiza os itens selecionados cuja versão fixada mudou no inventory desta versão do SetupVibe.

Sintaxe

```
setupvibe update [--dry-run] [--verbose] [--cache-dir <dir>] [--ca-file <file>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente
`--verbose`	flag	não	ausente	presente ou ausente
`--cache-dir`	caminho	não	`SETUPVIBE_CACHE_DIR`	pasta de cache
`--ca-file`	caminho	não	`SETUPVIBE_CA_FILE`	arquivo PEM

Saída

Texto: cada item alterado, de qual versão para qual, e as etapas. `data` traz `outdated`, com `id`, `from` e `to`, e, na execução, `log` e `results`, com `checks` quando houver falha.

Efeitos persistentes

Reinstala os itens desatualizados e atualiza `selection.json`. Grava um run log.

Permissões

Exige init concluído. Usa sudo quando o método exige.

Recusas e exit codes

Sem init: `CONFIG` (3). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Etapa ou verificação com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

Não consulta versões novas na internet: compara a seleção com o inventory embutido no binário. Para atualizar o binário, use `upgrade`. Item fora do inventory não entra no `update`.

Exemplos

```
setupvibe update --dry-run   # mostra quais itens mudariam de versão
```

```
setupvibe update   # reinstala os itens cuja versão fixada mudou
```

```
setupvibe update --json   # lista `from` e `to` e o resultado de cada etapa
```

```
setupvibe update --cache-dir /var/cache/setupvibe   # reaproveita downloads
```

```
setupvibe update --dry-run | grep '↑'   # só os itens que seriam trocados
```

`setupvibe check`

Verifica se os itens selecionados estão instalados nas versões fixadas.

Sintaxe

```
setupvibe check [items]... [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`items`	texto, variádico	não	`selection.json`, ou o profile `essencial`	ids do inventory

Saída

Texto: uma linha por item, com ✓ ou ✗, a saída da verificação e a versão esperada. `data` traz `system` e `results`, e cada resultado traz `id`, `status`, `output` e `expected`.

Efeitos persistentes

Nenhum. Só executa os comandos de verificação.

Permissões

Nenhuma além do que o próprio item exige.

Recusas e exit codes

Item desconhecido: `USAGE` (2). Item fora da versão ou ausente: `PARTIAL` (10). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11).

Limites

Sem argumentos, usa a seleção; sem seleção, usa o profile `essencial`. Itens `initOnly`, do módulo de segurança e não instaláveis no sistema não entram na verificação.

Exemplos

```
setupvibe check   # verifica toda a seleção atual
```

```
setupvibe check node php --json   # verifica dois itens e entrega o resultado em JSON
```

```
setupvibe check docker   # confere só o Docker e a versão fixada
```

```
setupvibe check --json | jq '.data.results[] | select(.status=="fail")'   # só o que falhou
```

```
setupvibe check node || echo "rode: setupvibe install node"   # trata a falha em um script
```

`setupvibe shell`

Agrupa a configuração do ambiente do terminal. `apply` escreve os blocos e `remove` tira os blocos do SetupVibe.

Sintaxe

```
setupvibe shell <apply|remove> [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte as seções `shell apply` e `shell remove`.

Efeitos persistentes

Altera `~/.zshrc`, `~/.bashrc` no Omarchy e `~/.tmux.conf`, além dos arquivos em `~/.setupvibe/shell/`.

Permissões

A troca do shell de login usa sudo quando necessário.

Recusas e exit codes

Subcommand inválido: `USAGE` (2). Falha em etapa: `PARTIAL` (10).

Limites

O conteúdo fora dos marcadores não é alterado. Uma cópia do arquivo original fica em `~/.setupvibe/backups/` antes da primeira mudança.

Exemplos

```
setupvibe shell apply --dry-run   # mostra os arquivos e blocos que seriam escritos
```

```
setupvibe shell apply   # escreve o bloco, os aliases, o preset e o tmux
```

```
setupvibe shell remove   # tira somente os blocos do SetupVibe
```

```
setupvibe shell apply --json   # plano e resultado em JSON
```

```
setupvibe shell remove --dry-run   # confere o que sairia antes de remover
```

`setupvibe shell apply`

Escreve o bloco do SetupVibe no shell, os aliases, o preset do Starship e as configurações do tmux. Também converte arquivos do SetupVibe 1 quando encontra.

Sintaxe

```
setupvibe shell apply [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa com o resultado e o caminho do log. `data` traz `dryRun` e `steps` no plano; `log` e `results` na execução.

Efeitos persistentes

Grava o bloco entre `# >>> setupvibe >>>` e `# <<< setupvibe <<<`, os aliases em `~/.setupvibe/shell/aliases.sh` e `aliases-agents.sh`, o preset `gruvbox-rainbow` quando não existe configuração do Starship, o `~/.setupvibe/shell/tmux.conf` e os arquivos `.local` vazios. Faz backup dos arquivos antes da primeira mudança. No Linux, troca o shell de login para ZSH com `chsh`.

Permissões

Usa sudo no `chsh` quando necessário. Não exige TTY.

Recusas e exit codes

Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Etapa com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

O bloco é substituído por inteiro a cada execução. Suas linhas vão para `~/.zshrc.local`, `~/.bashrc.local` e `~/.tmux.conf.local`, que nunca são sobrescritos. Alias que colide com programa, função ou alias existente é pulado; `doctor` lista os pulados. No Omarchy, os arquivos do sistema ficam intactos.

Exemplos

```
setupvibe shell apply --dry-run   # plano dos blocos e arquivos
```

```
setupvibe shell apply   # aplica o ambiente e os aliases
```

```
setupvibe config set shell.agentAliases false && setupvibe shell apply   # deixa os aliases de agente de fora
```

```
setupvibe shell apply --json   # resultado em JSON para um agente
```

```
setupvibe shell apply && exec $SHELL -l   # aplica e recarrega o shell na sessão atual
```

`setupvibe shell remove`

Remove os blocos do SetupVibe de `~/.zshrc`, `~/.bashrc` e `~/.tmux.conf`, mantendo o restante dos arquivos.

Sintaxe

```
setupvibe shell remove [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e, na execução, o caminho do log. `data` traz `dryRun` e `steps` no plano; `log` e `results` na execução.

Efeitos persistentes

Apaga apenas o conteúdo entre os marcadores. Os arquivos `.local`, os backups e os itens instalados ficam.

Permissões

Nenhuma além da escrita nos arquivos do seu home.

Recusas e exit codes

Etapa com falha: `PARTIAL` (10).

Limites

Não desinstala pacotes, não apaga `~/.setupvibe/` e não desfaz a troca do shell de login. Para isso, use `self uninstall`.

Exemplos

```
setupvibe shell remove --dry-run   # mostra o que sairia dos arquivos
```

```
setupvibe shell remove   # tira os blocos e mantém o resto
```

```
setupvibe shell remove && exec $SHELL -l   # tira os blocos e recarrega o shell
```

```
setupvibe shell remove --json   # resultado em JSON
```

```
setupvibe shell remove && setupvibe shell apply   # reescreve o bloco do zero
```

`setupvibe migrate`

Move uma máquina configurada pelo SetupVibe 1 para o SetupVibe 2, preservando as configurações.

Sintaxe

```
setupvibe migrate [--dry-run] [--remove-legacy <items>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente
`--remove-legacy`	texto	não	-	`go`, `rbenv`, `npm-global`, `nodesource` ou `all`

Saída

Texto: o plano, as linhas que mudam de arquivo e as duplicidades mantidas. `data` traz `v1`, `moved`, `removeLegacy` e `keptLegacy`; no plano, `dryRun` e `steps`; na execução, `log` e `results`; `linuxbrewUninstall` aparece quando houver Linuxbrew.

Efeitos persistentes

Faz backup do `.zshrc` e do `.tmux.conf` antigos, move as linhas pessoais para os arquivos `.local`, escreve os blocos do SetupVibe 2, mantém `ssh_copy_id` como atalho, registra `migratedFromV1` em `config.json` e adota os itens já instalados em `selection.json`. Remove as duplicidades confirmadas.

Permissões

Em terminal, pergunta uma vez por duplicidade. Sem terminal, remove apenas os ids passados em `--remove-legacy`. Usa sudo quando a remoção exige.

Recusas e exit codes

Valor desconhecido em `--remove-legacy`: `USAGE` (2). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Etapa com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

O Linuxbrew nunca é removido, porque outras fórmulas podem depender dele; o texto mostra o comando de desinstalação. Segredos nas linhas movidas aparecem mascarados no plano. Sem vestígios do SetupVibe 1, o comando termina com 0 e não altera nada.

Exemplos

```
setupvibe migrate --dry-run   # plano, linhas que mudam e duplicidades encontradas
```

```
setupvibe migrate   # converte os arquivos e pergunta sobre cada duplicidade
```

```
setupvibe migrate --remove-legacy go,rbenv   # sem terminal: remove só essas duplicidades
```

```
setupvibe migrate --remove-legacy all   # sem terminal: remove todas as duplicidades
```

```
setupvibe migrate --json --dry-run   # plano completo em JSON
```

`setupvibe upgrade`

Substitui o binário pela release mais nova do canal, conferida pelo `SHA256SUMS` e, quando o cosign está instalado, pela assinatura Sigstore.

Sintaxe

```
setupvibe upgrade [--check] [--channel <channel>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--check`	flag	não	ausente	só informa se há versão nova
`--channel`	enum	não	`stable`	`stable`, `beta`

Saída

Texto: a versão atual, a nova e o resultado da verificação. `data` traz `current`, `latest`, `minimum`, `changelog`, `updateAvailable`, `updateRequired`, `checkedAt`, `source` e, quando troca, `upgraded`, `path` e `signature` (`verified`, `skipped` ou `invalid`).

Efeitos persistentes

Baixa o binário da release, confere o SHA-256 e substitui o executável no próprio caminho. Grava o cache da consulta de versão em `~/.setupvibe/state/update-check.json`.

Permissões

Precisa de permissão de escrita no diretório do binário. Não usa sudo por conta própria.

Recusas e exit codes

Falha de rede ou resposta inesperada: `NETWORK_OR_API` (5). Sem permissão para trocar o arquivo: `PERMISSION` (7). Execução a partir do código-fonte: `USAGE` (2). Plataforma sem binário na release: `UNSUPPORTED_SYSTEM` (11). Assinatura Sigstore inválida: `NETWORK_OR_API` (5).

Limites

Só troca binários compilados. A release precisa listar o arquivo da plataforma no `SHA256SUMS`. Sem cosign, a conferência fica no SHA-256 e o texto informa isso. O canal também pode vir de `config.json`.

Exemplos

```
setupvibe upgrade --check   # só informa se existe versão nova
```

```
setupvibe upgrade   # baixa, confere e substitui o binário
```

```
setupvibe upgrade --channel beta   # acompanha o canal beta
```

```
setupvibe upgrade --check --json   # estado da atualização para um script
```

```
sudo setupvibe upgrade   # quando o binário está em /usr/local/bin
```

`setupvibe skill`

Executa o skills CLI da Vercel fixado no inventory. Os argumentos vão depois de `--`.

Sintaxe

```
setupvibe skill [--] <args>...
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`args`	texto, variádico	não	-	argumentos do skills CLI

Saída

Texto: a saída do próprio skills CLI. `data` traz `command`, `exitCode` e, com `--json`, `output` com o texto capturado.

Efeitos persistentes

Dependem do subcomando do skills. Instalação e remoção de skills gravam nos diretórios do agente escolhido.

Permissões

As do skills CLI e do diretório de destino. Não exige root.

Recusas e exit codes

Falha do skills CLI: `GENERIC_ERROR` (1). Argumento inválido para o binário: `USAGE` (2).

Limites

Usa o item `skills` quando ele está instalado; sem ele, roda `npx skills@<versão fixada>`, que exige rede. O `--` é necessário para que `--json` e outras options cheguem ao skills. A documentação embutida do SetupVibe fica em `docs` e `--skill`.

Exemplos

```
setupvibe skill -- list   # lista as skills instaladas
```

```
setupvibe skill -- add promovaweb/herdrfy -g -a claude-code   # instala a skill para Claude Code
```

```
setupvibe skill -- add https://github.com/promovaweb/setupvibe/tree/v2/desktop/skills/setupvibe   # instala a skill que opera o binário
```

```
setupvibe skill -- update   # atualiza as skills instaladas
```

```
setupvibe skill -- list --json   # guarda a saída do skills no envelope
```

`setupvibe report`

Gera um relatório Markdown da máquina para anexar em uma issue do GitHub, sem segredos.

Sintaxe

```
setupvibe report [--output <file>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--output`	caminho	não	stdout	arquivo de saída

Saída

Texto: o Markdown. `data` traz `markdown`. Com `--output`, o texto é uma confirmação e o arquivo recebe o relatório.

Efeitos persistentes

Cria o arquivo indicado. Não altera o estado do SetupVibe.

Permissões

Nenhuma. Pode rodar como root. Não exige TTY.

Recusas e exit codes

Falha de escrita no arquivo: `PERMISSION` (7). Nenhuma outra recusa prevista.

Limites

O relatório inclui sistema, achados do doctor, itens, apps externos, último run e as últimas linhas do log, com valores de variáveis sensíveis mascarados. Ele não substitui o log completo quando o problema depende de detalhe.

Exemplos

```
setupvibe report   # imprime o relatório para copiar na issue
```

```
setupvibe report --output report.md   # grava o relatório em arquivo
```

```
setupvibe report --json   # entrega o Markdown no campo data
```

```
setupvibe report --output report.md && gh issue create --body-file report.md   # abre a issue com o relatório
```

```
setupvibe report | head -40   # lê só o começo, com sistema e versão
```

`setupvibe self`

Agrupa os comandos que gerenciam a própria instalação do SetupVibe.

Sintaxe

```
setupvibe self <uninstall> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte `self uninstall`.

Efeitos persistentes

O subcommand remove arquivos e configurações da máquina.

Permissões

Usa sudo para a regra em `/etc/sudoers.d/`.

Recusas e exit codes

Subcommand inválido: `USAGE` (2).

Limites

O grupo não tem ação própria. Use `self uninstall` com `--dry-run` antes de remover.

Exemplos

```
setupvibe self uninstall --dry-run   # plano de remoção sem alterar nada
```

```
setupvibe self uninstall --yes   # remove o SetupVibe e mantém backups
```

```
setupvibe self uninstall --yes --keep-data   # remove o binário e mantém
~/.setupvibe
```

```
setupvibe self uninstall --items --yes   # remove também os itens do espaço do
usuário
```

```
setupvibe self --help   # lista o subcommand disponível
```

`setupvibe self uninstall`

Remove o SetupVibe: blocos do shell, regra de sudo, binário e estado. Backups e volumes do Portainer ficam.

Sintaxe

```
setupvibe self uninstall [--items] [--keep-data] [--yes] [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--items`	flag	não	ausente	remove também os itens do espaço do usuário
`--keep-data`	flag	não	ausente	mantém `~/.setupvibe/`
`--yes`	flag	não	ausente	não pede confirmação (`-y`)
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e o resumo. `data` traz `dryRun` e `steps` no plano; `log` e `results` na execução.

Efeitos persistentes

Remove os blocos do shell, a regra `/etc/sudoers.d/90-setupvibe-<user>`, os binários encontrados e, sem `--keep-data`, `~/.setupvibe/`. Com `--items`, remove também os itens selecionados no espaço do usuário.

Permissões

Usa sudo para a regra de sudo e para pacotes. Sem terminal, exige `--yes`.

Recusas e exit codes

Sem `--yes` e sem terminal: `USAGE` (2). Resposta negativa: `INTERRUPTED` (130). Etapa com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

Backups dos seus arquivos, volumes do Portainer e restos do SetupVibe 1 ficam. `--keep-data` preserva estado, logs e backups. Os itens do espaço do usuário só saem com `--items`.

Exemplos

```
setupvibe self uninstall --dry-run   # mostra tudo o que seria removido
```

```
setupvibe self uninstall --yes   # remove o SetupVibe e mantém backups e Portainer
```

```
setupvibe self uninstall --yes --keep-data   # mantém o estado e os logs em
~/.setupvibe
```

```
setupvibe self uninstall --items --yes   # remove também os itens instalados no
espaço do usuário
```

```
setupvibe self uninstall --yes --json   # resultado da remoção em JSON
```

`setupvibe ssh`

Agrupa os utilitários de SSH.

Sintaxe

```
setupvibe ssh <copy-id> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte `ssh copy-id`.

Efeitos persistentes

O subcommand altera o `authorized_keys` do servidor remoto.

Permissões

As da sessão SSH de destino.

Recusas e exit codes

Subcommand inválido: `USAGE` (2).

Limites

O grupo não tem ação própria. A chave privada nunca sai da máquina.

Exemplos

```
setupvibe ssh copy-id deploy@203.0.113.10   # autoriza a chave padrão no servidor
```

```
setupvibe ssh copy-id deploy@host --port 2222   # usa uma porta SSH diferente
```

```
printf %s "$PASS" | setupvibe ssh copy-id root@host --password-stdin   # responde a senha pelo stdin
```

```
setupvibe ssh copy-id deploy@host --key ~/.ssh/id_ed25519.pub   # escolhe a chave pública
```

```
setupvibe ssh copy-id deploy@host --json   # resultado da cópia em JSON
```

`setupvibe ssh copy-id`

Instala a sua chave pública no servidor remoto uma vez, com modos de arquivo seguros.

Sintaxe

```
setupvibe ssh copy-id <target> [--port <port>] [--key <file>] [--password-stdin]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`target`	texto	sim	-	`user@host` ou `host`
`--port`	inteiro	não	`22`	1 a 65535
`--key`	caminho	não	primeira de `~/.ssh/id_ed25519.pub`, `id_ecdsa.pub`, `id_rsa.pub`	chave pública
`--password-stdin`	flag	não	ausente	lê a senha do stdin e usa `sshpass`

Saída

Texto: confirmação com o arquivo da chave e o destino. `data` traz `target`, `key` e `exitCode`.

Efeitos persistentes

Acrescenta a chave ao `authorized_keys` do servidor remoto. Na máquina local, nada muda.

Permissões

Precisa de acesso SSH ao destino. Com `--password-stdin`, exige `sshpass` instalado. A chave viaja pelo stdin, nunca pela linha de comando.

Recusas e exit codes

Destino inválido: `USAGE` (2). Chave pública ausente ou `sshpass` não instalado: `NOT_FOUND` (6). SSH recusou a conexão ou o comando: `GENERIC_ERROR` (1).

Limites

Instala uma chave por execução. Não cria chave nova; use `ssh-keygen -t ed25519`. Não altera a configuração do servidor além do `authorized_keys`.

Exemplos

```
setupvibe ssh copy-id deploy@203.0.113.10   # autoriza a chave padrão no destino
```

```
printf %s "$PASS" | setupvibe ssh copy-id root@host --password-stdin   # usa sshpass com a senha vinda do stdin
```

```
setupvibe ssh copy-id deploy@host --port 2222   # servidor com SSH em outra porta
```

```
setupvibe ssh copy-id deploy@host --key ~/.ssh/id_ed25519.pub   # escolhe a chave pública
```

```
setupvibe ssh copy-id deploy@host --json   # confirmação e exit code em JSON
```

`setupvibe user`

Cria o usuário de trabalho e gerencia a regra de sudo dele.

Sintaxe

```
setupvibe user <create|nopasswd> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte `user create` e `user nopasswd`.

Efeitos persistentes

Cria usuário, home, grupo de sudo e regra em `/etc/sudoers.d/`.

Permissões

Roda como root.

Recusas e exit codes

Subcommand inválido: `USAGE` (2).

Limites

O grupo não tem ação própria. Use `user create` antes do init na máquina nova.

Exemplos

```
setupvibe user create deploy --ssh-key ~/.ssh/id_ed25519.pub   # cria o usuário com a chave
```

```
setupvibe user create deploy --nopasswd   # cria com sudo sem senha para automação
```

```
setupvibe user nopasswd deploy   # acrescenta a regra de sudo sem senha
```

```
setupvibe user nopasswd deploy --remove   # tira a regra
```

```
setupvibe user create deploy --dry-run   # mostra as etapas sem criar o usuário
```

`setupvibe user create`

Cria o usuário de trabalho com sudo, uma chave SSH e, opcionalmente, a regra de sudo sem senha.

Sintaxe

```
setupvibe user create [name] [--ssh-key <key-or-file>] [--nopasswd] [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`name`	texto	não	pergunta no terminal	letras minúsculas, dígitos, `-` e `_`
`--ssh-key`	texto ou caminho	não	-	chave pública ou arquivo
`--nopasswd`	flag	não	ausente	presente ou ausente
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e o próximo passo, que é entrar como o usuário e rodar `init`. `data` traz `user`; no plano, `dryRun` e `steps`; na execução, `log`, `results` e `next`.

Efeitos persistentes

Cria o usuário, o home, o grupo de sudo (`wheel` no Arch e no Omarchy), o `authorized_keys` com a chave e, com `--nopasswd`, a regra `/etc/sudoers.d/90-setupvibe-<name>` validada com `visudo -cf`. Grava `user` e `nopasswd` em `config.json`.

Permissões

Exige root. Sem nome e com terminal, pergunta o nome. Não usa sudo: já roda como root.

Recusas e exit codes

Nome ausente sem terminal: `USAGE` (2). Nome inválido: `USAGE` (2). Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Etapa com falha: `PARTIAL` (10).

Limites

Cria um usuário por execução. A chave pode ser o texto da chave pública ou um arquivo; só a primeira linha do arquivo é usada. Depois de criar, o `init` roda como o usuário novo, não como root.

Exemplos

```
setupvibe user create deploy --ssh-key ~/.ssh/id_ed25519.pub   # cria com a chave do arquivo
```

```
setupvibe user create deploy --ssh-key "ssh-ed25519 AAAA..."   # cria com a chave no próprio argumento
```

```
setupvibe user create deploy --nopasswd   # cria com sudo sem senha para automação
```

```
setupvibe user create deploy --dry-run   # plano completo sem criar nada
```

```
setupvibe user create deploy --json   # resultado e próximo passo em JSON
```

`setupvibe user nopasswd`

Acrescenta ou remove a regra de sudo sem senha do SetupVibe para um usuário.

Sintaxe

```
setupvibe user nopasswd <name> [--remove] [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`name`	texto	sim	-	nome de usuário existente
`--remove`	flag	não	ausente	remove a regra
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e, na execução, o caminho do log. `data` traz `user`; no plano, `dryRun` e `steps`; na execução, `log` e `results`.

Efeitos persistentes

Cria ou apaga `/etc/sudoers.d/90-setupvibe-<name>`. A regra é validada com `visudo -cf` antes de entrar no lugar.

Permissões

Exige root ou sudo.

Recusas e exit codes

Etapa com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

Não cria usuário. Só mexe na regra do SetupVibe; outras regras em `/etc/sudoers.d/` ficam intactas.

Exemplos

```
setupvibe user nopasswd deploy   # acrescenta a regra sem senha
```

```
setupvibe user nopasswd deploy --remove   # remove a regra sem senha
```

```
setupvibe user nopasswd deploy --dry-run   # mostra a regra sem gravar
```

```
setupvibe user nopasswd deploy --remove --dry-run   # mostra a remoção sem gravar
```

```
setupvibe user nopasswd deploy --json   # resultado da regra em JSON
```

`setupvibe security`

Agrupa o módulo de segurança compartilhado. `apply` aplica firewall, fail2ban, atualizações automáticas, login SSH só por chave e sincronização de horário; `remove` desfaz o que foi aplicado.

Sintaxe

```
setupvibe security <apply|remove> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte `security apply` e `security remove`.

Efeitos persistentes

Altera UFW, fail2ban, atualizações automáticas, `/etc/ssh/sshd_config.d/10-setupvibe.conf`, NTP e o fuso, e grava `~/.setupvibe/state/security.json`.

Permissões

Usa sudo e exige root ou sudo sem senha em execução sem terminal.

Recusas e exit codes

Subcommand inválido: `USAGE` (2).

Limites

O módulo é aplicado pelo `auto` em servidor e em desktop exposto. Use `security apply` para aplicar sob demanda.

Exemplos

```
setupvibe security apply --dry-run   # mostra o plano do módulo
```

```
setupvibe security apply --timezone America/Sao_Paulo   # aplica com o fuso
escolhido
```

```
setupvibe security remove --dry-run   # mostra o que seria desfeito
```

```
setupvibe security remove   # desfaz o módulo, mantendo os pacotes
```

```
setupvibe security apply --json   # plano ou resultado em JSON
```

`setupvibe security apply`

Aplica o firewall, o fail2ban, as atualizações automáticas, o login SSH só por chave e a sincronização de horário.

Sintaxe

```
setupvibe security apply [--timezone <zone>] [--swarm] [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--timezone`	texto	não	fuso atual	zona IANA, como `America/Sao_Paulo`
`--swarm`	flag	não	ausente	server: abre as portas do Swarm
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada parte com ✓ ou ✗ e as partes puladas com o motivo. `data` traz `dryRun`, `skipped` e `steps` no plano; `log` e `results` na execução.

Efeitos persistentes

Configura UFW negando entrada exceto SSH (e portas do Swarm com `--swarm`), o jail `sshd` do fail2ban, atualizações automáticas no Debian e Ubuntu, NTP e fuso. Move o SSH para login só por chave quando o usuário tem chave autorizada. Grava o estado anterior em `~/.setupvibe/state/security.json` antes da primeira mudança.

Permissões

Exige root ou sudo. Em execução sem terminal, o sudo precisa funcionar sem senha.

Recusas e exit codes

Sistema sem suporte: `UNSUPPORTED_SYSTEM` (11). Parte com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

Sem chave autorizada, a parte de SSH é pulada e o fechamento informa isso; o SetupVibe nunca liga autenticação por senha. O primeiro registro de estado é mantido nas execuções seguintes. Containers que publicam porta em todas as interfaces passam pelo firewall do Docker; o `doctor` aponta esses casos.

Exemplos

```
setupvibe security apply --dry-run   # plano completo sem alterar nada
```

```
setupvibe security apply --timezone America/Sao_Paulo   # aplica com o fuso de São Paulo
```

```
setupvibe security apply --json   # partes puladas e resultados em JSON
```

```
setupvibe security apply --swarm   # servidor: abre também as portas do Swarm
```

```
setupvibe security apply --timezone UTC --dry-run | grep -i time   # confere só a parte de horário
```

`setupvibe security remove`

Desfaz o módulo de segurança a partir do estado registrado antes da aplicação. Os pacotes continuam instalados.

Sintaxe

```
setupvibe security remove [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e, na execução, o caminho do log. `data` traz `record`; no plano, `dryRun` e `steps`; na execução, `log` e `results`.

Efeitos persistentes

Apaga o drop-in de SSH depois de `sshd -t`, o jail do fail2ban e a configuração de atualizações automáticas. Desliga UFW e fail2ban quando estavam desligados antes. Apaga o registro `~/.setupvibe/state/security.json`.

Permissões

Exige root ou sudo.

Recusas e exit codes

Sem registro: `CONFIG` (3). Parte com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

Arquivos que já existiam antes do SetupVibe ficam. Os pacotes não são removidos. Sem o registro, o comando não tenta adivinhar o estado anterior.

Exemplos

```
setupvibe security remove --dry-run   # mostra o que seria desfeito
```

```
setupvibe security remove   # desfaz o módulo e apaga o registro
```

```
setupvibe security remove --json   # resultado em JSON
```

```
setupvibe security remove; echo "exit=$?"   # confere o code quando não há registro
```

```
setupvibe security remove && setupvibe doctor   # remove e confere o estado da máquina
```

`setupvibe about`

Mostra versão, executável, runtime, plataforma e credits.

Sintaxe

```
setupvibe about [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Saída

Texto: título, descrição e as tabelas `Binary`, `Project` e `Credits`. `data` traz `bin`, `title`, `version`, `description`, `author`, `site`, `repository`, `license`, `executable`, `runtime`, `platform`, `system`, `pid`, `dataDir` e `credits`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma. Pode rodar como root.

Recusas e exit codes

Nenhuma recusa prevista. Retorna 0.

Limites

`system` é um resumo de uma linha; para os detalhes, use `system`. O `pid` muda a cada execução.

Exemplos

```
setupvibe about   # versão, executável, runtime, plataforma e credits
```

```
setupvibe about --json   # identidade completa para um agente
```

```
setupvibe about | grep -i credit -A 10   # só a lista de credits
```

```
setupvibe about --json | jq -r '.data.executable'   # caminho do binário em uso
```

```
setupvibe about --json | jq -r '.data.version'   # versão para comparar com a
release
```

`setupvibe config`

Lê e grava preferências em `~/.setupvibe/settings.json`.

Sintaxe

```
setupvibe config <subcommand> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Consulte as seções de cada um.

Efeitos persistentes

`set`, `unset` e `import` gravam `settings.json` com modo 0600. `export` grava um arquivo de respostas quando recebe um caminho.

Permissões

Nenhuma. Chaves desconhecidas são aceitas como texto, para extensões de projeto.

Recusas e exit codes

JSON inválido no arquivo: `CONFIG` (3). Falha de leitura ou escrita: `PERMISSION` (7). Chave inexistente em `get`: `CONFIG` (3). Subcommand inválido: `USAGE` (2).

Limites

Segredos não entram em `settings.json`; use `credentials`. Keys usam notação de ponto.

Exemplos

```
setupvibe config list   # todas as chaves com o valor efetivo
```

```
setupvibe config get output.format   # valor de uma chave
```

```
setupvibe config set output.format json   # grava uma preferência
```

```
setupvibe config export setupvibe.json   # grava a seleção para outra máquina
```

```
setupvibe config import setupvibe.json --dry-run   # confere o arquivo antes de instalar
```

`setupvibe config list`

Lista cada chave com o valor efetivo, juntando o arquivo com os padrões.

Sintaxe

```
setupvibe config list [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

O alias `ls` tem o mesmo efeito.

Saída

Texto: o caminho do arquivo e uma tabela `chave valor`. `data` traz `file`, `exists` e `values`, com um objeto de chave para valor.

Efeitos persistentes

Nenhum. O arquivo não é criado.

Permissões

Nenhuma.

Recusas e exit codes

JSON inválido: `CONFIG` (3). Arquivo ilegível: `PERMISSION` (7).

Limites

O valor é o efetivo, não só o que está no arquivo. `exists` informa se o arquivo já foi criado.

Exemplos

```
setupvibe config list   # todas as chaves efetivas
```

```
setupvibe config list --json   # objeto values para um script
```

```
setupvibe config list | grep tui   # só as preferências da TUI
```

```
setupvibe config list --json | jq -r '.data.values["output.format"]'   # formato de saída efetivo
```

```
setupvibe config list --json | jq '.data.exists'   # confere se settings.json já existe
```

`setupvibe config get`

Mostra o valor de uma chave.

Sintaxe

```
setupvibe config get <key> [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`key`	texto	sim	-	chave em notação de ponto, como `output.format`

Saída

Texto: o valor. Strings saem sem aspas; números, booleanos e objetos saem em JSON. `data` traz `key` e `value`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma.

Recusas e exit codes

Chave inexistente: `CONFIG` (3), com a sugestão de `config schema`. JSON inválido: `CONFIG` (3). Arquivo ilegível: `PERMISSION` (7).

Limites

Devolve o valor efetivo, inclusive padrão. Não distingue valor gravado de valor padrão; para isso, leia o arquivo ou use `list --json` com `exists`.

Exemplos

```
setupvibe config get output.format   # formato de saída efetivo
```

```
setupvibe config get tui.theme   # paleta da TUI
```

```
setupvibe config get tui.initialTab --json   # chave e valor em JSON
```

```
setupvibe config get chave.inexistente; echo "exit=$?"   # confirma o exit code 3
```

```
setupvibe config get shell.agentAliases   # confere se os aliases de agente estão ligados
```

`setupvibe config set`

Grava um valor em `settings.json`, convertendo o tipo conforme o schema.

Sintaxe

```
setupvibe config set <key> <value> [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`key`	texto	sim	-	chave em notação de ponto
`value`	texto	sim	-	texto; booleanos aceitam `true`, `false`, `yes`, `no`, `1`, `0`, `on`, `off`

Saída

Texto: confirmação com o valor convertido e um aviso quando a chave está fora do schema. `data` traz `key`, `value`, `file` e `unknownKey`.

Efeitos persistentes

Cria ou atualiza `settings.json`, com modo 0600 e indentação de dois espaços. Chave fora do schema é gravada como texto.

Permissões

Nenhuma.

Recusas e exit codes

Valor fora do tipo ou da lista aceita: `CONFIG` (3). Falha de escrita: `PERMISSION` (7).

Limites

Não valida o efeito da preferência em outros arquivos. Depois de mudar `shell.agentAliases`, rode `shell apply`. `output.format = json` muda o formato padrão de todos os comandos.

Exemplos

```
setupvibe config set output.format json   # passa a usar JSON por padrão
```

```
setupvibe config set output.colors false   # desliga cores no texto
```

```
setupvibe config set shell.agentAliases false   # deixa os aliases de agente de fora no próximo shell apply
```

```
setupvibe config set tui.theme day   # troca a paleta da TUI
```

```
setupvibe config set minha.chave valor --json   # chave fora do schema, gravada como texto
```

`setupvibe config unset`

Remove uma chave do arquivo. A chave volta ao padrão, quando existe.

Sintaxe

```
setupvibe config unset <key> [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`key`	texto	sim	-	chave em notação de ponto

O alias `rm` tem o mesmo efeito.

Saída

Texto: confirmação ou aviso de que a chave não estava gravada. `data` traz `key` e `removed`.

Efeitos persistentes

Reescreve `settings.json` só quando algo foi removido.

Permissões

Nenhuma.

Recusas e exit codes

JSON inválido: `CONFIG` (3). Falha de escrita: `PERMISSION` (7).

Limites

Chave ausente não é erro: retorna 0 com `removed = false`. Para voltar ao padrão, remova a chave em vez de gravar o valor padrão.

Exemplos

```
setupvibe config unset telemetry.enabled   # volta ao padrão
```

```
setupvibe config unset output.format   # volta a usar texto por padrão
```

```
setupvibe config unset tui.theme --json   # confirma a remoção em JSON
```

```
setupvibe config unset minha.chave; echo "exit=$?"   # chave ausente também retorna 0
```

```
setupvibe config unset shell.agentAliases && setupvibe shell apply   # volta aos aliases padrão e reaplica
```

`setupvibe config path`

Imprime o diretório de estado e os caminhos dos arquivos.

Sintaxe

```
setupvibe config path [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Saída

Texto: tabela com `Directory`, `settings.json`, `credentials.json` e `master.key`. `data` traz `base`, `settings`, `credentials` e `masterKey`.

Efeitos persistentes

Nenhum. Não cria a pasta.

Permissões

Nenhuma.

Recusas e exit codes

Nenhuma recusa prevista.

Limites

Com `SETUPVIBE_HOME`, os caminhos apontam para a pasta alternativa. As duas edições usam `~/.setupvibe/`.

Exemplos

```
setupvibe config path   # diretório e arquivos de estado
```

```
setupvibe config path --json   # caminhos para um script
```

```
SETUPVIBE_HOME=/tmp/sv setupvibe config path   # confere o desvio de pasta
```

```
setupvibe config path --json | jq -r '.data.settings'   # caminho do settings.json
```

```
setupvibe config path --json | jq -r '.data.masterKey'   # caminho da chave mestra
```

`setupvibe config schema`

Lista as chaves conhecidas com tipo, padrão e descrição.

Sintaxe

```
setupvibe config schema [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Saída

Texto: tabela com chave, descrição, tipo e padrão. `data` traz `keys`, e cada chave traz `key`, `type`, `default`, `description` e `values` quando for enum.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma.

Recusas e exit codes

Nenhuma recusa prevista.

Limites

O schema cobre as chaves do SetupVibe. Projetos derivados podem ter chaves próprias; essas aparecem em `set` com `unknownKey = true`.

Exemplos

```
setupvibe config schema   # todas as chaves conhecidas
```

```
setupvibe config schema --json   # schema completo para validação
```

```
setupvibe config schema | grep -A 1 output.format   # detalhe de uma chave
```

```
setupvibe config schema --json | jq -r '.data.keys[].key'   # só os nomes das chaves
```

```
setupvibe config schema | grep enum   # chaves com lista fechada de valores
```

`setupvibe config export`

Grava a seleção e a identidade como arquivo de respostas para `config import` e `auto --config`.

Sintaxe

```
setupvibe config export [file] [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`file`	caminho	não	stdout	arquivo de saída

Saída

Sem arquivo, o JSON do arquivo de respostas. Com arquivo, uma confirmação com a quantidade de itens. `data` traz `schemaVersion`, `edition`, `user` quando houver, `git` quando houver, `profile`, `only` e `timezone` quando houver.

Efeitos persistentes

Cria o arquivo indicado. Não altera a máquina.

Permissões

Nenhuma.

Recusas e exit codes

Falha de escrita: `PERMISSION` (7).

Limites

O arquivo não leva segredos. A lista `only` traz os ids da seleção naquele momento; itens fora do inventory não entram. O campo `profile` é `none`; no `auto --config`, isso vale como sem profile e a seleção vem de `only`.

Exemplos

```
setupvibe config export   # imprime o arquivo de respostas em stdout
```

```
setupvibe config export setupvibe.json   # grava o arquivo para outra máquina
```

```
setupvibe config export --json   # envelope com o conteúdo em data
```

```
setupvibe config export | jq -r '.only[]'   # lista os ids exportados
```

```
setupvibe config export setupvibe.json && scp setupvibe.json deploy@host:   # leva a seleção para o servidor
```

`setupvibe config import`

Instala os itens e a identidade do Git de um arquivo exportado em outra máquina.

Sintaxe

```
setupvibe config import <file> [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`file`	caminho	sim	-	arquivo de `config export`
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada etapa e, na execução, o caminho do log. `data` traz `dryRun`, `items` e `skipped` no plano; `log` e `results` na execução, com `checks` quando houver falha.

Efeitos persistentes

Instala os itens do arquivo, grava a identidade do Git com `git config --global` e atualiza `selection.json`.

Permissões

Exige init concluído. Usa sudo quando o método exige.

Recusas e exit codes

JSON inválido ou arquivo com segredo: `USAGE` (2). Sem init: `CONFIG` (3). Etapa com falha: `PARTIAL` (10). Sudo com senha: `SUDO_PASSWORD` (13).

Limites

O arquivo precisa vir de `config export`; campos com `tailscaleAuthKey`, `ghToken`, `token` ou `swarmToken` são recusados. Apps instalados com `--external` não entram na importação.

Exemplos

```
setupvibe config import setupvibe.json --dry-run   # mostra o que seria instalado
```

```
setupvibe config import setupvibe.json   # instala os itens e a identidade do Git
```

```
setupvibe config import setupvibe.json --json   # resultado em JSON
```

```
setupvibe config import setupvibe.json 2>/dev/null; echo "exit=$?"   # confere o resultado em um script
```

```
setupvibe config import setupvibe.json && setupvibe check   # importa e verifica a seleção
```

`setupvibe credentials`

Agrupa o cofre de segredos cifrado com AES-256-GCM em `~/.setupvibe/credentials.json`.

Sintaxe

```
setupvibe credentials <subcommand> [options]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

O alias `cred` tem o mesmo efeito. Sem subcommand, imprime o help e sai com 0; com `--json`, o help vem em `data.help`.

Saída

Depende do subcommand. Nenhum subcommand devolve o valor completo sem `--reveal`.

Efeitos persistentes

Grava `credentials.json` e `master.key` com modo 0600. `clear` apaga o cofre; `rotate-key` troca a chave.

Permissões

Nenhuma além da escrita em `~/.setupvibe/`.

Recusas e exit codes

Cofre ilegível ou credencial ausente: `CREDENTIALS` (4). Falha de escrita: `PERMISSION` (7). Entrada cancelada: `INTERRUPTED` (130). Nome inválido: `USAGE` (2).

Limites

A chave vem de `SETUPVIBE_MASTER_KEY` ou de `master.key`. Os comandos que chamam API leem com `getCredential(name)` e saem com 4 quando o valor não existe. Nenhuma tool MCP devolve o valor de uma credencial.

Exemplos

```
setupvibe credentials set openai.api_key   # grava sem eco no terminal
```

```
setupvibe credentials get openai.api_key   # mostra o valor mascarado
```

```
setupvibe credentials list   # nomes salvos, sem valores
```

```
setupvibe credentials remove github.token   # apaga uma credencial
```

```
setupvibe credentials status   # arquivo, origem da chave e quantidade
```

`setupvibe credentials set`

Grava ou substitui uma credencial. O valor é lido sem eco ou pelo stdin.

Sintaxe

```
setupvibe credentials set <name> [--stdin]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`name`	texto	sim	-	letras minúsculas, dígitos, ponto, hífen e `_`, até 64 caracteres
`--stdin`	flag	não	ausente	lê a primeira linha do stdin mesmo em terminal

Saída

Texto: confirmação com o caminho do cofre. `data` traz `name`, `file` e `saved`.

Efeitos persistentes

Cria ou atualiza `credentials.json`. Cria `master.key` com modo 0600 na primeira gravação, quando não há chave no ambiente.

Permissões

Exige terminal para digitar sem eco; sem terminal ou com `--stdin`, lê a primeira linha do stdin.

Recusas e exit codes

Nome inválido: `USAGE` (2). Valor vazio no stdin: `USAGE` (2). Ctrl+C: `INTERRUPTED` (130). Falha de chave ou de cofre: `CREDENTIALS` (4). Falha de escrita: `PERMISSION` (7).

Limites

Não aceita o valor por argumento, para não deixar o segredo no histórico e na lista de processos. Uma credencial por execução. O valor é gravado como texto e cifrado no arquivo.

Exemplos

```
setupvibe credentials set openai.api_key   # pede o valor sem eco
```

```
printf '%s' "$TOKEN" | setupvibe credentials set github.token --stdin   # valor vindo de uma variável
```

```
setupvibe credentials set openai.api_key   # repete o nome para substituir o valor
```

```
printf '%s\n' "$SECRET" | setupvibe credentials set api.token --stdin   # primeira linha do stdin
```

```
setupvibe credentials set openai.api_key --json   # confirmação em JSON
```

`setupvibe credentials get`

Mostra uma credencial mascarada. Use `--reveal` para o valor completo.

Sintaxe

```
setupvibe credentials get <name> [--reveal] [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`name`	texto	sim	-	nome da credencial
`--reveal`	flag	não	ausente	imprime o valor completo

Saída

Sem `--reveal`, texto com a máscara e `data` com `name`, `masked` e `length`. Com `--reveal`, texto com o valor e `data` com `name` e `value`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma, mas o valor completo aparece no terminal e no histórico de saída.

Recusas e exit codes

Nome inválido: `USAGE` (2). Credencial ausente: `CREDENTIALS` (4). Cofre ilegível: `CREDENTIALS` (4).

Limites

A máscara mostra poucos caracteres. `--reveal` em um agente grava o segredo no log da sessão; prefira consumir o valor direto no comando que precisa dele. Não existe opção para copiar para a área de transferência.

Exemplos

```
setupvibe credentials get openai.api_key   # valor mascarado
```

```
setupvibe credentials get openai.api_key --reveal   # valor completo no terminal
```

```
setupvibe credentials get openai.api_key --json   # nome, máscara e tamanho
```

```
setupvibe credentials get credencial.ausente; echo "exit=$?"   # confirma o exit code 4
```

```
TOKEN=$(setupvibe credentials get github.token --reveal) && curl -H "Authorization: Bearer $TOKEN" https://api.github.com/user   # usa o valor sem imprimir
```

`setupvibe credentials list`

Lista os nomes salvos. Nunca mostra valores.

Sintaxe

```
setupvibe credentials list [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

O alias `ls` tem o mesmo efeito.

Saída

Texto: um nome por linha, ou o aviso de cofre vazio. `data` traz `count` e `names`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma.

Recusas e exit codes

Cofre ilegível: `CREDENTIALS` (4).

Limites

Lista apenas nomes. Para saber a origem da chave, use `status`. Cofre inexistente devolve lista vazia e code 0.

Exemplos

```
setupvibe credentials list   # nomes salvos
```

```
setupvibe credentials list --json   # quantidade e nomes em JSON
```

```
setupvibe credentials list | wc -l   # quantas credenciais existem
```

```
for name in $(setupvibe credentials list); do echo "$name"; done   # percorre os nomes
```

```
setupvibe credentials list --json | jq -r '.data.names[]'   # nomes para um script
```

`setupvibe credentials remove`

Remove uma credencial do cofre.

Sintaxe

```
setupvibe credentials remove <name> [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`name`	texto	sim	-	nome da credencial

O alias `rm` tem o mesmo efeito.

Saída

Texto: confirmação ou aviso de que a credencial não existia. `data` traz `name` e `removed`.

Efeitos persistentes

Reescreve `credentials.json` sem a credencial.

Permissões

Nenhuma além da escrita no cofre.

Recusas e exit codes

Nome inválido: `USAGE` (2). Falha de cofre: `CREDENTIALS` (4). Falha de escrita: `PERMISSION` (7).

Limites

Nome ausente não é erro: retorna 0 com `removed = false`. A chave mestra não é trocada.

Exemplos

```
setupvibe credentials remove github.token   # apaga a credencial
```

```
setupvibe credentials remove github.token --json   # confirma o que saiu
```

```
setupvibe credentials remove antiga.chave; echo "exit=$?"   # nome ausente retorna 0
```

```
setupvibe credentials remove github.token && setupvibe credentials list   # confere a lista depois
```

```
setupvibe credentials remove github.token 2>/dev/null || true   # remove sem falhar quando já não existe
```

`setupvibe credentials status`

Mostra o arquivo, a origem da chave mestra e a quantidade de credenciais.

Sintaxe

```
setupvibe credentials status [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
(nenhum)	-	-	-	-

Saída

Texto: tabela com `File`, `Cipher`, `Master key` e `Credentials`, seguida dos nomes. `data` traz `file`, `exists`, `masterKeySource`, `count`, `names`, `masterKeyVariable` e `algorithm`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma.

Recusas e exit codes

Nenhuma recusa prevista. Cofre ilegível aparece como lista vazia no texto, sem apagar nada.

Limites

`masterKeySource` é `env`, `file` ou `none`. O status não decifra os valores, só tenta ler os nomes.

Exemplos

```
setupvibe credentials status   # arquivo, cifra, chave e quantidade
```

```
setupvibe credentials status --json   # estado completo para um agente
```

```
setupvibe credentials status | grep Master   # só a origem da chave
```

```
setupvibe credentials status --json | jq -r '.data.masterKeySource'   # env, file ou
none
```

```
setupvibe credentials status --json | jq '.data.count'   # quantas credenciais
existem
```

`setupvibe credentials clear`

Apaga `credentials.json` por inteiro. Mantém `master.key`. Pede confirmação em terminal.

Sintaxe

```
setupvibe credentials clear [--yes]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--yes`	flag	não	ausente	não pede confirmação (`-y`)

Saída

Texto: confirmação ou aviso de que não havia cofre. `data` traz `file` e `deleted`.

Efeitos persistentes

Apaga o cofre cifrado. A chave mestra e os demais arquivos ficam.

Permissões

Em terminal, pede confirmação. Sem terminal e sem `--yes`, considera a resposta negativa.

Recusas e exit codes

Resposta negativa ou sem `--yes` fora de terminal: `INTERRUPTED` (130). Falha de escrita: `PERMISSION` (7).

Limites

Não apaga `master.key`, então um cofre novo usará a mesma chave. Para trocar a chave, use `rotate-key`. Não há como recuperar os valores apagados.

Exemplos

```
setupvibe credentials clear   # pergunta antes de apagar
```

```
setupvibe credentials clear --yes   # apaga sem perguntar
```

```
setupvibe credentials clear --yes --json   # confirma o que foi apagado
```

```
setupvibe credentials clear --yes && setupvibe credentials status   # confere que o cofre sumiu
```

```
setupvibe credentials clear; echo "exit=$?"   # sem terminal e sem --yes, confirma o 130
```

`setupvibe credentials rotate-key`

Recifra o cofre com uma chave mestra nova e grava `master.key` com modo 0600.

Sintaxe

```
setupvibe credentials rotate-key [--from-env]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--from-env`	flag	não	ausente	usa `SETUPVIBE_MASTER_KEY` como chave nova

Saída

Texto: confirmação. `data` traz `rotated` e `source` (`env` ou `file`).

Efeitos persistentes

Substitui `master.key` e recifra `credentials.json`. Com `--from-env`, a chave atual é lida de `master.key` e a nova vem da variável.

Permissões

Nenhuma além da escrita no cofre.

Recusas e exit codes

Cofre inexistente: `CREDENTIALS` (4). Variável ausente ou fora do formato: `CREDENTIALS` (4). Falha de escrita: `PERMISSION` (7).

Limites

A variável precisa ter 64 caracteres hexadecimais. Sem `--from-env`, a chave nova é aleatória e a variável antiga deve ser removida do shell. Em caso de falha, o cofre original é preservado.

Exemplos

```
setupvibe credentials rotate-key   # gera uma chave nova e recifra o cofre
```

```
setupvibe credentials rotate-key --from-env   # usa a chave da variável
```

```
setupvibe credentials rotate-key && unset SETUPVIBE_MASTER_KEY   # remove a variável antiga da sessão
```

```
setupvibe credentials rotate-key 2>/dev/null; echo "exit=$?"   # sem cofre, confirma o code 4
```

```
setupvibe credentials rotate-key --json   # origem da chave nova em JSON
```

`setupvibe mcp`

Inicia o servidor Model Context Protocol via stdio, com tools, resources e prompts para agentes.

Sintaxe

```
setupvibe mcp [--list]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--list`	flag	não	ausente	só lista o que o servidor registra

Saída

Com `--list`, texto com `Tools`, `Resources` e `Prompts`, e `data` com `name`, `version`, `tools`, `resources` e `prompts`. Sem `--list`, o processo fala JSON-RPC em stdin e stdout até o cliente fechar, e o envelope final traz `data = { "closed": true }`.

Efeitos persistentes

Nenhum. O servidor só lê o estado e executa tools quando o cliente pede.

Permissões

As mesmas do binário. Tools que instalam ou removem itens usam sudo e exigem init, como no CLI.

Recusas e exit codes

Falha ao iniciar ou perda do transporte: `MCP` (9). Tool com falha devolve erro no JSON-RPC, sem derrubar o servidor.

Limites

Nada além de JSON-RPC pode ir para stdout; diagnósticos vão para stderr. O cliente configura `{ "command": "setupvibe", "args": ["mcp"] }`. Nenhuma tool devolve o valor de uma credencial.

Exemplos

```
setupvibe mcp   # inicia o servidor para o cliente MCP
```

```
setupvibe mcp --list   # lista tools, resources e prompts
```

```
setupvibe mcp --list --json   # catálogo completo para um agente
```

```
setupvibe mcp install   # configura o servidor nos agentes instalados
```

```
setupvibe mcp --list | sed -n '/Tools:/,/Resources:/p'   # só as tools
```

`setupvibe mcp install`

Configura o servidor MCP do SetupVibe no Claude Code, no Codex e no OpenCode.

Sintaxe

```
setupvibe mcp install [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: cada agente configurado, os pulados e o caminho do log. `data` traz `dryRun`, `skipped` e `steps` no plano; `log` e `results` na execução.

Efeitos persistentes

No Claude Code, `claude mcp add --scope user`. No Codex, `codex mcp add`. No OpenCode, o bloco `mcp.setupvibe` em `~/.config/opencode/opencode.json`, com backup em `.setupvibe.bak` antes da primeira alteração. O restante do arquivo é preservado.

Permissões

Nenhuma além da escrita nos arquivos dos agentes. Não exige TTY.

Recusas e exit codes

Etapa com falha: `PARTIAL` (10).

Limites

Agente não instalado é pulado, e o texto informa qual. Rodar de novo não muda o que já está configurado. Não instala os agentes; use `install` para isso.

Exemplos

```
setupvibe mcp install   # configura nos agentes encontrados
```

```
setupvibe mcp install --dry-run   # mostra os comandos e arquivos
```

```
setupvibe mcp install --json   # agentes pulados e resultados em JSON
```

```
setupvibe mcp install 2>&1 | grep not   # só os agentes que não estão instalados
```

```
setupvibe mcp install && claude mcp get setupvibe   # confere o registro no Claude Code
```

`setupvibe docs`

Imprime a documentação embutida, inteira ou por topic. Tem o mesmo efeito de `--skill`.

Sintaxe

```
setupvibe docs [topic] [--lang <language>]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`topic`	enum	não	tudo	`cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory`, `reference`
`--lang`	enum	não	`SETUPVIBE_LANG`, depois `LANG`	`pt-BR`, `en`

Saída

Texto: o Markdown. `data` traz `topic` (`all` sem argumento) e `markdown`.

Efeitos persistentes

Nenhum.

Permissões

Nenhuma. Pode rodar como root.

Recusas e exit codes

Topic inválido: `USAGE` (2). Idioma fora da lista: `USAGE` (2).

Limites

Todos os topics valem em `--skill`, em `docs` e na tool MCP `skill`. `troubleshooting` traz o guia de diagnóstico e `migration` o guia de migração do SetupVibe 1. `inventory` gera a lista a partir do inventory embutido. `reference` traz apenas a referência do registry, com commands, arguments, options e exit codes. O texto embutido corresponde à versão do binário.

Exemplos

```
setupvibe docs   # toda a documentação embutida
```

```
setupvibe docs cli   # só o manual do CLI
```

```
setupvibe docs mcp --json   # uma seção no envelope, para um agente
```

```
setupvibe docs troubleshooting --lang en   # guia de diagnóstico em inglês
```

```
setupvibe docs inventory > inventory.md   # grava a lista de itens e versões
```

`setupvibe doctor`

Confere o runtime, o estado do SetupVibe, os pré-requisitos pendentes, os itens e os restos do SetupVibe 1. Com `--fix`, aplica as correções seguras.

Sintaxe

```
setupvibe doctor [--fix] [--repair-windows] [--dry-run]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`--fix`	flag	não	ausente	aplica as correções seguras
`--repair-windows`	flag	não	ausente	Windows: DISM `/RestoreHealth` e `sfc /scannow`
`--dry-run`	flag	não	ausente	presente ou ausente

Saída

Texto: seções `Runtime`, `SetupVibe` e, com `--fix`, `Applied`. Cada linha usa ✓, ! ou ✗. `data` traz `ok`, `runtime`, `findings` e, com `--fix`, `applied`. Cada verificação tem `name`, `status` e `detail`; cada achado tem `id`, `level`, `text` e `fix` quando houver.

Efeitos persistentes

Sem `--fix`, nenhum. Com `--fix`, cria `~/.local/bin`, ajusta `~/.setupvibe/` para 0700 e escreve o bloco do shell. Antes de escrever o bloco, converte os arquivos do SetupVibe 1, como `auto` e `shell apply`. Com `--repair-windows`, executa DISM e `sfc`.

Permissões

Sem `--fix`, roda como usuário comum e pode rodar como root. Com `--fix`, pode usar sudo no bloco do shell. `--repair-windows` exige PowerShell elevado.

Recusas e exit codes

Verificação com `fail`: `CONFIG` (3). Reparo no Windows chamado em outro sistema: `UNSUPPORTED_SYSTEM` (11). Etapa do reparo com falha: `PARTIAL` (10).

Limites

O doctor não cria arquivos nem mostra segredos sem `--fix`. TTY ausente e arquivos ainda não criados geram `warn`, não `fail`. Ele aponta containers que publicam porta em todas as interfaces, item resolvido por outro gerenciador e reinício pendente. Não corrige item com falha; isso é `install --force`.

Exemplos

```
setupvibe doctor   # diagnóstico completo em texto
```

```
setupvibe doctor --fix   # aplica as correções seguras e mostra as que exigem escolha
```

```
setupvibe doctor --json   # verificações e achados em JSON
```

```
setupvibe doctor --repair-windows   # Windows elevado: repara o component store e os arquivos
```

```
setupvibe doctor --json | jq '.data.findings[] | select(.level=="fail")'   # só os achados com falha
```

`setupvibe completion`

Imprime um script de completion gerado pelo registry para bash, zsh, fish ou PowerShell.

Sintaxe

```
setupvibe completion <shell> [--json]
```

Argumentos e options

NOME	TIPO	OBRIGATÓRIO	PADRÃO	VALORES
`shell`	enum	sim	-	`bash`, `zsh`, `fish`, `powershell`

Saída

Texto: o script. `data` traz `shell` e `script`.

Efeitos persistentes

Nenhum. O script só entra no shell quando você o carrega ou acrescenta a linha ao arquivo de inicialização.

Permissões

Nenhuma. Pode rodar como root.

Recusas e exit codes

Shell fora da lista: `USAGE` (2), com os valores aceitos.

Limites

O script cobre commands, subcommands, options locais e globais e valores de enum do registry. Uma pessoa que muda de versão precisa gerar o script de novo. O Zsh usa `compdef` e `compadd`, sem `bashcompinit`.

Exemplos

```
eval "$(setupvibe completion bash)"   # completion na sessão Bash atual
```

```
eval "$(setupvibe completion zsh)"   # completion na sessão Zsh atual
```

```
setupvibe completion fish | source   # completion na sessão Fish atual
```

```
setupvibe completion powershell | Out-String | Invoke-Expression   # completion no PowerShell
```

```
setupvibe completion zsh > ~/.zsh/completions/_setupvibe   # arquivo para carregar depois
```

Apêndice B. Configuração e estado

Tudo que o `setupvibe` persiste fica em `~/.setupvibe/`. A pasta é criada com 0700 e os arquivos de dados são gravados com 0600. O `setupvibe` e o `setupvibe-server` usam a mesma pasta e o mesmo prefixo `SETUPVIBE_`, porque só um deles fica instalado por máquina.

Arquivos em `~/.setupvibe/`

CAMINHO	O QUE GUARDA	QUEM ESCREVE
`settings.json`	preferências em texto claro; keys em notação de ponto	`config set`, `config unset`, a TUI quando você troca o idioma, o tema, o mouse ou os ícones
`credentials.json`	envelope AES-256-GCM com os segredos	`credentials set`, `credentials remove`, `credentials clear`, `credentials rotate-key`
`master.key`	32 bytes em hexadecimal, criados na primeira gravação do cofre	`credentials set` e `credentials rotate-key`
`config.json`	usuário, sistema detectado, registro do `init`, identidade do Git, canal de release, `network` e `migratedFromV1`	`init`, `auto`, `user create`, `migrate`; o `self uninstall` apaga
`selection.json`	itens escolhidos, versão instalada e status do último check	`install`, `remove`, `update`, `auto`, `init`
`external.json`	apps instalados pelo mise fora do inventário	`install --external`, `remove --external`
`last-run.json`	resultado completo do último `auto`, passo por passo	`auto`
`run.lock`	PID da execução que está alterando a máquina	`auto`
`logs/<data>-<command>.log`	um evento NDJSON por linha, com segredos mascarados	todo command que executa um plano
`backups/`	cópia de um arquivo do usuário antes da primeira mudança e antes de cada migração de schema	`shell apply`, `migrate`, `readState`
`state/update-check.json`	cache de seis horas da checagem de versão	toda execução que anuncia versão nova

CAMINHO	O QUE GUARDA	QUEM ESCREVE
`state/security.json`	estado de UFW, fail2ban e dos arquivos antes do módulo de segurança	`security apply`
`shell/aliases.sh`, `shell/aliases-agents.sh`, `shell/tmux.conf`	arquivos carregados por `~/.zshrc`, `~/.bashrc` e `~/.tmux.conf`	`shell apply`
`tmp/`	instalador ou script baixado durante a execução	métodos `installer` e `archive`

Restos do SetupVibe 1 também aparecem nessa pasta e não são apagados sem confirmação: `bin/ssh_copy_id`, `portainer-compose.yml` e, no Windows, `windows-utilities.json`. O `setupvibe migrate` trata os dois primeiros; o `setupvibe doctor` lista todos.

`config.json`

O arquivo é estado, não preferência. Use os commands; o campo `channel` é a exceção e hoje só muda editando o arquivo.

CAMPO	CONTEÚDO
`schemaVersion`	versão do formato; um binário antigo recusa um arquivo mais novo e pede `upgrade`
`user`	usuário de trabalho criado ou adotado
`edition`	edição que rodou o `init`: `desktop` ou `server`
`system`	sistema detectado (`ubuntu`, `debian`, `mint`, `zorin`, `arch`, `omarchy`, `macos`, `windows` ou `wsl`)
`init`	`done`, `at` e a versão que concluiu o `init`
`git`	`name` e `email` usados quando o Git global estava vazio
`channel`	canal lido pelo `upgrade` e pelo aviso de versão: `stable` ou `beta`
`network`	`caFile` e `cacheDir` fixos da máquina, ver Rede e cache
`migratedFromV1`	data e edição de origem gravadas pelo `migrate`
`shell`	lugar antigo do interruptor `agentAliases`; o `settings.json` é lido primeiro

CAMPO	CONTEÚDO
`nopasswd`	`true` quando `user create --nopasswd` criou a regra de sudo

O bloco `network` evita repetir opção em toda execução. Grave-o no `~/.setupvibe/config.json`; não há command que escreva esse campo.

```
{ "network": { "caFile": "/etc/ssl/certs/corp.pem", "cacheDir": "/srv/setupvibe-cache" } }
```

`settings.json`

O schema vem de `SETTINGS_SCHEMA` em `shared/core/config/settings.ts`. Uma key fora do schema é aceita como texto, para projetos derivados.

KEY	TIPO	PADRÃO	VALORES ACEITOS	EFEITO
`output.format`	enum	`text`	`text`, `json`	formato padrão da saída do CLI quando `--json` não é passado
`output.colors`	boolean	`true`	`true`, `false`	cores ANSI na saída de texto; `NO_COLOR` sempre desliga
`tui.initialTab`	enum	`overview`	`overview` (Início), `inventory`, `user`, `doctor`, `logs`, `tailscale`, `machine`, `about`	módulo aberto quando a TUI inicia; `--tab` tem prioridade
`tui.language`	enum	`pt-BR`	`pt-BR`, `en`	idioma da TUI; a tecla `L` e o seletor `PT · EN` gravam aqui
`tui.icons`	boolean	`true`	`true`, `false`	ícones da Nerd Font; `false` troca por símbolos que toda fonte tem
`tui.mouse`	boolean	`true`	`true`, `false`	clique em cartões, itens, botões e teclas, e roda para rolar

KEY	TIPO	PADRÃO	VALORES ACEITOS	EFEITO
`tui.theme`	enum	`night`	`night`, `day`	paleta da TUI; a tecla `T` e o ícone no cabeçalho trocam e gravam aqui
`shell.agentAliases`	boolean	`true`	`true`, `false`	escreve `cco`, `cxo`, `oco`, `cx` e `oc`, que abrem os agentes sem confirmação; rode `setupvibe shell apply` depois
`telemetry.enabled`	boolean	`false`	`true`, `false`	reservado para projetos derivados; o binário não envia nada

Para boolean, o `config set` aceita `true`/`false`, `yes`/`no`, `1`/`0` e `on`/`off`. Para enum, aceita só um valor da coluna de valores. Para number, aceita um número finito. Qualquer outro texto devolve exit code 3 (`CONFIG`).

Exemplos de `config set`

```
setupvibe config set output.format json                # scripts que já leem --json
setupvibe config set tui.theme day                     # terminal com fundo claro
setupvibe config set tui.mouse false                   # sem mouse, só teclado
setupvibe config set shell.agentAliases false          # sem aliases que pulam
confirmação
setupvibe config set output.colors off                 # saída sem ANSI em um pipe
```

O resultado é `✓ key = valor`. Uma key fora do schema é gravada como texto e a saída avisa `(key outside the schema, stored as text)`.

Exemplos de `config get`

```
setupvibe config get tui.language                       # pt-BR
setupvibe config get tui.initialTab --json              #
{"key":"tui.initialTab","value":"overview"}
setupvibe config get output.format                      # text, quando nunca foi
gravado
setupvibe config get shell.agentAliases                 # true
setupvibe config get editor.preferred                   # exit 3 quando a key não
existe
```

O `get` lê o arquivo já misturado com os padrões. Por isso um valor aparece mesmo quando `settings.json` ainda não existe.

Exemplos de `config unset`

```
setupvibe config unset output.format                    # remove o json gravado; o
CLI volta a text
setupvibe config unset tui.theme                        # remove o day gravado;
volta a night
setupvibe config unset telemetry.enabled                # remove a key reservada;
volta ao padrão false
setupvibe config unset shell.agentAliases               # os aliases voltam ligados
setupvibe config unset editor.preferred                 # exit 0, avisa que não
estava gravada
setupvibe config unset editor.preferred --json          #
{"key":"editor.preferred","removed":false}
```

O `unset` remove só a key gravada no arquivo. A key continua existindo com o padrão do schema.

Os outros subcommands

SUBCOMMAND	O QUE FAZ	SAÍDA
`config list` (alias `ls`)	lista cada key com o valor efetivo, do arquivo misturado com os padrões	caminho do arquivo e tabela `key` `valor`
`config path`	mostra a pasta de dados e o caminho de cada arquivo	`Directory`, `settings.json`, `credentials.json`, `master.key`
`config schema`	lista as keys conhecidas com tipo, padrão e descrição	tabela gerada de `SETTINGS_SCHEMA`

SUBCOMMAND	O QUE FAZ	SAÍDA
`config export [arquivo]`	grava a seleção e a identidade do Git no formato de respostas; sem arquivo, imprime	JSON com `schemaVersion`, `edition`, `user`, `git`, `profile` e `only`
`config import <arquivo>`	instala os itens e a identidade do arquivo exportado	mesmo resultado de `install`, mais o Git global

```
setupvibe config path
setupvibe config schema --json
setupvibe config export selecao.json
setupvibe config import selecao.json --dry-run
setupvibe config list
```

O arquivo de respostas não carrega segredo. O `profile` sai como `none`, que o `auto --config` lê como base vazia: entram só os itens da lista `only` e os requisitos deles. A lista fica ordenada, para o diff entre duas máquinas ser legível. `config import` aplica a mesma seleção no lugar.

Para dar tipo, padrão e descrição a uma key nova, acrescente-a a `SETTINGS_SCHEMA`. O `config set`, o `config get` e o `config schema` passam a conhecer a key sem outra mudança.

O servidor MCP expõe `config_list` e `config_get` para leitura; nenhuma tool grava `settings.json`.

`credentials.json`

O arquivo guarda um envelope:

```
{ "version": 1, "algorithm": "aes-256-gcm", "iv": "<base64>", "tag": "<base64>",
"data": "<base64>" }
```

`data` é o JSON `{ "name": "value" }` cifrado. Names seguem `[a-z0-9][a-z0-9_.-]{0,63}`, por exemplo `openai.api_key` ou `github.token`.

A master key tem 32 bytes e vem, nesta ordem:

1. da variável `SETUPVIBE_MASTER_KEY` (64 caracteres hexadecimais);
2. do arquivo `master.key`, gerado automaticamente.

Gere uma chave para a variável com `openssl rand -hex 32`. Quando a variável está definida, `master.key` é ignorado.

SUBCOMMAND	O QUE FAZ
`credentials set <name>`	grava um valor; sem `--stdin`, digita sem eco
`credentials get <name>`	mostra o valor mascarado; `--reveal` mostra em claro
`credentials list`	lista os names, sem valores
`credentials remove <name>`	apaga um name
`credentials status`	mostra a origem da chave e quantos names existem
`credentials clear --yes`	apaga o cofre inteiro
`credentials rotate-key`	gera chave nova, recifra o cofre e grava `master.key`

```
setupvibe credentials set openai.api_key                # pergunta o valor sem eco
printf '%s' "$TOKEN" | setupvibe credentials set github.token --stdin
setupvibe credentials get github.token                  # abc1******et
setupvibe credentials get openai.api_key --reveal       # valor em claro, para um
script
setupvibe credentials status --json                     # file, masterKeySource,
count e names
setupvibe credentials remove github.token
setupvibe credentials clear --yes
```

A tool MCP `credentials_status` mostra arquivo, origem da chave e nomes, sem valor. Um name inexistente, cofre ilegível ou chave ausente devolve exit code 4 (`CREDENTIALS`). Falha de permissão no arquivo devolve 7 (`PERMISSION`) e um prompt cancelado com Ctrl+C devolve 130 (`INTERRUPTED`). Numa máquina nova, `credentials status` responde `(not created)` e `Credentials 0`, sem criar arquivo; o cofre só nasce na primeira gravação.

O que a cifra protege

A cifra protege o arquivo contra leitura casual, cópia em backup e varredura do disco por outras ferramentas. Quem tem acesso à mesma conta do sistema consegue ler `master.key` e decifrar o conteúdo. Para um nível acima, injete a chave por `SETUPVIBE_MASTER_KEY` a partir de um gerenciador de segredos e não grave `master.key` em disco.

Rotação da master key

COMMAND OU OPTION	EFEITO
`credentials rotate-key`	gera chave aleatória, recifra o cofre e grava `master.key` com 0600
`credentials rotate-key --from-env`	lê a chave atual de `master.key` e usa `SETUPVIBE_MASTER_KEY` como chave nova

O `credentials rotate-key` abre o cofre com a chave atual e preserva os valores ao recifrar. A opção `--from-env` exige a chave atual em disco e uma chave nova de 64 caracteres hexadecimais na variável. A chave nova também é gravada em `master.key` com 0600. Sem cofre ou chave atual, o command devolve exit code 4. Em JSON, devolve `{ rotated, source }`, sem o conteúdo das chaves.

Sem `--from-env`, se a chave anterior veio da variável, remova essa variável do shell após a rotação: `unset SETUPVIBE_MASTER_KEY`. Ela tem prioridade sobre o arquivo nas próximas execuções. Não execute gravações no cofre durante a rotação. Os dois arquivos são preparados na pasta `.rotate-*` antes da substituição; falhas comuns de gravação restauram a chave anterior por substituição atômica. Se essa restauração também falhar, os arquivos de recuperação continuam na pasta `.rotate-*` indicada pelo erro, e a cópia `previous.key` mantém permissão 0600. Uma interrupção do processo entre as duas substituições também exige restauração por essa cópia.

Usar uma credencial no código

```
import { getCredential } from "../config/credentials.ts";

const token = getCredential("github.token"); // lança AppError(4) quando não existe
```

Nunca escreva o valor em log, em erro, em `data` do `--json` nem em resposta de tool MCP.

Variáveis de ambiente

Dados, idioma e interface

VARIÁVEL	EFEITO
`SETUPVIBE_HOME`	troca `~/.setupvibe/` por outra pasta, incluindo os arquivos de estado
`SETUPVIBE_LANG`	fixa o idioma em `pt-BR` ou `en`; vale para a TUI e para `--skill`

VARIÁVEL	EFEITO
`SETUPVIBE_ICONS`	`0` troca os ícones da Nerd Font da TUI por símbolos comuns
`NO_COLOR`	qualquer valor não vazio desliga as cores na saída de texto
`LANG`, `LC_ALL`, `LC_MESSAGES`	escolhem o idioma da documentação quando `SETUPVIBE_LANG` não está definida
`LC_CTYPE`, `LANG`	o `init` e o `auto` avisam quando o locale não é UTF-8

Rede e cache

VARIÁVEL	EFEITO
`HTTPS_PROXY`, `HTTP_PROXY`, `NO_PROXY`, `ALL_PROXY`	proxy lido em maiúsculas e minúsculas, inclusive pelos comandos que rodam com `sudo`
`SETUPVIBE_CA_FILE`	bundle de CA corporativa; o binário exporta para `NODE_EXTRA_CA_CERTS`, `SSL_CERT_FILE`, `CURL_CA_BUNDLE`, `REQUESTS_CA_BUNDLE`, `GIT_SSL_CAINFO` e `PIP_CERT`
`SETUPVIBE_CACHE_DIR`	pasta única de downloads; o binário exporta `MISE_CACHE_DIR`, `npm_config_cache`, `UV_CACHE_DIR` e `HOMEBREW_CACHE`
`SETUPVIBE_VERBOSE`	`1` mostra a saída completa de cada comando, igual a `--verbose`
`NODE_EXTRA_CA_CERTS`	definida pelo binário a partir de `SETUPVIBE_CA_FILE`; não precisa preencher
`MISE_CACHE_DIR`, `npm_config_cache`, `UV_CACHE_DIR`, `HOMEBREW_CACHE`	definidas pelo binário a partir de `SETUPVIBE_CACHE_DIR`

Respostas do `auto`

VARIÁVEL	RESPOSTA
`SETUPVIBE_USER`	usuário de trabalho
`SETUPVIBE_SSH_KEY`	chave pública ou arquivo com ela
`SETUPVIBE_GIT_NAME`, `SETUPVIBE_GIT_EMAIL`	identidade do Git
`SETUPVIBE_PROFILE`	perfil do inventário

VARIÁVEL	RESPOSTA
`SETUPVIBE_ONLY`, `SETUPVIBE_EXCLUDE`	itens a acrescentar ou deixar de fora, separados por vírgula
`SETUPVIBE_TIMEZONE`	fuso horário
`SETUPVIBE_REBOOT`	`1` reinicia e retoma o `auto`
`SETUPVIBE_KEEP_NOPASSWD`	`1` mantém a regra de sudo criada pelo `auto`
`SETUPVIBE_PM2_AUTOSTART`	`1` sobe o PM2 e os apps salvos no boot
`SETUPVIBE_TAILSCALE_SSH`	`1` liga o Tailscale SSH depois de entrar na tailnet
`TAILSCALE_AUTHKEY`	chave de autenticação do Tailscale
`GH_TOKEN`, `GITHUB_TOKEN`	token herdado pelas ferramentas que o leem, como o mise; o log mascara

As respostas valem quando a opção correspondente não foi passada. Segredos nunca entram em `argv`: o token do Tailscale e o do Swarm chegam ao comando por variável e o log substitui o valor por `***`.

Respostas do servidor

O `setupvibe-server` lê as mesmas variáveis e mais quatro, descritas na documentação dele: `SETUPVIBE_SWARM_MANAGER`, `SETUPVIBE_SWARM_TOKEN`, `SETUPVIBE_SWARM_ADVERTISE_ADDR` e `SETUPVIBE_SWARM_ENCRYPTED`.

Internas e do sistema

VARIÁVEL	QUEM LÊ
`SETUPVIBE_AUTO_CREATED_NOPASSWD`	o segundo processo do `auto`, iniciado com `runuser`, para saber que a regra de sudo é temporária
`SSHPASS`	`ssh copy-id` usa como senha do servidor remoto quando `--password-stdin` não é passado
`HOME`, `USERPROFILE`	pasta do usuário em Unix e no Windows
`USER`, `LOGNAME`, `USERNAME`	nome do usuário quando `id -un` não responde
`SHELL`, `ComSpec`	shell mostrado no módulo Máquina
`PATH`, `Path`	procura de comandos; o `doctor` e a TUI usam para achar `~/.local/bin`
`LOCALAPPDATA`	pasta do mise no Windows

VARIÁVEL	QUEM LÊ
`ProgramData`, `PROGRAMDATA`	estado do `desktop.ps1` que o `migrate` reconhece no Windows
`WSL_DISTRO_NAME`, `WSL_INTEROP`	detecção de WSL
`DEBIAN_FRONTEND`, `NEEDRESTART_MODE`, `NONINTERACTIVE`, `CI`	definidas pelo binário em cada comando do plano, para que instaladores não abram prompt

Rede e cache

Proxy, CA corporativa e cache valem para todo download e para todo comando executado pelo plano. A ordem é a opção, depois a variável, depois o bloco `network` do `config.json`.

```
setupvibe auto --ca-file /etc/ssl/certs/corp.pem --cache-dir /srv/setupvibe-cache
setupvibe install node --verbose
setupvibe update --ca-file /etc/ssl/certs/corp.pem
```

Com o cache quente, um download fixado no inventário instala sem rede. O `--verbose` mostra cada `argv` e a saída completa, sem mascarar o que o próprio comando imprime. Para investigar uma falha de rede, comece pelo `doctor`: ele roda as checagens do runtime e mostra o que está faltando.

Apêndice C. Códigos de saída

Zero é sucesso. Cada família de falha tem um número fixo. Scripts e agentes devem tratar o code antes de interpretar a mensagem.

CODE	NAME	COMANDOS QUE DEVOLVEM	O QUE FAZER
0	`SUCCESS`	todos, quando a operação termina	seguir para o próximo passo
1	`GENERIC_ERROR`	todos, quando uma falha escapa sem classificação; `skill` quando o CLI externo falha; `init` e `auto` quando o relógio está mais de cinco minutos fora; `ssh copy-id` quando o `ssh` remoto falha	leia a mensagem e o `report`; sem causa clara, rode `setupvibe report` e anexe numa issue
2	`USAGE`	arguments, options ou subcommand inválidos; o binário sem argumentos fora de um TTY, que imprime a ajuda; `init` e `auto` quando a outra edição já está configurada; `migrate` com id desconhecido em `--remove-legacy`; `upgrade` a partir do código-fonte	confira `--help` do command e o valor passado
3	`CONFIG`	`settings.json` ausente, inválido ou com key desconhecida; `install` e `update` antes do `init`; `security remove` sem registro; `doctor` com achado de nível fail; `config import` quando a instalação não pode continuar	leia o campo indicado, corrija o arquivo de estado ou rode o command que a mensagem pede

CODE	NAME	COMANDOS QUE DEVOLVEM	O QUE FAZER
4	`CREDENTIALS`	`credentials` e seus subcommands, quando o cofre está ilegível, a master key falta ou o name não existe	grave a credencial com `setupvibe credentials set <name>` ou recupere a chave
5	`NETWORK_OR_API`	`init` e `auto` quando `github.com` não responde; `upgrade` quando `version.json`, o binário, o `SHA256SUMS` ou a assinatura não conferem	veja proxy, CA e DNS na documentação de problemas, no repositório
6	`NOT_FOUND`	`config` e seus subcommands quando um caminho do arquivo não existe; `ssh copy-id` sem chave pública; `install --external` com ferramenta desconhecida; `remove --external` de app que não foi instalado assim	crie o arquivo, escolha outra key ou instale a ferramenta primeiro
7	`PERMISSION`	`config`, `credentials` e `upgrade` quando o arquivo ou o binário não pode ser lido, gravado ou substituído	corrija o dono e o modo do caminho indicado
8	`NO_TTY`	`tui`, quando stdin ou stdout não são um terminal	rode no terminal, ou use o CLI e `--json`
9	`MCP`	`mcp`, quando o servidor não inicia ou perde o transporte stdio	confira a configuração do cliente MCP e o stderr do servidor

CODE	NAME	COMANDOS QUE DEVOLVEM	O QUE FAZER
10	`PARTIAL`	`init`, `auto`, `install`, `remove`, `update`, `check`, `shell apply`, `shell remove`, `migrate`, `self uninstall`, `user create`, `user nopasswd`, `security apply`, `security remove`, `config import`, `mcp install`, `doctor --repair-windows` quando parte dos passos falhou e o resto ficou aplicado	leia o log da execução; a mensagem final traz o caminho
11	`UNSUPPORTED_SYSTEM`	`system`, `init`, `auto`, `list`, `install`, `remove`, `update`, `check`, `shell apply`, `migrate`, `user create`, `security apply`; `upgrade` quando não existe binário para a plataforma	confira `setupvibe system`; a edição ou a arquitetura não tem suporte
12	`ROOT`	`init` e `auto` quando rodam como root sem `--user`	crie o usuário com `setupvibe user create <nome>` ou rode `auto --user <nome>`
13	`SUDO_PASSWORD`	`auto` e `migrate` quando o `sudo` pede senha e não há terminal	rode no terminal, ou crie a regra com `setupvibe user nopasswd <nome>`
14	`DISK_SPACE`	`init` e `auto` quando a pasta do usuário tem menos de 10 GB livres no desktop ou 5 GB no servidor	libere espaço e rode de novo
20	`REBOOT_PENDING`	`auto` quando existe reinício pendente e `--reboot` não foi passado	reinicie e rode `auto` de novo, ou passe `--reboot`
30	`LOCKED`	`auto` quando outra execução segura `~/.setupvibe/run.lock`	espere a outra execução terminar; confira o PID do lock

CODE	NAME	COMANDOS QUE DEVOLVEM	O QUE FAZER
130	`INTERRUPTED`	qualquer command interrompido com Ctrl+C; `remove`, `self uninstall` e `credentials` quando você recusa a confirmação	nada foi concluído; rode de novo quando quiser

A fonte única é `shared/core/app/exit-codes.ts`. Ela alimenta a seção de exit codes de cada `--help`, o `--opencli`, a tool MCP `exit_codes` e a resource `setupvibe://exit-codes`.

Por command

Cada command declara seus codes em `--opencli`, além de 0, 1 e 2, que valem para todos.

COMMAND	CODES DECLARADOS
`tui`	8
`system`	11
`init`	5, 10, 11, 12, 14
`auto`	5, 10, 11, 12, 13, 14, 20, 30
`list`	11
`install`	3, 6, 10, 11
`remove`	6, 10, 11, 130
`update`	3, 10, 11
`check`	10, 11
`shell apply`	10, 11
`shell remove`	10
`migrate`	10, 11, 13
`upgrade`	5, 7, 11
`self uninstall`	10, 130
`ssh copy-id`	6
`user create`	10, 11

COMMAND	CODES DECLARADOS
`user nopasswd`	10
`security apply`	10, 11
`security remove`	3, 10
`config` e subcommands	3, 6, 7
`config import`	3, 10
`credentials` e subcommands	4, 7, 130
`mcp`	9
`mcp install`	10
`doctor`	3, 10

Alguns caminhos devolvem um code que a lista do command não declara: `upgrade` devolve 2 quando roda a partir do código-fonte, `config import` devolve 2 com arquivo ilegível e `self uninstall` devolve 2 sem `--yes` fora de um terminal. Trate o code observado, não só a lista.

As tools MCP que recusam uma execução real sem confirmação devolvem `exitCode: 2` no resultado da tool, sem iniciar processo. `doctor`, `check`, `install` e `update` marcam `isError` com os dados no envelope quando o code não é zero.

Em shell

Leia o code numa variável própria. Negar o comando com `!` perde o valor real, porque `$?` passa a ser o resultado da negação.

```
setupvibe credentials get github.token --json > /tmp/saida.json
code=$?
case $code in
  0) echo "token lido" ;;
  4) echo "grave a credencial com: setupvibe credentials set github.token" ;;
  *) echo "falhou com $code; veja /tmp/saida.json" ;;
esac
```

Um planejamento que aceita falha parcial testa 10 antes de 1:

```
setupvibe auto --json --user deploy
code=$?
if [ "$code" -eq 10 ]; then
  echo "parte dos itens falhou; o log está em ~/.setupvibe/logs"
elif [ "$code" -ne 0 ]; then
  echo "auto parou com $code"
fi
```

No `--json`

Um erro sai no envelope, com code, nome e detalhes:

```
{ "ok": false, "command": "setupvibe credentials get", "error": { "code": 4, "name": "CREDENTIALS", "message": "Credential \"github.token\" not found." } }
```

O `data` do envelope nunca carrega valor de credencial. Leia `error.code` antes de `error.message`, porque a mensagem pode mudar entre versões.

No código

```
import { error } from "../app/errors.ts";

throw error.network("A API respondeu 503.", { status: 503 }); // encerra com 5
```

Para uma família nova, acrescente a constante e a description em `shared/core/app/exit-codes.ts`, um atalho em `shared/core/app/errors.ts` e o code em `exitCodes` de cada command que o devolve. Não reutilize números.

Apêndice D. MCP

`setupvibe mcp` transforma o processo em um servidor Model Context Protocol sobre stdio. Nada além do protocolo é escrito em stdout; diagnósticos vão para stderr. O mesmo executável reúne TUI, CLI e servidor MCP.

O servidor expõe 20 tools, 6 resources e 1 prompt. As tools que rodam o próprio CLI devolvem o mesmo envelope do `--json`, para o agente ver o que uma pessoa veria no terminal. Tools que alteram a máquina fazem dry run por padrão; `remove` e `migrate` também exigem `confirm: true`.

Como o servidor responde

Uma tool devolve `content[0].text` com JSON e, quando a saída é estruturada, `structuredContent` com o mesmo objeto.

As tools que rodam o CLI devolvem o envelope `{ exitCode, ok, command, data }` ou `{ exitCode, ok: false, error }`. As tools nativas (`about`, `exit_codes`, `skill`, `config_list`, `config_get`, `credentials_status`, `health`, `logs`, `tailscale`, `machine`) devolvem o objeto descrito em cada seção.

`isError: true` aparece quando o exit code do CLI não é zero ou quando a tool recusa a chamada. O `data` pode vir junto: `doctor` com achado fail devolve os achados e marca `isError`, com exit code 3. Leia `exitCode` antes de `isError`.

Nenhuma tool devolve o valor de uma credencial. Nenhuma tool grava `settings.json`; escrita de configuração continua no CLI, onde há confirmação humana. As tools que alteram a máquina (`install`, `remove`, `update`, `init`, `migrate`) fazem dry run por padrão.

Antes de sugerir um command, leia `skill` com o tópico `cli`. Antes de chamar uma tool que instala ou remove, rode o dry run e mostre o plano à pessoa.

Configuração no cliente

```
{ "mcpServers": { "setupvibe": { "command": "setupvibe", "args": ["mcp"] } } }
```

No OpenCode, o bloco fica na configuração do aplicativo:

```
{ "mcp": { "setupvibe": { "type": "local", "command": ["setupvibe", "mcp"],
"enabled": true } } }
```

`setupvibe mcp install` registra o servidor no Claude Code (`claude mcp add --scope user`), no Codex (`codex mcp add`) e no OpenCode (bloco `mcp` de `~/.config/opencode/opencode.json`, com cópia de segurança). Agente que não está instalado fica de fora, e rodar de novo não muda o que já está configurado.

Para desenvolvimento, sem compilar, use `bun` com o entry point da edição no lugar do binário.

Listar sem iniciar

```
setupvibe mcp --list
setupvibe mcp --list --json
```

A lista mostra nome e descrição de cada tool, o URI de cada resource e o prompt, na ordem em que o servidor registra. É a mesma lista que este documento descreve.

Tools

TOOL	O QUE FAZ	ALTERA A MÁQUINA
`about`	identidade e runtime do binário	não
`exit_codes`	tabela de exit codes	não
`skill`	documentação embutida, inteira ou por tópico	não
`config_list`	keys de settings.json em vigor e o schema	não
`config_get`	valor de uma key	não
`credentials_status`	nomes e origem da master key, sem valores	não
`system`	sistema, arquitetura, WSL, headless, IP público e variante	não
`list`	itens do inventário com o status neste sistema	não
`check`	itens instalados contra as versões fixadas	não
`install`	instala itens e dependências	só com `dry_run: false`
`remove`	remove itens do espaço do usuário	só com `dry_run: false` e `confirm: true`

TOOL	O QUE FAZ	ALTERA A MÁQUINA
`update`	atualiza itens cuja versão fixada mudou	só com `dry_run: false`
`health`	só os achados do SetupVibe do doctor	não
`doctor`	runtime e achados do SetupVibe, com a correção	não
`report`	relatório em Markdown para uma issue	não
`init`	pré-requisitos do SetupVibe	só com `dry_run: false`
`migrate`	migra uma máquina do SetupVibe 1	só com `dry_run: false` e `confirm: true`
`logs`	execuções e eventos de uma delas	não
`tailscale`	cliente local do Tailscale	não
`machine`	sistema, CPU, memória, discos, rede e processos	não

Tool `about`

Finalidade. Identifica o binário em execução: nome, edição, versão, runtime, plataforma, pasta de dados e créditos.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `bin`, `title`, `version`, `description`, `author`, `site`, `repository`, `license`, `executable`, `runtime`, `platform`, `system`, `pid`, `dataDir` e `credits`, uma lista com `name`, `role` e `url`.

Efeitos e permissões. Só leitura. Não grava arquivo nem consulta a rede. `pid` e `executable` são do processo do servidor MCP, não do cliente que chama a tool.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Confirmar a edição:** `about {}` devolve `bin: "setupvibe"` e o `dataDir` da sessão, o que mostra qual binário atende a sessão.
- **Conferir a versão:** `about {}` devolve `version`, útil antes de sugerir uma flag que só existe na versão nova.

- **Ver a plataforma:** `about {}` devolve `platform: "linux-x64"`, o que explica por que um item existe em `list` e outro não.
- **Checar a pasta de dados:** `about {}` devolve o `dataDir` efetivo, inclusive quando a sessão roda com `SETUPVIBE_HOME` apontando para outro lugar.
- **Coletar créditos:** `about {}` devolve `credits` com o papel e a URL de cada projeto usado pelo binário.

Tool `exit_codes`

Finalidade. Devolve a tabela fixa de exit codes do CLI, com nome simbólico e descrição.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `codes`, uma lista com `code`, `name` e `description` de cada código, em ordem.

Efeitos e permissões. Só leitura. A tabela vem do código compilado, sem ler arquivo.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Explicar uma falha parcial:** `exit_codes {}` mostra que 10 é `PARTIAL` e que o resto do plano ficou aplicado.
- **Orientar sobre sudo:** `exit_codes {}` mostra que 13 é `SUDO_PASSWORD` e que a regra sem senha resolve.
- **Investigar concorrência:** `exit_codes {}` mostra que 30 é `LOCKED`, com outra execução segurando `run.lock`.
- **Tratar interrupção:** `exit_codes {}` mostra que 130 é `INTERRUPTED` e que nada foi concluído.
- **Montar a tabela do agente:** `exit_codes {}` devolve os 18 codes para o agente decidir sem interpretar texto.

Tool `skill`

Finalidade. Entrega a documentação embutida no binário em Markdown, inteira ou por tópico.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`topic`	string	não	tudo	`cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory` ou `reference`

Saída. Markdown em `content[0].text`. A tool não devolve `structuredContent`.

Efeitos e permissões. Só leitura. A documentação já está compilada no binário, então a resposta sai sem rede.

Recusas e limites. Um tópico fora da lista é recusado pelo schema, antes de a tool rodar.

Exemplos.

- **Aprender os commands:** `skill { "topic": "cli" }` devolve o manual do CLI com a referência gerada do registry.
- **Entender as teclas:** `skill { "topic": "tui" }` devolve os módulos, as teclas por módulo e o tema.
- **Investigar uma falha:** `skill { "topic": "troubleshooting" }` devolve sintoma, causa, comando e exit code de cada situação.
- **Migrar uma máquina antiga:** `skill { "topic": "migration" }` devolve o guia do SetupVibe 1.
- **Descobrir ids de itens:** `skill { "topic": "inventory" }` devolve perfis e uma tabela por categoria.

Tool `config_list`

Finalidade. Lista as preferências de `settings.json` já misturadas com os padrões, mais o schema que o binário conhece.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `file` (caminho), `values` (key achatada para o valor efetivo) e `schema`, uma lista com `key`, `type`, `default`, `values` quando enum e `description`.

Efeitos e permissões. Só leitura. Não cria `settings.json`; numa máquina sem o arquivo, `values` traz os padrões.

Recusas e limites. Nenhuma recusa própria. Um arquivo inválido vira erro da tool ao ler o JSON.

Exemplos.

- **Ver o formato efetivo:** `config_list {}` mostra `output.format` com o valor em vigor, não o que está escrito no arquivo.
- **Conferir a paleta:** `config_list {}` mostra `tui.theme` e o padrão `night` no schema.
- **Listar os padrões:** `config_list {}` mostra o valor de cada key mesmo numa máquina sem `settings.json`.
- **Ler o schema:** `config_list {}` devolve `values` e `default` de cada key, para o agente validar uma escolha.

- **Localizar o arquivo:** `config_list {}` devolve `file`, útil quando `SETUPVIBE_HOME` aponta para outro lugar.

Tool `config_get`

Finalidade. Devolve o valor efetivo de uma key de `settings.json`.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`key`	string	sim	nenhum	Key em notação de ponto, como `output.format`; mínimo de 1 caractere

Saída. `key`, `value` e `exists`. Quando a key não existe, `value` é `null` e `exists` é `false`.

Efeitos e permissões. Só leitura. Não cria o arquivo e não altera a key.

Recusas e limites. Key desconhecida devolve `exists: false` com `isError: true`; não há exit code de CLI nesse caminho. A tool lê só `settings.json`, não `config.json`.

Exemplos.

- **Ler o formato de saída:** `config_get { "key": "output.format" }` devolve `text` quando nada foi gravado.
- **Ler o idioma:** `config_get { "key": "tui.language" }` devolve `pt-BR` ou `en`.
- **Tratar key inexistente:** `config_get { "key": "editor.preferred" }` devolve `exists: false` e marca `isError`.
- **Conferir ícones:** `config_get { "key": "tui.icons" }` devolve `true` ou `false`.
- **Conferir aliases:** `config_get { "key": "shell.agentAliases" }` devolve o interruptor dos aliases dos agentes.

Tool `credentials_status`

Finalidade. Mostra o estado do cofre: arquivo, existência, origem da master key e nomes guardados, sem valores.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `file`, `exists`, `masterKeySource` (`env`, `file` ou `none`), `count` e `names`, uma lista de nomes.

Efeitos e permissões. Só leitura. Não cria o cofre nem a chave. Nunca devolve o valor de uma credencial.

Recusas e limites. Nenhuma recusa própria. Um cofre ilegível aparece com `count: 0` e nomes vazios, sem expor o conteúdo.

Exemplos.

- **Ver se o cofre existe:** `credentials_status {}` devolve `exists: false` numa máquina nova.
- **Conferir a origem da chave:** `credentials_status {}` devolve `masterKeySource: "env"` quando a chave vem da variável.
- **Contar credenciais:** `credentials_status {}` devolve `count` e `names`, sem nenhum valor.
- **Conferir nomes:** `credentials_status {}` mostra `github.token` ou `openai.api_key` quando já foram gravados.
- **Investigar cofre vazio:** `credentials_status {}` mostra `count: 0` depois de `credentials clear`, com `exists: true`.

Tool `system`

Finalidade. Descreve a máquina detectada: sistema, versão, arquitetura, WSL, headless, IP público, variante e traços do SetupVibe 1.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. Envelope do CLI com `data.edition`, `system`, `id`, `name`, `version`, `arch`, `wsl`, `headless`, `publicIp`, `variant`, `distro` quando houver e `setupvibe1`, com `found`, `edition` e `traces`.

Efeitos e permissões. Só leitura. Roda `system --no-update-check`, então não imprime aviso de versão nem altera arquivo.

Recusas e limites. Um sistema sem suporte devolve exit code 11 (`UNSUPPORTED_SYSTEM`) e a tool marca `isError`.

Exemplos.

- **Decidir compatibilidade:** `system {}` devolve `system: "debian"` e `arch: "x64"` antes de sugerir um item.
- **Conferir WSL:** `system {}` devolve `wsl: true` quando a distribuição roda dentro do WSL.
- **Ver máquina sem interface:** `system {}` devolve `headless: true` e explica por que itens gráficos saem da seleção.
- **Escolher o tipo de Portainer:** `system {}` devolve `variant: "remote-exposed"` quando a máquina tem IP público.
- **Detectar o SetupVibe 1:** `system {}` devolve `setupvibe1.found` e a lista de `traces`, antes de sugerir `migrate`.

Tool `list`

Finalidade. Lista os itens do inventário da edição com o status de cada um nesta máquina.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`category`	string	não	todas	Id de uma categoria, como `ai`, `languages` ou `security`

Saída. Envelope com `data.system` e `data.items`, cada item com `id`, `name`, `category`, `version`, `default`, `status` e `elsewhere` quando o comando vem de outro gerenciador.

Efeitos e permissões. Só leitura. Não instala, não remove e não altera `selection.json`.

Recusas e limites. Sistema sem suporte devolve exit code 11. Uma categoria desconhecida devolve a lista vazia, sem erro.

Exemplos.

- **Ver o inventário inteiro:** `list {}` devolve todos os itens da edição com o status atual.
- **Focar nos agentes de IA:** `list { "category": "ai" }` devolve só a categoria `ai`.
- **Encontrar itens de outro gerenciador:** `list {}` mostra `status: "other"` e `elsewhere` com o caminho da cópia atual.
- **Separar o que falta:** `list {}` mostra `available`, `failed` e `outdated`, que o agente usa para escolher o próximo passo.
- **Alimentar um install:** `list { "category": "languages" }` devolve ids válidos para a tool `install`.

Tool `check`

Finalidade. Confere itens instalados contra a versão fixada no inventário, rodando o check de cada um.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`items`	array de string	não	a seleção	Ids de itens, cada um no formato `[a-z0-9][a-z0-9-]*`

Saída. Envelope com `data.system` e `data.results`, cada resultado com `id`, `status` (`ok` ou `fail`), `output` e `expected` quando o item declara a versão esperada.

Efeitos e permissões. Só leitura. Executa o comando de verificação de cada item e não instala nem corrige nada.

Recusas e limites. Item desconhecido devolve exit code 2 (`USAGE`). Um item que falha faz o CLI sair com 10 (`PARTIAL`) e a tool marcar `isError`, mesmo com os resultados no `data`.

Exemplos.

- **Conferir a seleção:** `check {}` devolve um resultado por item selecionado.
- **Conferir um item:** `check { "items": ["node"] }` mostra a versão encontrada e a esperada.
- **Achar versão errada:** `check { "items": ["node", "uv"] }` devolve `status: "fail"` no item fora da versão.
- **Explicar a falha:** `check { "items": ["docker"] }` devolve `output` e `expected` para mostrar à pessoa o que difere.
- **Antes de atualizar:** `check { "items": ["node", "docker"] }` confirma o que está fora antes de `update`.

Tool `install`

Finalidade. Instala itens do inventário e as dependências deles, ou uma ferramenta do mise fora do inventário com `external: true`, com plano por padrão.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`items`	array de string	sim	nenhum	Ids de itens, ou uma ferramenta do mise com `external: true`; mínimo de 1
`dry_run`	boolean	não	`true`	`true` devolve o plano; `false` instala
`force`	boolean	não	`false`	`true` reinstala itens que já passam no check
`external`	boolean	não	`false`	`items` leva uma ferramenta do mise fora do inventário, como `terraform`; a versão é fixada e gravada em `external.json`

Saída. Envelope com `data.dryRun` e `data.steps` (id, description, commands, action) no plano, ou `data.results` e `data.log` na execução real. Com `external`, devolve `tool` e `version`. Sem `init`, devolve `data.init: false`.

Efeitos e permissões. Com `dry_run: true`, nenhum arquivo muda. Com `false`, instala no espaço do usuário, pode usar `sudo` e rede, grava o log e atualiza `selection.json`; com `external`, grava a ferramenta e a versão em `external.json`.

Recusas e limites. Sem `init` devolve exit code 3 (`CONFIG`). Id desconhecido devolve 2 (`USAGE`) e ferramenta que o mise não conhece devolve 6 (`NOT_FOUND`). `sudo` sem senha devolve 13 (`SUDO_PASSWORD`), item sem suporte devolve 11 e falha parcial devolve 10. Todos marcam `isError`. Com `external: true`, o schema aceita só `tool` sem versão; `tool@version` e nome com ponto são recusados pela validação, então use o CLI quando a versão exata importa.

Exemplos.

- **Planejar antes de instalar:** `install { "items": ["node"], "dry_run": true }` devolve os passos sem alterar a máquina.
- **Instalar dois itens:** `install { "items": ["node", "uv"], "dry_run": false }` instala e devolve `results`.
- **Reinstalar um item quebrado:** `install { "items": ["docker"], "dry_run": false, "force": true }` refaz mesmo com o check passando.
- **Descobrir o bloqueio:** `install { "items": ["node"], "dry_run": false }` antes do `init` devolve exit 3 com `data.init: false`.
- **Instalar uma ferramenta externa:** `install { "items": ["terraform"], "external": true, "dry_run": true }` mostra o plano do mise para a versão mais recente, que fica em `external.json`.

Tool `remove`

Finalidade. Remove itens instalados no espaço do usuário ou um app instalado com `install --external`, com plano por padrão e confirmação obrigatória na execução real.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`items`	array de string	sim	nenhum	Ids de itens, ou um app com `external: true`; mínimo de 1
`dry_run`	boolean	não	`true`	`true` devolve o plano; `false` remove

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`confirm`	boolean	não	`false`	A pessoa confirmou a remoção; obrigatório com `dry_run: false`
`include_native`	boolean	não	`false`	Também remove pacotes do gerenciador do sistema
`force`	boolean	não	`false`	Remove mesmo quando outro item selecionado exige o item, ou quando o `init` encontrou o item já instalado
`external`	boolean	não	`false`	`items` leva um app instalado com `install --external`

Saída. Envelope com `data.steps` no plano ou `data.results` na execução real. A recusa local devolve `{ exitCode: 2, ok: false, error }`.

Efeitos e permissões. Com `dry_run: true`, nada muda. Com `false` e `confirm: true`, remove o que foi instalado pelo SetupVibe, atualiza `selection.json` e grava o log; com `external`, remove o app e atualiza `external.json`; com `force`, remove item adotado ou exigido por outro selecionado.

Recusas e limites. Sem `confirm` e com `dry_run: false`, a tool recusa antes de rodar o CLI, com exit code 2. Id desconhecido devolve 2, item exigido por outro selecionado ou adotado pelo `init` só sai com `force: true`, e a recusa sem força devolve 10 (`PARTIAL`). App que não foi instalado com `external` devolve 6 (`NOT_FOUND`).

Exemplos.

- **Ver o plano:** `remove { "items": ["node"], "dry_run": true }` devolve os passos sem remover nada.
- **Remover depois da confirmação:** `remove { "items": ["node"], "dry_run": false, "confirm": true }` remove e devolve `results`.
- **Testar a recusa:** `remove { "items": ["node"], "dry_run": false }` devolve exit 2 e a mensagem para pedir confirmação primeiro.

- **Incluir pacote do sistema:** `remove { "items": ["docker"], "dry_run": true, "include_native": true }` mostra o plano com o pacote nativo.
- **Remover app externo:** `remove { "items": ["terraform"], "external": true, "dry_run": false, "confirm": true }` remove o app e atualiza `external.json`.

Tool `update`

Finalidade. Atualiza os itens selecionados cuja versão fixada mudou no inventário deste binário.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`dry_run`	boolean	não	`true`	`true` devolve o plano; `false` atualiza

Saída. Envelope com `data.steps` no plano ou `data.results` na execução real, mais `data.outdated`, com `id`, `from` e `to` de cada item.

Efeitos e permissões. Com `dry_run: true`, nada muda. Com `false`, reinstala os itens desatualizados, grava o log e atualiza `selection.json`.

Recusas e limites. Sem `init` devolve 3 (`CONFIG`), falha parcial devolve 10, sistema sem suporte devolve 11 e `sudo` sem senha devolve 13. A tool não expõe `--external`.

Exemplos.

- **Planejar a atualização:** `update { "dry_run": true }` devolve `outdated` e os passos.
- **Aplicar:** `update { "dry_run": false }` reinstala o que mudou e devolve `results`.
- **Confirmar que está em dia:** `update {}` devolve `outdated: []` quando nada mudou.
- **Depois de um upgrade:** `update { "dry_run": true }` mostra o que o binário novo passou a fixar.
- **Ver uma falha parcial:** `update { "dry_run": false }` devolve exit 10 com os passos que falharam.

Tool `health`

Finalidade. Devolve só os achados do SetupVibe que o `doctor` mostra: pré-requisitos, itens, shell, restos do SetupVibe 1 e containers expostos.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `ok` e `findings`, cada achado com `id`, `level` (`ok`, `warn` ou `fail`), `text`, `fix` quando existe, `safe` quando o `doctor --fix` aplica e `params` com os valores usados no texto.

Efeitos e permissões. Só leitura. Confere arquivos, PATH e containers; não aplica correção e não grava nada.

Recusas e limites. Não tem recusa própria e não marca `isError` por achado: `ok: false` aparece quando existe aviso ou problema.

Exemplos.

- **Ver o que falta antes do init:** `health {}` devolve o achado `init` com os pré-requisitos pendentes.
- **Depois de uma falha:** `health {}` mostra os itens que falham na verificação e o comando que resolve.
- **Achar o SetupVibe 1:** `health {}` devolve o achado `v1` com os traços encontrados.
- **Conferir containers expostos:** `health {}` devolve `docker-exposed` quando um container publica porta em todas as interfaces.
- **Comparar com o doctor:** `health {}` traz só os achados do SetupVibe; para o runtime e o terminal, use a tool `doctor`.

Tool `doctor`

Finalidade. Roda o `doctor` do CLI: checagens de runtime, terminal e configuração mais os achados do SetupVibe, cada um com o comando que resolve.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. Envelope com `data.ok`, `data.runtime` (cada check com `name`, `status` e `detail`) e `data.findings`, como na tool `health`.

Efeitos e permissões. Só leitura. Não aplica correção; para isso use `setupvibe doctor --fix` no terminal.

Recusas e limites. Com um achado de nível fail, o CLI devolve exit code 3 (`CONFIG`) e a tool marca `isError`, mesmo com os dados no envelope. Rodando pelo MCP, os checks de stdin e stdout avisam que não são um TTY, porque o subprocesso usa pipes.

Exemplos.

- **Diagnóstico completo:** `doctor {}` devolve runtime e achados na mesma chamada.
- **Antes de instalar:** `doctor {}` mostra init pendente e itens que falham, antes de um `install`.
- **Depois de uma falha:** `doctor {}` devolve o achado com o comando que resolve e `isError` quando há fail.
- **Conferir a configuração:** `doctor {}` mostra settings ausente, master key e origem do cofre.
- **Separar runtime de achados:** use `doctor {}` para o todo e `health {}` só para os achados do SetupVibe.

Tool `report`

Finalidade. Monta o relatório da máquina em Markdown para anexar numa issue, sem segredos.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. Envelope com `data.markdown`, o mesmo texto de `setupvibe report`: binário, sistema, init, doctor, itens selecionados, apps externos, última execução e os eventos finais do log.

Efeitos e permissões. Só leitura. Não grava arquivo; para gerar um, use `setupvibe report --output arquivo.md`.

Recusas e limites. Nenhuma recusa própria. O CLI devolve 0 e a tool não marca `isError`.

Exemplos.

- **Abrir uma issue:** `report {}` devolve o Markdown para colar no formulário de bug.
- **Depois de uma falha:** `report {}` inclui os passos que falharam e o final do log.
- **Pedir apoio:** `report {}` traz sistema, versão e achados do doctor sem pedir comando à pessoa.
- **Comparar duas máquinas:** `report {}` em cada uma mostra diferenças de sistema e seleção.
- **Conferir o mascaramento:** `report {}` não traz valor de credencial nem despejo de ambiente.

Tool `init`

Finalidade. Confere os requisitos e instala os pré-requisitos do SetupVibe (dependências de build e mise), com plano por padrão.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`dry_run`	boolean	não	`true`	`true` devolve o plano e as checagens; `false` instala

Saída. Envelope com `data.checks` e `data.steps` no plano, ou `data.results` na execução real. O sucesso grava o registro do `init` em `config.json`.

Efeitos e permissões. Com `dry_run: true`, nada muda, mas as checagens de rede e relógio rodam. Com `false`, instala os itens `initOnly` e grava `config.json`.

Recusas e limites. Root sem `--user` devolve 12, a outra edição configurada devolve 2, disco curto devolve 14, rede ou relógio fora devolve 5, sistema sem suporte devolve 11, `sudo` sem senha devolve 13 e falha parcial devolve 10. A tool não expõe `--skip-network-check`.

Exemplos.

- **Conferir antes de instalar:** `init { "dry_run": true }` devolve as checagens e o plano dos pré-requisitos.
- **Instalar os pré-requisitos:** `init { "dry_run": false }` instala build deps e mise e grava o `init`.
- **Ver o bloqueio de disco:** `init { "dry_run": true }` devolve o check `disk` com `fail` e o exit code 14.
- **Conferir a edição:** `init { "dry_run": true }` mostra o check `edition` quando a outra edição já está configurada.
- **Depois de corrigir a rede:** `init { "dry_run": false }` repete as checagens e segue para o plano.

Tool `migrate`

Finalidade. Move uma máquina preparada pelo SetupVibe 1 para o SetupVibe 2, com plano por padrão e confirmação obrigatória na execução real.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`dry_run`	boolean	não	`true`	`true` devolve o plano, as linhas movidas e as duplicidades mantidas
`confirm`	boolean	não	`false`	A pessoa confirmou a migração; obrigatório com `dry_run: false`
`remove_legacy`	array de enum	não	nenhum	`go`, `rbenv`, `npm-global`, `nodesource` ou `all`, para a execução real

Saída. Envelope com `data.v1`, `data.moved` (linhas por arquivo), `data.removeLegacy`, `data.keptLegacy`, `data.steps` no plano, ou `data.results` na execução real.

Efeitos e permissões. Com `dry_run: true`, nada muda. Com `false` e `confirm: true`, copia os arquivos para `backups/`, move as linhas pessoais para os `.local`, grava os blocos, pode remover duplicidades e grava `migratedFromV1` em `config.json`.

Recusas e limites. Sem `confirm` e com `dry_run: false`, a tool recusa com exit code 2. Sistema sem suporte devolve 11, `sudo` sem senha devolve 13 e falha parcial devolve 10. Linuxbrew nunca é removido, e o plano mostra o comando de desinstalação.

Exemplos.

- **Ler o plano:** `migrate { "dry_run": true }` devolve os traços do SetupVibe 1 e as linhas que vão para os `.local`.
- **Conferir os segredos mascarados:** `migrate { "dry_run": true }` mostra `TOKEN` e `KEY` como `****` nas linhas movidas.
- **Migrar e remover duplicidades:** `migrate { "dry_run": false, "confirm": true, "remove_legacy": ["go", "rbenv"] }`.
- **Manter as duplicidades:** `migrate { "dry_run": false, "confirm": true }` migra sem remover nada.
- **Máquina sem vestígios:** `migrate { "dry_run": true }` devolve `found: false` e a mensagem de que não há o que migrar.

Tool `logs`

Finalidade. Lista as execuções gravadas em `~/.setupvibe/logs` e devolve os eventos de uma delas.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`run`	string	não	a execução mais nova	Nome do arquivo de log devolvido em `runs`

Saída. `ok`, `runs` (mais nova primeiro), `run` (o escolhido ou `null`), `events` com `time`, `item`, `action`, `status` e `message`, e `failed`, a contagem de eventos com falha.

Efeitos e permissões. Só leitura. Não apaga nem escreve log.

Recusas e limites. Um nome fora de `runs` devolve `{ ok: false, error, runs }` e marca `isError`, sem rodar o CLI.

Exemplos.

- **Ver a última execução:** `logs {}` devolve os eventos da execução mais nova.
- **Abrir uma execução específica:** `logs { "run": "2026-11-13T...-auto.log" }` devolve os eventos daquele arquivo.

- **Contar falhas:** `logs {}` devolve `failed`, a contagem de eventos com status fail.
- **Máquina sem execução:** `logs {}` devolve `runs: []` e `run: null`.
- **Investigar um install:** `logs {}` mostra o evento do item com a mensagem de erro abaixo do passo.

Tool `tailscale`

Finalidade. Lê o cliente local do Tailscale: estado desta máquina, tailnet, máquinas com o tipo de conexão, serviços publicados e avisos de saúde.

Entrada.

Nenhum. A tool não recebe argumento.

Saída. `ok: true`, `installed`, `state`, `version`, `authUrl` quando precisa de login, `tailnet`, `self`, `health`, `peers`, `exitNodeInUse`, `serve` e `error` quando a leitura falha.

Efeitos e permissões. Só leitura. Chama o cliente local (`tailscale status` e `serve`), sem a API da Tailscale, e pode levar alguns segundos.

Recusas e limites. Nenhuma recusa própria: sem o cliente instalado, devolve `installed: false`. O teste de rede (`netcheck`) fica no módulo Tailscale da TUI.

Exemplos.

- **Ver se o cliente existe:** `tailscale {}` devolve `installed: false` quando o Tailscale não está na máquina.
- **Ler o estado:** `tailscale {}` devolve `state: "Running"` e o IP da máquina em `self.ips`.
- **Listar as máquinas:** `tailscale {}` devolve `peers` com `online`, `relay` e `lastSeen` de cada uma.
- **Conferir serviços:** `tailscale {}` devolve `serve` com o endereço publicado e o destino.
- **Ver o exit node:** `tailscale {}` devolve `exitNodeInUse` quando a máquina usa um exit node.

Tool `machine`

Finalidade. Devolve uma leitura da máquina: host, kernel, tempo ligado, CPU, carga, memória, swap, discos, rede e os processos que mais usam CPU.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`processes`	inteiro	não	`8`	Quantos processos devolver, de 0 a 50

Saída. `ok: true`, `info` (hostname, kernel, arquitetura, modelo e núcleos de CPU, carga, memória, swap, tempo ligado, discos e interfaces) e `live` (cpu, perCore, memUsed, memTotal, swapUsed, swapTotal, load, net, disks, temps, processes, processCount e uptimeSec).

Efeitos e permissões. Só leitura. Espera cerca de um segundo entre duas leituras, porque CPU e rede são calculadas por diferença.

Recusas e limites. Nenhuma recusa própria. Um valor de `processes` fora de 0 a 50 é recusado pelo schema.

Exemplos.

- **Fotografia da máquina:** `machine {}` devolve `info` e `live` com os oito processos de maior CPU.
- **Ver mais processos:** `machine { "processes": 20 }` devolve vinte processos.
- **Só o resumo:** `machine { "processes": 0 }` devolve `live.processes: []` e mantém CPU, memória e discos.
- **Conferir memória e swap:** `machine {}` devolve `live.memUsed`, `memTotal`, `swapUsed` e `swapTotal`.
- **Conferir disco:** `machine {}` devolve `info.disks` com uso, espaço livre e percentual por ponto de montagem.

Resources

Resources são leituras passivas, sem argumento além do URI. O esquema `<bin>://` evita colisão com outros servidores no mesmo cliente.

Resource `setupvibe://inventory`

Finalidade. Entrega o inventário da edição como JSON: o mesmo registro que o binário usa para instalar, verificar e listar.

Entrada.

Nenhum parâmetro além do URI.

Saída. `schemaVersion`, `edition`, `systems`, `categories`, `repositories`, `profiles` e `items`; cada item traz `id`, `name`, `description`, `version`, `versionPolicy`, `install`, `verify`, `requires`, `plugins` e `flags`, quando existirem.

Efeitos e permissões. Só leitura. O conteúdo vem embutido no binário e não depende de rede nem de arquivo local.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Descobrir ids:** leia `setupvibe://inventory` antes de montar `install` ou `remove`.

- **Conferir a versão fixada:** procure o item e leia `version` e `versionPolicy`.
- **Ver os perfis:** `profiles` traz os perfis da edição, como `essencial`, com a lista de itens de cada um.
- **Conferir o método:** `install` mostra o método por sistema, como `mise`, `native` ou `installer`.
- **Auditar um item:** `verify` mostra o comando e o valor esperado usados por `check`.

Resource `setupvibe://selection`

Finalidade. Entrega o `selection.json` desta máquina: os itens escolhidos, a versão instalada e o status do último check.

Entrada.

Nenhum parâmetro além do URI.

Saída. `schemaVersion`, `profile`, `items` com `id`, `version`, `status` (`ok` ou `fail`) e `adopted` quando o `init` encontrou o item já instalado. Sem o arquivo, devolve `items: []`.

Efeitos e permissões. Só leitura. Não cria nem altera a seleção.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Ver o que foi escolhido:** leia `setupvibe://selection` antes de sugerir `update`.
- **Achar item adotado:** `adopted: true` indica item que o `init` encontrou na máquina.
- **Ver falhas do check:** `status: "fail"` indica o item que precisa de `install --force` no CLI.
- **Conferir a versão instalada:** compare `version` com o inventário para achar o que está fora.
- **Máquina nova:** o resource devolve `items: []` antes do primeiro `init`.

Resource `setupvibe://about`

Finalidade. Entrega a identidade e o runtime do binário em JSON, igual à tool `about`.

Entrada.

Nenhum parâmetro além do URI.

Saída. Os mesmos campos da tool `about`: `bin`, `title`, `version`, `edition`, `runtime`, `platform`, `system`, `pid`, `dataDir` e `credits`.

Efeitos e permissões. Só leitura. Não grava nem consulta a rede.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Conferir a versão:** leia `setupvibe://about` antes de citar um comportamento novo.

- **Conferir a edição:** `bin` e `title` mostram se a sessão é desktop ou servidor.
- **Conferir a pasta:** `dataDir` mostra a pasta de dados efetiva.
- **Coletar créditos:** `credits` traz nome, papel e URL de cada projeto.
- **Conferir a plataforma:** `platform` mostra sistema e arquitetura do processo.

Resource `setupvibe://exit-codes`

Finalidade. Entrega a tabela de exit codes em JSON, igual à tool `exit_codes`.

Entrada.

Nenhum parâmetro além do URI.

Saída. `codes`, com `code`, `name` e `description` de cada código.

Efeitos e permissões. Só leitura.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Montar o tratamento:** leia `setupvibe://exit-codes` para mapear os codes no agente.
- **Explicar o 10:** confirme `PARTIAL` e a mensagem antes de responder à pessoa.
- **Explicar o 13:** confirme `SUDO_PASSWORD` e a regra sem senha.
- **Explicar o 30:** confirme `LOCKED` e o lock de execução.
- **Explicar o 130:** confirme `INTERRUPTED` e que nada foi concluído.

Resource `setupvibe://skill`

Finalidade. Entrega toda a documentação embutida em Markdown, como `setupvibe --skill` sem tópico.

Entrada.

Nenhum parâmetro além do URI.

Saída. Markdown com o README, o CLI, a TUI, o MCP, a configuração, os exit codes, o troubleshooting, a migração e o inventário.

Efeitos e permissões. Só leitura. A resposta é longa e consome contexto.

Recusas e limites. Nenhuma recusa própria.

Exemplos.

- **Ler tudo antes de operar:** leia `setupvibe://skill` numa sessão sem conhecimento prévio.
- **Conferir a documentação de uma versão:** o texto vem do binário em execução, não do repositório.
- **Preparar um agente novo:** entregue o Markdown como instrução inicial.

- **Auditar a doc embutida:** compare o Markdown com os arquivos de `docs/`.
- **Evitar quando o contexto é curto:** prefira `setupvibe://skill/cli` para uma tarefa de CLI.

Resource `setupvibe://skill/{topic}`

Finalidade. Entrega uma seção da documentação embutida.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`topic`	string no URI	sim	nenhum	`cli`, `tui`, `mcp`, `config`, `exit-codes`, `troubleshooting`, `migration`, `inventory` ou `reference`

Saída. Markdown da seção pedida. O template lista um resource por tópico.

Efeitos e permissões. Só leitura. O idioma segue `SETUPVIBE_LANG`.

Recusas e limites. Um tópico fora da lista devolve erro de leitura do resource, com a lista de tópicos aceitos.

Exemplos.

- **Aprender o CLI:** leia `setupvibe://skill/cli` antes de sugerir um command.
- **Conferir teclas:** leia `setupvibe://skill/tui` para orientar alguém na TUI.
- **Resolver uma falha:** leia `setupvibe://skill/troubleshooting`.
- **Migrar uma máquina:** leia `setupvibe://skill/migration` antes de chamar `migrate`.
- **Achar itens:** leia `setupvibe://skill/inventory` para uma tabela por categoria.

Prompts

Prompt `operate`

Finalidade. Monta a instrução inicial de um agente para operar o binário: ler a documentação, rodar o CLI com `--json` e tratar o exit code.

Entrada.

CAMPO	TIPO	OBRIGATÓRIO	PADRÃO	EFEITO
`task`	string	sim	nenhum	O que a pessoa quer fazer com o binário

Saída. Uma mensagem de usuário em `messages`, com o nome e a versão do binário, a ordem de leitura e a tarefa recebida.

Efeitos e permissões. Nenhum. O prompt não executa comando; ele prepara a conversa.

Recusas e limites. Sem `task`, o schema recusa a chamada.

Exemplos.

- **Começar uma sessão:** `operate { "task": "instalar o ambiente de desenvolvimento" }` devolve a instrução completa.
- **Diagnosticar:** `operate { "task": "descobrir por que o node está fora da versão" }` orienta a ler o CLI e rodar `check`.
- **Migrar:** `operate { "task": "migrar esta máquina do SetupVibe 1" }` orienta a ler `migration` antes de `migrate`.
- **Provisionar servidor:** `operate { "task": "preparar um VPS com Docker Swarm" }` orienta o fluxo do servidor.
- **Revisar configuração:** `operate { "task": "conferir o settings.json e o cofre" }` orienta a ler `config` e usar `config_list`.

Configurar nos agentes

`setupvibe mcp install` encontra os agentes no `PATH`, em `~/.local/bin` e nos shims do mise, e registra o servidor com o caminho absoluto do binário. No OpenCode, o arquivo é lido, o bloco `mcp` é mesclado e a cópia anterior fica em `opencode.json.setupvibe.bak`.

```
setupvibe mcp install --dry-run
setupvibe mcp install
setupvibe mcp install --json
```

Agentes ausentes aparecem como `not installed` no stderr e em `data.skipped`. A execução é idempotente: `mcp get` evita registrar de novo.

Como adicionar uma tool

1. Crie o arquivo em `shared/core/mcp/tools/<nome>.ts` (tool do framework) ou acrescente a tool em `shared/mcp/tools.ts` (operação do SetupVibe).

2. Declare `inputSchema` com Zod e descreva cada campo com `.describe()`; use `.default()` para o padrão que aparece no schema.
3. Devolva `content` em texto e `structuredContent` com o mesmo objeto. Marque `isError` quando a chamada falhar.
4. Registre no `catalog` (`tools`, `resources` ou `prompts`) para `mcp --list` enxergar o item.
5. Registre a função em `registerAll` de `shared/core/mcp/server.ts`.
6. Documente a tool neste arquivo, com as cinco situações de uso, e rode `bun test`.

Nomes de tools usam `snake_case`. A description diz quando usar e o que a tool devolve. Tool que grava precisa declarar isso e nunca ecoar segredo.

Testar

```
npx @modelcontextprotocol/inspector setupvibe mcp
setupvibe mcp --list --json
bun test shared/tests/mcp-tools.test.ts shared/tests/docs-mcp-reference.test.ts
```

O Inspector abre uma página local para chamar tools, ler resources e acompanhar as mensagens JSON-RPC. O teste de documentação compara `mcp --list` com este arquivo nas duas línguas e nas duas edições.